

UNIVERZA V LJUBLJANI
FAKULTETA ZA RAČUNALNIŠTVO IN INFORMATIKO

Žan Pižmoht

**Sistem za zagotavljanje zaupanja v
dobavni verigi zdravil z uporabo
verige blokov**

MAGISTRSKO DELO
MAGISTRSKI ŠTUDIJSKI PROGRAM DRUGE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

MENTOR: prof. dr. Vlado Stankovski

SOMENTOR: doc. dr. Petar Kochovski

Ljubljana, 2025

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati zaključnega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.



Izvorna koda zaključnega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco Apache License, različica 2.0 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani <https://www.apache.org/licenses/LICENSE-2.0>.

ZAHVALA

Rad bi se iskreno zahvalil mentorju prof. dr. Vladu Stankovskemu za njegovo strokovno vodstvo in hitro odzivnost skozi celoten proces nastajanja te magistrske naloge. Zahvalil bi se tudi somentorju doc. dr. Petru Kochovskemu za koristne napotke pri implementaciji rešitve in pripravi tehničnih delov naloge. Nazadnje bi se rad zahvalil tudi svoji družini in prijateljem za njihovo podporo in spodbudo skozi celoten študij.

Žan Pižmoht, 2025

*"The best way to find out if you can trust
somebody is to trust them."*

— Ernest Hemingway

Kazalo

Povzetek

Abstract

1	Uvod	1
2	Pregled literature in ozadje	5
2.1	Zaupanje	5
2.2	Upravljanje zaupanja v razpršenih sistemih	10
2.3	Farmacevtske dobavne verige	14
2.4	Tehnologija veriženja blokov	17
2.5	Semantični splet in ontologije	22
2.6	Analiza vrzeli in prispevek te naloge	23
3	Implementacija sistema	27
3.1	Uporabljene tehnologije	28
3.2	Arhitektura sistema	32
3.3	Tok podatkov v sistemu	42
4	Evalvacija	57
4.1	Predstavitev scenarijev delovanja	58
4.2	Varnostna analiza sistema	78
4.3	Razprava	92
4.4	Pot do produkcijskega sistema	92
4.5	Operativni in regulativni vidiki	94

KAZALO

4.6	Širši potencial in vizija uporabe sistema	95
5	Zaključek in nadaljnje delo	97
5.1	Povzetek dela in prispevki	97
5.2	Nadaljnje delo	98

Seznam uporabljenih kratic

kratica	angleško	slovensko
API	Application Programming Interface	programski vmesnik aplikacije
DID	Decentralized Identifier	decentralizirani identifikator
DoS	Denial of Service	zavrnitev storitve
DON	Decentralized Oracle Network	decentralizirano omrežje orakljev
DSA	Digital Signature Algorithm	algoritem za digitalno podpisovanje sporočil
DSCSA	Drug Supply Chain Act	zakon o sledenju zdravil, sprejet v ZDA
ECDSA	Elliptic Curve Digital Signature Algorithm	algoritem eliptičnih krivulj za digitalno podpisovanje
EMA	European Medicines Agency	Evropska agencija za zdravila
EMVS	European Medicines Verification System	Evropski sistem za sledenje zdravil
EWMA	Exponentially Weighted Moving Average	eksponentno uteženo drseče povprečje
EVM	Ethereum Virtual Machine	virtualni stroj Ethereum
GDP	Good Distribution Practice	dobra distribucijska praksa
GDPR	General Data Protection Regulation	Splošna uredba o varstvu podatkov
GMP	Good Manufacturing Practice	dobra proizvodna praksa
HSM	Hardware Security Module	strojni varnostni modul

kratica	angleško	slovensko
HTTP	Hypertext Transfer Protocol	protokol za prenos hiperteksta
HTTPS	Hypertext Transfer Protocol Secure	varni protokol za prenos hiperteksta
IoT	Internet of Things	internet stvari
JSON	JavaScript Object Notation	zapis JSON
JSON-LD	JSON for Linking Data	JSON za povezane podatke
OWASP	Open Worldwide Application Security Project	projekt za varnost aplikacij OWASP
OWL	Web Ontology Language	jezik spletnih ontologij OWL
PII	Personally Identifiable Informa- tion	osebni identifikacijski podatki
RPC	Remote Procedure Call	oddaljeni klic postopkov
RDF	Resource Description Frame- work	ogrodje za opis virov RDF
TMS	Transport Management System	sistem za upravljanje trans- porta
URI	Uniform Resource Identifier	enotni identifikator vira
VC	Verifiable Credential	preverljiva poverilnica

Seznam uporabljenih pojmov

pojem	razlaga
Akter	Udeleženec sistema, ki sodeluje v procesih izmenjave podatkov in sprejemanja odločitev. Akter lahko nastopa kot ocenjevalec (zahteva oceno) ali kot ocenjevani subjekt.
Entiteta	Splošen pojem za subjekt ali objekt, ki nastopa v obravnavanem okolju. V kontekstu naloge so entitete praviloma akterji, izraz pa se uporablja tudi kadar govorimo splošno (npr. o udeležencih ali objektih v sistemu) brez vezave na konkretno vlogo.
Časovni žig	Zaporedje znakov, ki zagotavlja podpis dokumenta z veljavnim digitalnim potrdilom v določenem časovnem trenutku s povezovanjem datuma, časa podpisa in podatkov na kriptografsko varen način.
Digitalni podpis	Matematična shema za preverjanje pristnosti digitalnih dokumentov. Vsebuje tri algoritme: generiranje javnih in zasebnih ključev, podpisni algoritem in algoritem preverjanja podpisa.
Digitalno potrdilo	Strukturiran in kriptografsko podpisan zapis, ki javni ključ poveže z identiteto določenega subjekta izdan s strani zaupanja vrednega izdajatelja. Prejemniku omogoča preverjanje, komu pripada dani javni ključ.

pojem	razlaga
Javni ključ	Del para javnega in zasebnega ključa v asimetrični kriptografiji, namenjen deljenju z drugimi, ki ga uporabljajo za preverjanje digitalnih podpisov imetnika ali za šifriranje sporočil, ki jih lahko dešifrira le ustrezni zasebni ključ.
Porazdeljeni sistem	Sistem, sestavljen iz več medsebojno povezanih komponent (vozlišč), ki komunicirajo prek omrežja in skupaj izvajajo funkcije sistema.
Proof of Stake	Konsenzni mehanizem, kjer vpliv udeleženca na predlaganje in potrjevanje blokov temelji na količini vloženih (z zaklenitvijo zavarovanih) sredstev v omrežju, pri čemer ekonomski vložek deluje kot spodbuda za pošteno vedenje in varovalka pred napadi.
Proof of Work	Konsenzni mehanizem verig blokov, pri katerem mora vozlišče za predlaganje novega bloka izvesti računsko zahteven izračun. Otežuje ponarejanje blokov in napade z ustvarjanjem velikega števila lažnih blokov.
Veriga blokov	Razpršena, kriptografsko povezana knjižnica digitalnih zapisov (blokov), ki vsebujejo podatke o transakcijah in se povezujejo z zgoščenimi povzetki prejšnjih blokov.
Zasebni ključ	Tajni del para javnega in zasebnega ključa, ki ga pozna le imetnik in se uporablja za ustvarjanje digitalnih podpisov ali šifriranje.
Zgostitev	Rezultat zgoščevalne funkcije.
Zgoščevalna funkcija	Matematična funkcija, ki vzame vhodne podatke poljubne velikosti in jih preslika v zgoščeno vrednost fiksne velikosti, pri čemer že majhna sprememba vhodnih podatkov povzroči veliko spremembo zgoščene vrednosti. Pomembna lastnost zgoščevalne funkcije je, da iz izhodne zgoščene vrednosti v doglednem času ne moremo izračunati vhodnih podatkov.

Povzetek

Naslov: Sistem za zagotavljanje zaupanja v dobavni verigi zdravil z uporabo verige blokov

Magistrska naloga obravnava upravljanje zaupanja v farmacevtski dobavni verigi, kjer več neodvisnih akterjev izmenjuje podatke, certifikate in druge vire informacij. Predlagamo arhitekturo, v kateri ontologija, preverljive poverilnice in pametna pogodba na verigi blokov skupaj tvorijo graf zaupanja. Ključni element je decentralizirano omrežje orakljev, pri katerem ima vsak orakelj lastna pravila zaupanja. Oraklji na podlagi različnih virov podatkov izračunajo binarno odločitev o zaupanju in številčno oceno zaupanja ter rezultate prek agregatorja zapišejo v pametno pogodbo na omrežju Ethereum. Prototip je implementiran z uporabo vsebnikov Docker in več orakljev z lastnimi pravili. V evalvaciji sistem testiramo na ponovljivih scenarijih in potrdimo, da se dinamično in pravilno odziva na spremembe vhodnih podatkov. Predstavimo tudi varnostno analizo po modelu STRIDE, v kateri raziščemo glavne grožnje za pametno pogodbo in omrežje orakljev, ter opišemo potrebne nadgradnje za prehod v produkcijsko okolje in širšo uporabo v drugih domenah.

Ključne besede

upravljanje zaupanja, tehnologija veriženja blokov, dobavna veriga zdravil, ontologija, decentralizirano omrežje orakljev

Abstract

Title: Trust System in Pharmaceutical Supply Chain Using Blockchain

The thesis addresses trust management in the pharmaceutical supply chain, where multiple independent actors exchange data, certificates and other information sources. It proposes an architecture in which an ontology, verifiable credentials and a smart contract on a blockchain jointly form a trust graph. The key element is a decentralized oracle network, where each oracle applies its own trust rules. Oracles consume heterogeneous data sources to compute a binary trust decision and a numerical trust score, and record these results in a smart contract on an Ethereum test network. The prototype is implemented using Docker containers and multiple oracles with different policies, and is evaluated on repeatable scenarios that show how trust scores change as input data deteriorate or improve. Finally, the thesis presents a STRIDE-based security analysis of the smart contract and oracle network and outlines the enhancements required for deployment in a production environment and for broader use in other domains.

Keywords

trust management, blockchain, pharmaceutical supply chain, ontology, decentralized oracle network

Poglavje 1

Uvod

Farmacevtske dobavne verige sodijo med najbolj regulirana in podatkovno zahtevna okolja v sodobni industriji. Zdravila, cepiva in drugi farmacevtski proizvodi potujejo med številnimi akterji, kot so proizvajalci, distributerji, transporterji, regulatorji in zdravstvene ustanove. Od vseh se pričakuje visoka stopnja zanesljivosti in skladnosti, vendar so tovrstne verige kljub temu izpostavljene številnim tveganjem. Pogosti so primeri ponarejenih zdravil, nepravilno ravnanje z izdelki, nepopolna poročila ter pomanjkanje transparentnosti [47]. V veliki meri ti problemi izhajajo iz nezmožnosti pravočasnega, jasnega in doslednega preverjanja zaupanja v posamezne akterje [26].

Obstoječi sistemi za upravljanje zaupanja v farmacevtskih dobavnih verigah pogosto temeljijo na delno centraliziranih podatkovnih virih, ročnih revizijah ali statičnih ocenah [12, 11, 43]. V literaturi so bili predlagani različni pristopi za izboljšanje teh mehanizmov, vključno z reputacijskimi modeli zaupanja [24], semantičnim modeliranjem [4], uporabo verige blokov za zagotavljanje sledljivosti [28] ter integracijo podatkov iz senzorjev in zunanjih virov [42]. Kljub temu večina obstoječih rešitev obravnava posamezne vidike problema ločeno in ne ponuja splošne zasnove, ki bi te elemente povezala v enoten, razširljiv in razložljiv okvir za dinamično ocenjevanje zaupanja.

V magistrski nalogi se zato osredotočamo na vprašanje, kako v farmacevt-

ski dobavni verigi oblikovati model in mehanizem, ki omogoča transparentno, preverljivo in razširljivo upravljanje zaupanja med akterji, ne da bi bilo treba razkrivati občutljive operativne ali poslovne podatke.

Cilj naloge je razviti konceptualni in arhitekturni okvir ter na njegovi podlagi izdelati prototip sistema, ki omogoča:

- formalno predstavitev akterjev, njihovih lastnosti, politik in dokazil,
- uporabo več neodvisnih pogledov na zaupanje in različnih kriterijev ocenjevanja,
- zbiranje, preverjanje in usklajevanje ocen iz več virov,
- zapis končnih odločitev v nespremenljivo in preverljivo podatkovno strukturo.

Hipoteza naloge je, da je mogoče tak prototip zasnovati tako, da demonstrira izvedljivost decentraliziranega sistema za izračun zaupanja ter hkrati ohrani lastnosti, ki so ključne za regulativno okolje: razložljivost postopkov, razširljivost arhitekture, odpornost na posamične okvare ter minimizacijo razkritja podatkov.

Predlagani sistem temelji na kombinaciji semantičnega modeliranja, decentralizirane infrastrukture in preverljivih dokazil. Akterji v dobavni verigi so formalno opisani z uporabo ontologije, ki omogoča predstavitev njihovih lastnosti. Zaupanje se ocenjuje na podlagi več neodvisnih virov z uporabo orakljev, ki izračune izvajajo zunaj verige blokov, končni rezultati pa se zapisujejo v verigo blokov.

V nadaljevanju naloge najprej predstavimo teoretično ozadje upravljanja zaupanja v razpršenih sistemih ter obstoječe pristope, uporabljene v farmacevtskih dobavnih verigah. Nato opišemo konceptualni model zaupanja in arhitekturo predlaganega sistema, vključno z vlogo ontologije, orakljev in verige blokov. Sledi poglavje implementacije, kjer podrobno predstavimo zasnovo in delovanje prototipa. V poglavju evalvacije prikažemo eksperimentalne scenarije, ki demonstrirajo praktično delovanje sistema, ter analiziramo

njegove varnostne lastnosti z uporabo modela groženj. V zaključku povzamemo ključne ugotovitve in predlagamo možnosti za nadaljnji razvoj rešitve.

Poglavje 2

Pregled literature in ozadje

To poglavje raziskuje koncepte, na katerih temelji sistem, razvit v okviru magistrske naloge. Začnemo z razumevanjem zaupanja in njegovih lastnosti ter s pregledom pristopov, ki se v razpršenih sistemih uporabljajo za njegovo modeliranje in ocenjevanje. V nadaljevanju se posvetimo farmacevtskim dobavnim verigam, kjer izpostavimo ključne težave, ki se pojavljajo pri zagotavljanju sledljivosti, varnosti in sodelovanja med akterji. Tu omenimo obstoječe rešitve, ki se že uporabljajo v praksi in predstavimo probleme, ki jih te rešitve imajo. Sledi pregled tehnologije veriženja blokov, pametnih pogodb in orakljev, ki danes predstavljajo pomemben del modernih decentraliziranih arhitektur. Poglavje zaključimo s predstavitvijo semantičnega spleta in metod, ki so potrebne za formalno opisovanje akterjev in njihovih lastnosti. Ta poglavja skupaj strnemo v množico, v kateri lahko prepoznamo vrzeli obstoječih pristopov in pripravimo teren za rešitev, ki jo predstavimo v nadaljevanju naloge.

2.1 Zaupanje

Pojem zaupanja je temeljni gradnik pri zasnovi našega sistema za farmacevtsko dobavno verigo. Zaupanje je kompleksen, večdimenzionalen in interdisciplinaren koncept, ki ga obravnava različne vede. Obravnava se v sociologiji,

psihologiji, ekonomiji, računalništvu in še ostalih vedah. Prav zaradi te raznolikosti v literaturi ne obstaja enotna definicija, temveč več pristopov, ki poudarjajo različne vidike [49, 21].

2.1.1 Definicije zaupanja

V družboslovnem kontekstu je zaupanje pogosto opredeljeno kot stanje pozitivnih pričakovanj glede dejanj druge osebe v okoliščinah, kjer obstaja določena stopnja tveganja [6]. Gambetta [20] zaupanje definira kot subjektivno verjetnost, da bo akter opravil določeno dejanje, še preden je to mogoče preveriti. Mayer, Davis in Schoorman [32] pa ga opredelijo kot pripravljenost ene stranke, da se izpostavi ranljivosti glede na dejanja druge stranke, ob pričakovanju, da bo ta delovala v skladu s pričakovanji.

V digitalnem okolju se koncept zaupanja prenaša iz družbenega v tehnični kontekst. Denning [16] trdi, da je zaupanje v sistem lastnost, ki jo je mogoče formalno določiti in jo modelirati. Sistem je torej zaupanja vreden, če mu njegovi uporabniki zaupajo glede na opaženo skladnost z vnaprej določenimi standardi. Grandison in Sloman [21] zaupanje obravnavata kot kvalificirano prepričanje zaupnika o kompetentnosti, integriteti, varnosti in zanesljivosti zaupanja vrednega subjekta.

2.1.2 Lastnosti zaupanja

Zaupanje je po svoji naravi večdimenzionalen pojem, ki ga ni mogoče zajeti z eno samo definicijo. V literaturi se ponavlja, da zaupanje vključuje čustvene in vedenjske komponente, ki se med seboj prepletajo in vplivajo na odločanje posameznika ali sistema [34, 25]. V družbenem kontekstu gre za pripravljenost posameznika, da se izpostavi tveganju na podlagi prepričanja, da bo druga stran delovala predvidljivo in skladno s pričakovanji. V informacijskih sistemih pa to pomeni sposobnost sistema, da sprejema odločitve o sodelovanju na podlagi preteklih izkušenj in ocenjene zanesljivosti entitet. V tej nalogi izraz *entiteta* uporabljamo kot splošni pojem za udeležene su-

bjekte v sistemu, pri čemer so v obravnavanem kontekstu to praviloma *akterji* dobavne verige.

Zaupanje obstaja le v razmerju med dvema ali več entitetami, kjer ena zaupa drugi v določeni situaciji ali kontekstu. Takšno razmerje je dinamično, saj se lahko stopnja zaupanja s časom povečuje ali zmanjšuje. Poleg tega je zaupanje asimetrično: dejstvo, da entiteta A zaupa entiteti B, še ne pomeni, da bo tudi B zaupala A [32].

Ena od osrednjih značilnosti zaupanja je njegova povezanost s tveganjem in negotovostjo. Zaupanje je smiselno le v okoliščinah, kjer obstaja možnost, da drugi akter ne bo ravnal skladno s pričakovanji [6]. Če bi imeli popoln nadzor ali vse informacije, zaupanje sploh ne bi bilo potrebno. V kontekstu varnostnih in distribucijskih sistemov zaupanje dopolnjuje formalne zaščitne mehanizme, kot so avtentikacija in šifriranje, saj omogoča oceno vedenja akterjev tam, kjer tehnični ukrepi ne zadostujejo [16].

Zaupanje je tudi subjektivno in kontekstualno. Različni akterji lahko enake vedenjske vzorce interpretirajo različno, odvisno od svojih ciljev, regulatornih zahtev, izkušenj, prepričanj itn.

2.1.3 Atributi zaupanja

Da bi bilo zaupanje uporabno v digitalnem okolju, ga je potrebno izraziti z merljivimi atributi, ki opisujejo lastnosti ali vedenje zaupanja vrednega subjekta. Ti atributi omogočajo, da se konceptualna ideja zaupanja preslika v formalne modele. V literaturi se pojavljajo različne skupine atributov.

Klasični modeli, kot sta predstavljena v delih avtorjev Mayer, Davis in Schoorman [32] ter McKnight in Chervany [34], opredeljujejo tri temeljne dimenzije zaupanja, ki jih slednja avtorja klasificirata kot prepričanja o lastnostih partnerja: kompetentnost, dobrohotnost in integriteta. Kompetentnost (angl. *Competence*) poleg usposobljenosti opraviti nekaj, pomeni zlasti prepričanje, da ima entiteta zmožnosti in vire za učinkovito izvedbo nalog, ki so zanj pričakovane. Dobrohotnost (angl. *Benevolence*) izraža prepričanje, da druga stran pri svojem delovanju upošteva interese uporabnika in ne deluje

oportunistično. Integriteta (angl. *Integrity*) oziroma načelnost pa se nanaša na zaznano moralno držo subjekta, ki vključuje spoštovanje dogovorov in dosledno izpolnjevanje obljub. Tem trem dimenzijam se v številnih delih pridružuje še predvidljivost (angl. *Predictability*), ki temelji na konsistentnosti preteklih ravnanj in omogoča sklepanje o prihodnjem vedenju entitete v podobnih okoliščinah.

V digitalnih in porazdeljenih okoljih so se pojavili dodatni atributi, ki izhajajo iz tehničnih vidikov zaupanja. Grandison in Sloman [21] zaupanje opišeta kot prepričanje o pristojnosti, varnosti in verodostojnosti drugega subjekta. Denning [16] poudarja, da je zaupanje sistema mogoče ocenjevati tudi z vidika varnosti, skladnosti s pravili ter preverljivostjo delovanja. Ti atributi omogočajo formalno ocenjevanje zaupanja med akterji.

V kontekstu farmacevtskih dobavnih verig imajo atributi zaupanja izrazito regulatorno in sledilno komponento. Uddin in sodelavci [42] poudarjajo, da so pri zagotavljanju zaupanja ključni kazalniki sledljivost, pristnost in celovitost podatkov. Kayhan in sodelavci [26] izpostavljajo še pomen transparentnosti, nespremenljivosti in izvirne sledljivosti (angl. *provenance*), ki jih omogoča tehnologija veriženja blokov.

Predstavljeni atributi ne predstavljajo univerzalne množice meril, temveč zbirko značilnosti, ki jih lahko posamezen akter uporabi pri lastni presoji zaupanja. V sistemu, ki je predstavljen v tej nalogi, se atributi uporabljajo predvsem kot podatkovne lastnosti v ontologiji. Te omogočajo, da akterji formalno zapišejo svoje kriterije za ocenjevanje drugih entitet. S tem v sistemu omogočamo standardizirano predstavitev vhodnih meril, ki jih posamezni akterji uporabljajo pri svojem algoritmu za izračunavanje zaupanja.

2.1.4 Modeliranje zaupanja v računalniških sistemih

Modeliranje zaupanja v računalniških sistemih presega zgolj definicije in attribute, saj zahteva preslikavo subjektivnih konceptov v algoritme, ki morajo avtonomno odločati o tem, ali subjektu zaupajo ali ne. Modeli zaupanja določajo mehanizme za zbiranje, agregacijo in uporabo lastnosti ter atri-

butov zaupanja pri izračunu končne vrednosti. Osnovni cilj modeliranja je podpora pri odločanju v pogojih negotovosti, kjer se ocenjevalec odloča o interakciji z drugo entiteto.

Cho in sodelavci v članku *A Survey on Trust Modeling* [9] proces ocenjevanja zaupanja opisujejo kot dinamično in ponavljajočo se zanko, ki se začne z zbiranjem dokazov o preteklem vedenju, priporočil drugih ali neposrednih interakcijah. Temu sledi faza analize, kjer algoritem izračuna stopnjo zaupanja in oceni tveganje glede na ranljivost sistema ter potencialni vpliv negativnega izida. Na podlagi te ocene in določenega praga tolerance se akter odloči ali se strinja s sodelovanjem. Po končani interakciji sistem zabeleži izid, ki služi kot povratna informacija in nov dokaz za posodobitev ocene zaupanja v prihodnjih ciklih.

Zaradi heterogenosti sodobnih omrežij enodimenzionalni modeli pogosto ne zadoščajo, avtorji predlagajo koncept sestavljenega zaupanja. Ta pristop združuje štiri medsebojno povezane dimenzije: (i) komunikacijsko zaupanje, ki se nanaša na zanesljivost omrežja in kakovost storitve; (ii) informacijsko zaupanje, ki ocenjuje verodostojnost in celovitost podatkov; (iii) družbeno zaupanje, ki izhaja iz odnosov in ugleda v socialni strukturi; ter (iv) kognitivno zaupanje, ki modelira subjektivna pričakovanja in nagnjenost akterja k tveganju.

Modeli se razlikujejo tudi po načinu ocenjevanja zaupanja, ki neposredno vpliva na zmožnost sistema za odločanje. Vrednosti se lahko merijo na številski lestvici, na primer kot verjetnost med 0 in 1, ali pa na diskretni lestvici z opisnimi stopnjami zaupanja. Izbira ustreznih metril in modela zahteva iskanje ravnovesja med natančnostjo ocene in računsko zahtevnostjo, kar je posebej pomembno pri implementaciji v realnem času na napravah z omejenimi viri [9].

2.2 Upravljanje zaupanja v razpršenih sistemih

Huaizhi Li in Mukesh Singhal v članku *Trust Management in Distributed Systems* [30] opredelita upravljanje zaupanja kot proces zbiranja informacij, potrebnih za vzpostavitev zaupanja med entitetami, kot tudi dinamičnega spremljanja in prilagajanja obstoječih razmerij zaupanja. Avtorja poudarjata, da v razpršenih okoljih kot so internet, sistemi enakovrednih entitet (angl. *peer-to-peer networks*) in mobilna omrežja ad hoc pogosto sodelujejo entitete, ki se med seboj ne poznajo. Zato zaupanje postane ključni mehanizem za zmanjšanje tveganja in zagotavljanje sodelovanja. Namen sistema za upravljanje zaupanja je ohranjati ažurne in usklajene informacije o zaupanju med akterji v omrežju. V naslednjih podpoglavjih povzemamo različne pristope k upravljanju zaupanja, ki so jih avtorji predstavili v svojem pregledu literature. V sistemu, razvitem v sklopu magistrske naloge ti pristopi in enačbe niso predpisani. Služijo kot možne izbire za izračunavanje zaupanja za posamezne akterje.

2.2.1 Dokazni modeli zaupanja

Pri dokaznih modelih zaupanja se zaupanje ne gradi na preteklih interakcijah ali priporočilih, temveč na preverjanju formalnih dokazil, ki potrjujejo identiteto ali lastnosti entitet.

Eschenauer in sodelavci predstavijo ogrodje za upravljanje zaupanja v omrežjih ad hoc, ki temelji na preverjanju dokazil [18]. Zaupanje obravnavajo kot množico odnosov, vzpostavljenih na podlagi dokazil. Dokazila so lahko karkoli, kar zahteva politika za vzpostavitev zaupanja. Po navadi so to javni ključi, naslovi ali identitete. Vsaka entiteta lahko ustvari dokazilo zase ali za druge entitete. Najpogostejša primera sta hierarhični sistem X.509 [22] in decentralizirani sistem PGP [17], kjer se zaupanje posreduje prek verige certifikatov oziroma prek subjektivnih ocen uporabnikov. Tak model se uporablja predvsem pri inicializaciji zaupanja in zagotavlja osnovno preverjanje

pristnosti brez ocenjevanja vedenja.

2.2.2 Priporočilni modeli zaupanja

Priporočilni modeli gradijo zaupanje na osnovi izkušenj in posredovanih ocen drugih entitet. Vsaka entiteta ocenjuje druge glede na pretekle interakcije, te ocene pa se lahko prenašajo naprej kot priporočila. Model uporablja pogojno tranzitivnost zaupanja – entiteta A lahko zaupa entiteti C , če zaupa entiteti B kot priporočitelju in lahko ovrednoti kakovost njegovih priporočil.

Abdul-Rahman in Hailes predlagata porazdeljeni model zaupanja, ki temelji na izračunu zaupanja iz verig priporočil med akterji [1]. Vsak akter hrani diskretne vrednosti zaupanja na lestvici od -1 do 4 , kjer -1 pomeni popolno nezaupanje, 0 nevednost (akter si ne upa podati ocene), vrednosti 1 do 3 označujejo naraščajoče stopnje zaupanja, 4 pa predstavlja popolno zaupanje. Ločita med:

- *vrednostjo priporočiteljevega zaupanja* $tv(R_i)$, ki opisuje, koliko akter zaupa i -temu priporočitelju na poti in
- *priporočeno vrednostjo zaupanja* $rtv(T)$, ki jo o ciljni entiteti T poda zadnji priporočitelj v verigi.

Naj bo n število priporočiteljev, $R_1, R_2, \dots, R_n$ pa zaporedni priporočitelji na poti od izvirnega priporočitelja do tistega, ki priporočilo zahteva. Zaupanje v ciljno entiteto T po poti p označimo z $tv_p(T)$. Avtorja predlagata naslednjo formulo za izračun zaupanja po poti p [1]:

$$tv_p(T) = \left(\prod_{i=1}^n \frac{tv(R_i)}{4} \right) \cdot rtv(T). \quad (2.1)$$

Ker lahko akter za isto ciljno entiteto T prejme več priporočil po različnih poteh, naj bo r število teh poti, označenih z $i = 1, \dots, r$. Vse delne ocene zaupanja $tv_1(T), \dots, tv_r(T)$ se povprečijo, tako da se končna priporočilna ocena zaupanja v entiteto T izračuna kot $tv(T) = \frac{1}{r} \sum_{i=1}^r tv_i(T)$ [1].

2.2.3 Porazdeljeno ocenjevanje zaupanja

V omrežjih enakovrednih akterjev (P2P) se zaupanje ocenjuje na decentraliziran način. Vsako vozlišče vodi lastno evidenco interakcij in vrednoti druge glede na uspešnost sodelovanja. Xiong in Liu v okviru sistema PeerTrust predlagata merilo, ki zaupanje akterja u izrazi kot razmerje med združenimi ocenami zadovoljstva drugih z njegovimi storitvami in skupnim številom interakcij [48].

Naj bo P množica vseh vozlišč v omrežju, $u \in P$ opazovani akter, $v \in P$ ($v \neq u$) preostala vozlišča, t pa številka transakcije. Funkcije so definirane tako:

- $I(u, v, t)$ je število interakcij med u in v do transakcije t ,
- $S(u, v, t)$ je število interakcij vozlišča v z akterjem u do transakcije t , kjer ni prišlo do pritožb,
- $Cr(v, t) \in [0, 1]$ je faktor uravnoteženja zaupanja, ki zmanjšuje tveganje neverodostojnih povratnih informacij iz vozlišča v .

Ocena zaupanja v akterja u do časa t je nato definirana kot [48]

$$T(u, t) = \frac{\sum_{v \in P, v \neq u} S(u, v, t) Cr(v, t)}{\sum_{v \in P, v \neq u} I(u, v, t)}. \quad (2.2)$$

Vrednost $T(u, t)$ leži v intervalu $[0, 1]$ in predstavlja globalno reputacijsko oceno akterja u na podlagi skupnega, uteženega zadovoljstva vseh vozlišč, ki so z njim sodelovala.

Za odločanje, ali je akter zaupanja vreden, avtorja uvedeta pragova c_1 in c_2 . Prag c_1 določa minimalno število skupnih interakcij, ki jih mora imeti akter, da lahko opazovalec sprejme informirano odločitev. Prag c_2 pa izraža, kako visoko vrednost zaupanja je opazovalec pripravljen sprejeti kot še dovolj dobro: bolj toleranten akter lahko izbere nižji c_2 , manj toleranten pa višjega. Akter u se obravnava kot zaupanja vreden, če hkrati velja $I(u, t) > c_1$ in $T(u, t) > c_2$ [48].

2.2.4 Dinamično posodabljanje zaupanja

Ker se vedenje akterjev sčasoma spreminja, se mora zaupanje prilagajati novim informacijam. Posodobitev vrednosti se pogosto izvede z amortizacijskimi faktorji, ki dajejo večjo težo nedavnim interakcijam. Primer takšnega modela predstavijo Lin in sodelavci v članku *A Reputation and Trust Management Broker Framework for Web Applications* [31].

Avtorji predlagajo porazdeljen sistem posrednikov zaupanja, kjer vsak posrednik vzdržuje bazo podatkov o ugledu storitev na podlagi povratnih informacij uporabnikov po opravljenih transakcijah. Ko uporabnik posreduje novo oceno r za določeno storitev, posrednik posodobi shranjeno vrednost ugleda T_{old} v novo vrednost T_{new} . Pri tem upošteva število preteklih transakcij N , časovno razliko Δt med novo oceno in zadnjo shranjeno oceno ter parameter časovnega zastaranja β , ki določa, kako hitro starejše ocene postanejo manj pomembne. Formula za posodobitev ugleda je podana kot [31]:

$$T_{new} = e^{-\beta\Delta t} \frac{N}{N+1} T_{old} + (1 - e^{-\beta\Delta t}) \frac{N}{N+1} r, \quad (2.3)$$

kjer je $e^{-\beta\Delta t}$ amortizacijski faktor, ki zmanjšuje vpliv starejših ocen glede na časovno razliko Δt . Tako se zagotovi, da novejšje ocene bolj vplivajo na ugled, kar omogoča dinamično prilagajanje zaupanja glede na trenutno vedenje akterjev.

2.2.5 Povezava z našo rešitvijo

Predstavljeni modeli in enačbe v tem poglavju služijo kot pregled možnih pristopov k izračunu zaupanja v razpršenih okoljih. V okviru te naloge jih obravnavamo kot primere, ki jih je mogoče uporabiti pri implementaciji izračuna v orakljih, vendar njihova uporaba ni predpisana.

Osrednji poudarek naloge je na zasnovi in implementaciji sistema, ki omogoča, da različni akterji izvedejo izračun zaupanja po lastnih pravilih in z uporabo lastnih algoritmov. Sistem zato ne vsiljuje enotnega modela, temveč zagotavlja skupen mehanizem za pridobivanje vhodnih podatkov ter

za preverljiv in trajen zapis rezultatov v verigo blokov, neodvisno od izbrane metode izračuna.

2.3 Farmacevtske dobavne verige

2.3.1 Značilnosti in pomen sledljivosti

Farmacevtska dobavna veriga je ena najbolj reguliranih in kompleksnih industrijskih verig. Vključuje proizvajalce, distributerje, prevoznike, lekarne, regulatorne agencije in končne uporabnike [37]. Zaradi velikega števila vmesnih akterjev in globalnega obsega poslovanja je nadzor nad izvorom in kakovostjo zdravil zahtevna naloga.

Sledenje zdravilom omogoča identifikacijo posamezne serije skozi celoten življenjski cikel izdelka, od proizvodnje do izdaje pacientu [26]. Natančno beleženje gibanja zdravil preprečuje vstop ponaredkov v sistem in omogoča hitro ukrepanje ob odpoklicih ali neskladjih. Sledljivost je zato neposredno povezana z varnostjo pacientov in zaupljivostjo v zdravstveni sistem [46].

Za zagotavljanje sledljivosti so regulatorji uvedli stroge predpise. Evropska unija je sprejela Direktivo o ponarejenih zdravilih (2011/62/EU) [11] in Delegirano uredbo (EU) 2016/161 [12], ki zahtevata enolično označevanje embalaže z dvodimenzionalno kodo in uporabo evropskega sistema EMVS (European Medicines Verification System). V ZDA podobno vlogo opravlja zakon DSCSA, ki uvaža popolnoma digitalno sledenje zdravil do leta 2025 [43].

Ti ukrepi predstavljajo pomembne korake k večji sledljivosti, vendar še vedno temeljijo na centraliziranih bazah podatkov, kjer posamezni akterji nimajo popolnega vpogleda v celotno verigo.

2.3.2 Izzivi in tveganja

Kljub regulacijam dobavne verige zdravil ostajajo izpostavljene številnim tveganjem. Največji problem predstavljajo ponarejena in neustrezna zdravila,

katerih število v zadnjih letih narašča. Po podatkih Svetovne zdravstvene organizacije je bilo med letoma 2017 in 2021 zabeleženih 877 incidentov ponarejenih ali neustreznih medicinskih izdelkov, kar pomeni povprečno letno rast za 36 % [47]. Ti izdelki so bili zaznani v vseh regijah sveta, tudi v državah z razvitimi regulativnimi sistemi. Ponarejena zdravila so lahko neučinkovita ali nevarna, njihova prisotnost pa neposredno ogroža zdravje pacientov in zmanjšuje zaupanje javnosti v zdravstvene sisteme.

Drugo tveganje izhaja iz razdrobljenosti informacijskih sistemov. Podatki o serijah, skladiščih in pošiljkah se pogosto hranijo v ločenih bazah, ki med seboj niso povezane [42]. To povzroča zamude, napake in oteženo preverjanje izvora zdravila. Centralizirane rešitve, kot je EMVS, sicer izboljšajo preverjanje avtentičnosti, vendar ne rešujejo težave zaupanja med različnimi organizacijami, ki podatke ustvarjajo in posredujejo.

Poleg ponarejanja in pomanjkljive sledljivosti se pojavljajo tudi tveganja, povezana z motnjami v dobavi zdravil. V kriznih obdobjih, kot so pandemije ali politične napetosti, se zaradi pomanjkanja podatkov o zalogah in zanesljivosti partnerjev pogosto pojavijo nepričakovane prekinitve dobav. Takšni dogodki razkrivajo, kako pomembno je zaupanje med organizacijami in pravočasna izmenjava podatkov.

Kljub tehnološkim in pravnim izboljšavam obstoječi modeli še vedno ne zagotavljajo celovitega vpogleda v dogajanje znotraj verige. Potrebne so rešitve, ki združujejo varno upravljanje podatkov, decentralizirano preverjanje in semantično usklajevanje informacij med akterji.

2.3.3 Regulativni okvir in trenutne rešitve

Evropska zakonodaja temelji na Direktivi o ponarejenih zdravilih (2011/62/EU) [11] ter Delegirani uredbi (EU) 2016/161 [12]. Skupaj uvajata obvezne varnostne elemente na embalaži, dvodimenzionalno kodo ter sistem EMVS. EMVS omogoča preverjanje serijske številke ob izdaji zdravila, sistemi NMVS pa sodelujejo prek skupnega evropskega vozlišča. Sistem zaznava poskuse ponarejanja v fazi distribucije, ne omogoča pa vpogleda v celotno zgodovino

serije, saj temelji na centralizirani arhitekturi [26].

V ZDA sledljivost ureja zakon DSCSA (Drug Supply Chain Security Act), ki je del zakona DQSA iz leta 2013 [43]. DSCSA uvaja elektronsko izmenjavo treh vrst podatkov: transakcijske informacije, zgodovine transakcij in izjave o skladnosti. Sistem zahteva interoperabilnost in sledljivost do leta 2025. Kljub napredku ameriški model ostaja pretežno centraliziran, podatki pa se prenašajo predvsem neposredno med poslovnimi partnerji [42].

Na mednarodni ravni Svetovna zdravstvena organizacija (WHO) pripravlja smernice za dobre distribucijske prakse in spremlja pojav ponarejenih in neustreznih izdelkov.

Za izpolnjevanje regulativnih zahtev podjetja uporabljajo različne informacijske platforme, med katerimi so najpogostejše centralizirane rešitve, kot so že omenjeni EMVS, TraceLink in SAP ATTP. Te rešitve izboljšujejo sledljivost serij in omogočajo avtomatizirano poročanje regulatorjem, vendar ne omogočajo skupnega vpogleda v podatke ali preverjanja zanesljivosti akterjev po celotni verigi [26]. Podatkovni tokovi ostajajo razdrobljeni, zanašajo pa se tudi na zaupanja vredne posrednike.

V zadnjem desetletju se pojavljajo projekti, ki preučujejo uporabo tehnologije veriženja blokov v farmacevtskih dobavnih verigah. Projekta MediLedger v ZDA in PharmaLedger v Evropi uporabljata porazdeljene zapise za izboljšanje integritete in sledljivosti podatkov [42]. Takšni pristopi dokazujejo, da veriga blokov omogoča nespremenljivost zapisov in enostavnejše preverjanje informacij, vendar večinoma rešujejo le problem integritete podatkov.

Kljub temu večina rešitev ostaja usmerjena v zaupanje v podatke, medtem ko je upravljanje zaupanja med posameznimi organizacijami pogosto implicitno, vezano na statične pravice dostopa ali vnaprej dogovorjene pogodbeno-pravne odnose. Manj je pristopov, kjer bi lahko vsak akter izrecno definiral svoje kriterije zaupanja in iz njih izpeljal odločitve o sodelovanju. Prav to vrzel bo zapolnil sistem, predstavljen v tej nalogi, saj kombinira semantični model dobavne verige, subjektivne politike zaupanja in zapis re-

zultatov v verigo blokov.

2.4 Tehnologija veriženja blokov

Tehnologija veriženja blokov je bila prvič predstavljena leta 2008 v sklopu kriptovalute Bitcoin [35]. Osnovna ideja je zasnova podatkovne strukture, ki povezuje zapise v verigo blokov. Vsak blok vsebuje podatke, zgostitev prejšnjega bloka in časovni žig. Takšna vezava med bloki preprečuje naknadne spremembe zapisanih podatkov, saj bi vsaka sprememba v enem izmed starejših blokov spremenila njegovo zgostitev. Posledično pa se zgostitev ne bi več ujemala z vrednostjo, shranjeno v naslednjem bloku [13].

Veriga blokov deluje v porazdeljenem omrežju, kjer vsako vozlišče hrani kopijo celotne verige. Soglasje o pravilnem zaporedju blokov se doseže s konsenznim mehanizmom. V javnih verigah se pogosto uporabljata t. i. Proof of Work ali Proof of Stake, medtem ko zasebna omrežja uporabljajo lažje mehanizme, ki temeljijo na znanih udeležencih. Zaradi porazdeljene narave sistem ne potrebuje centralne avtoritete, kar omogoča zanesljivo beleženje podatkov v okolju z različnimi deležniki.

Poleg javnih omrežij (kot je Ethereum), so v praksi pogosta zasebna omrežja, kjer je dostop omejen na pooblašene organizacije. Primer takšnega sistema je Hyperledger Fabric [3]. Ta pristop omogoča višjo prepustnost, večjo zasebnost podatkov in nadzor nad identitetami v omrežju. V dobavnih verigah so takšne lastnosti ključne, saj morajo podjetja varovati poslovne podatke, hkrati pa zagotavljati zanesljivo izmenjavo informacij.

Verige blokov omogočajo nespremenljiv zapis dogodkov, preverljivo zgodovino podatkov in enoten pogled na transakcije. To so pomembne zahteve farmacevtskih dobavnih verig, saj želimo omogočiti transparenten pogled na transakcije v takšnem sistemu.

2.4.1 Pametne pogodbe

Pametne pogodbe predstavljajo enega ključnih konceptov sodobnih verig blokov. Gre za programe, ki se shranijo in izvajajo neposredno v verigi blokov ter izvajajo funkcije, ko so izpolnjeni določeni pogoji. Ethereum je prvi uvedel splošno platformo za izvajanje pametnih pogodb prek navideznega stroja EVM (Ethereum Virtual Machine), ki omogoča deterministično izvajanje kode na vseh vozliščih omrežja [44].

Pametne pogodbe omogočajo avtomatizacijo pravil in procesov, pri čemer ni potrebno zaupati enemu samemu posredniku. Njihovo delovanje je predvidljivo, saj se vedno izvršijo natančno tako, kot je določeno v kodi. Zaradi te lastnosti se pogosto uporabljajo v scenarijih, kjer je treba uveljaviti dogovorjena pravila med neodvisnimi akterji. V dobavnih verigah se uporabljajo za potrjevanje dogodkov, beleženje blaga, preverjanje skladnosti podatkov, itn.

V modelih, ki temeljijo na verigi blokov, pametne pogodbe služijo tudi kot enotna plast poslovne logike. S tem omogočajo, da vsi akterji delujejo na osnovi istega sklopa pravil. Zlasti je to pomembno v okoljih, kjer se morajo organizacije dogovarjati o interpretaciji podatkov ali izvajanju procesov. Pametne pogodbe z nespremenljivo kodo zagotovijo, da se pravila ne spreminjajo brez soglasja deležnikov.

V predstavljenem sistemu ima pametna pogodba osrednjo vlogo, saj hrani rezultate ocenjevanja zaupanja in omogoča preverjanje vzpostavljenih razmerij med akterji. Pogodba vsebuje funkcije za zapisovanje rezultatov, njihovo posodabljanje in preverjanje stanja zaupanja. Tako tvori register zaupanja, ki je dostopen vsem udeležencem verige.

2.4.2 Oraklji v pametnih pogodbah

Pametne pogodbe lahko zanesljivo izvajajo le pravila nad podatki, ki so že zapisani na verigi blokov. Ker so verige blokov zasnovane kot zaprt, determinističen sistem, nimajo neposrednega dostopa do zunanjega sveta. Da bi pametne pogodbe lahko reagirale na dogodke iz realnega okolja (cene na trgu,

vremenske razmere, meritve senzorjev, podatke informacijskih sistemov), potrebujemo posrednika, ki te podatke pridobi in jih v preverljivi obliki posreduje na verigo. V literaturi takšnega posrednika imenujemo *orakelj* [7]. V nadaljevanju naloge izraz *orakelj* uporabljamo v tem tehničnem pomenu za neodvisno komponento izven verige, ki iz zunanjih podatkov izračuna oceno zaupanja in pripravi kriptografsko podpisano poročilo, ki se lahko uporabi v pametni pogodbi.

Caldarelli oraklje opiše kot celoten ekosistem, ki povezuje zunanje vire podatkov z decentralizirano aplikacijo [8]. Tipično ga sestavljajo trije deli: vir podatkov, komunikacijski kanal in pametna pogodba. Vir podatkov je lahko spletni vmesnik API, podatkovna baza, senzor v internetu stvari ali celo človek, ki potrdi nek dogodek.

Komunikacijski kanal predstavlja vozlišče oziroma niz vozlišč, ki podatke zbrane iz vira prevedejo v obliko transakcije in jih pošljejo pametni pogodbi. Pametna pogodba na drugi strani vsebuje pravila, s katerimi sprejme ali zavrne posredovane podatke in na njihovi podlagi sproži ustrezno logiko.

Ker oraklji ponovno uvajajo zaupanje v zunanje subjekte, se je v literaturi uveljavil pojem *problem oraklja* (angl. oracle problem) [7]. Veriga blokov sama po sebi nudi lastnosti, kot so nespremenljivost, deterministično izvajanje in odpornost proti cenzuri, vendar te lastnosti ne veljajo avtomatično za podatke, ki prihajajo preko oraklja. Če vir podatkov ni zanesljiv ali je komunikacijski kanal centraliziran, lahko napačni podatki povzročijo napačno izvedbo pametne pogodbe, ne da bi to bilo na verigi neposredno razvidno. Zato je v skupnosti poudarjena potreba po jasnem modelu zaupanja za vsak orakelj. Ta mora opisati, kakšen je motiv oraklja, da deluje pošteno, in kako sistem zazna ali kaznuje odstopanja [7].

V praksi se pojavlja več arhitektur, ki omogočajo delovanje orakljev. Najpreprostejši so centralizirani oraklji, kjer ena organizacija zbira podatke in jih pošilja pametnim pogodbam. Tak pristop je enostaven za implementacijo, vendar ponovno uvaja enotno točko zaupanja in napake. Sledijo jim porazdeljeni oziroma decentralizirani oraklji, kjer več neodvisnih vozlišč pri-

dobiva in združuje podatke in tako zmanjšuje tveganje za manipulacijo podatkov. Primer takšne zasnove je sistem ASTRAEA, predlagan s strani Adlerja in sodelavcev, ki uporabi glasovalno igro z vložki in ločene vloge udeležencev (oddajatelji trditev, glasovalci, certificiranci), da z ekonomsko motivacijo spodbudi udeležence k poštenemu poročanju o resničnosti trditev [2]. Podobno pristopajo tudi druge rešitve, kjer se omrežja orakljev, kot je Chainlink, uporabljajo za zbiranje podatkov iz več API-jev in senzorjev ter za izračun dodatnih meril (npr. reputacije) še pred zapisom v verigo blokov. Tak pristop uporabijo Kochovski in sodelavci v članku *Trust management in a blockchain based fog computing platform with trustless smart oracles* [27].

V okviru te naloge oraklje obravnavamo kot ločeno plast med semantičnim delom sistema in verigo blokov. Podobno kot pri Chainlinku smo zasnovali decentralizirano omrežje pametnih orakljev, kjer se izvaja koda. Omrežje orakljev iz ontologije in pravil zaupanja prebere podatke, izvede izračun zaupanja ter rezultat posreduje pametni pogodbi. S tem sledimo pristopu, ki ga Kochovski in sodelavci imenujejo *pametni oraklji brez zaupanja* (angl. *trustless smart oracles*), kjer so oraklji zasnovani tako, da so preverljivi in zamenljivi, veriga blokov pa služi kot nespremenljiv dnevnik rezultatov, ki jih izračunajo oraklji [27]. Na ta način odpravimo potrebo po centraliziranem zaupnem posredniku, hkrati pa omogočimo, da se različne implementacije orakljev oz. različni izračuni zaupanja enotno zapišejo v register zaupanja na verigi blokov. Podrobnejši opis delovanja decentraliziranega sistema orakljev si bomo pogledali v poglavju 3.

2.4.3 Decentralizirane identitete in preverljiva dokazila

Decentralizirane identitete (DID) predstavljajo mehanizem za dosledno in preverljivo identifikacijo akterjev v decentraliziranih okoljih. Standard W3C DID opisuje identifikatorje, ki niso vezani na centraliziranega ponudnika, temveč so pod nadzorom lastnika identitete [39]. DID je globalno enoličen identifikator, na primer `did:ethr:0x1234...`, pri čemer se podatki o tem,

kako identiteto preveriti, hranijo v tako imenovanem DID dokumentu. Ta dokument vsebuje javne ključe, metode preverjanja in dodatne attribute, ki jih sistem lahko uporabi pri avtentikaciji.

Preverljiva dokazila (angl. Verifiable Credentials - VC) so standardizirana digitalna dokazila, ki jih neka entiteta izda drugi entiteti. Model W3C sledi arhitekturi izdajatelj–imetnik–preveritelj. Izdajatelj (Issuer) ustvari in podpiše dokazilo, imetnik (Holder) ga hrani, preveritelj (Verifier) pa ga validira s kriptografskim podpisom [41]. VC omogočajo prenos lastnosti ali potrdil na način, ki je preverljiv, varen in ne potrebuje zaupanja v posrednika. Preverljivost in varnost se zagotavljata z uporabo digitalnih podpisov. Imetnik dokazilo lahko predloži komurkoli kadarkoli, brez obveščanja ali dovoljenja izdajatelja. Preveritelj lahko podpis preveri lokalno, s čimer se potrdita pristnost izdajatelja in celovitost podatkov, brez potrebe po centralnem posredniku ali sprotne poizvedovanju pri izdajatelju [33].

V predstavljenem sistemu ima vsak akter v ontologiji lasten DID. Ta služi kot osnovni identifikator, na katerega se vežejo atributi in podatki v ontologiji. Na ta način sistem ohrani enotno identiteto akterjev med vsemi plastmi arhitekture.

Preverljiva dokazila uporabljamo za opis lastnosti, ki so pomembne za ocenjevanje zaupanja. Regulator, kot je EMA, lahko na primer izda VC, ki potrjuje veljavnost licence, certifikat GMP ali rezultat presoje kakovosti. Ta dokazila so digitalno podpisana in povezana z DID akterja. Orakelj lahko med izračunom zaupanja prebere VC, preveri podpis izdajatelja ter iz podatkov izlušči merila, katera akter uporablja v svoji politiki zaupanja. Če akter v svojih pravilih določi, da mora imeti partner veljavno licenco ali ustrezno oceno presoje kakovosti, lahko te kriterije pridobi neposredno iz VC.

2.5 Semantični splet in ontologije

Semantični splet predstavlja razširitev svetovnega spleta, kjer podatki niso opisani le sintaktično, temveč tudi semantično. To pomeni, da so podatki strukturirani na način, ki omogoča strojno razumevanje in logično sklepanje. Temeljna ideja je, da lahko različni sistemi izmenjujejo podatke v enotnih formatih ter iz njih izpeljujejo nova dejstva, ne da bi se morali dogovoriti o skupni implementaciji ali aplikaciji [29].

RDF (Resource Description Framework) je osnovni podatkovni model semantičnega spleta. Podatki so predstavljeni v obliki trojk subjekt-predikat-objekt, kar omogoča enostavno združevanje in razširjanje podatkovnih grafov [14]. RDF ne predpisuje sheme, zato se lahko prilagodi različnim domenam. Zapis se pogosto izvaja v obliki Turtle, RDF/XML ali JSON-LD.

Nad RDF je zasnovan OWL (Web Ontology Language), ki omogoča formalno opisovanje razredov, lastnosti in odnosov med pojmi. OWL temelji na opisni logiki, kar omogoča avtomatsko sklepanje in preverjanje konsistence ontologije [10]. Z uporabo sklepalnikov (angl. reasoner) lahko sistem samodejno razvršča primerke v ustrezne razrede, zazna protislovja in izpelje nova dejstva, ki niso bila eksplicitno zapisana.

Ontologije so osrednji gradnik semantičnega spleta. Predstavljajo formalne modele znanja, kjer so pojmi, lastnosti in hierarhije jasno definirani. Tak pristop je posebej uporaben v kompleksnih domenah, kjer nastopa veliko različnih akterjev in množica podatkovnih razmerij. Ontologije omogočajo enotno semantično podlago, ki ni vezana na implementacijo posameznega sistema, temveč na konceptualni model podatkov.

V predstavljenem sistemu ontologija služi kot centralni model znanja za vse akterje farmacevtske dobavne verige. RDF omogoča predstavitev akterjev in njihovih lastnosti kot povezav v grafu, OWL pa dodaja formalne razrede in omejitve, ki omogočajo avtomatsko sklepanje o lastnostih ali vlogah akterjev. Sklepalnik iz grafa izlušči podatke, ki jih orakelj nato uporabi pri izračunu zaupanja. Z uporabo semantičnih tehnologij je mogoče podatke strukturirati na način, ki je razširljiv, interoperabilen in neodvisen od posa-

mezne aplikacije. Zato je plast ontologije primerna kot osnova za izmenjavo informacij med različnimi organizacijami in se dobro povezuje s preostalimi deli sistema.

2.6 Analiza vrzeli in prispevek te naloge

V tem poglavju smo predstavili ključne gradnike, ki so pomembni za razumevanje razvitega sistema. Najprej smo opredelili pojem zaupanja in razložili, da gre za večdimenzionalen koncept, ki ga ni mogoče zajeti z eno samo definicijo. Različni modeli poudarjajo kompetentnost, integriteto, dobrohotnost in zanesljivost, v digitalnem okolju pa se temu pridružijo še vidiki varnosti, verodostojnosti in skladnosti z zahtevami sistema. Nato smo opisali modele upravljanja zaupanja v razpršenih sistemih ter pokazali, da se ti modeli večinoma osredotočajo na izračun številčne vrednosti zaupanja na podlagi dokazov, priporočil ali zgodovine interakcij.

V nadaljevanju smo predstavili posebnosti farmacevtskih dobavnih verig. Te verige so močno regulirane in vključujejo veliko različnih akterjev. Regulativa, kot sta evropska direktiva o ponarejenih zdravilih in zakon DSCSA v ZDA, uvaža sledljivost serij in elektronsko izmenjavo podatkov. Kljub temu ostajajo pomanjklivosti, ki jih je potrebno odpraviti. Obstoječi sistemi večinoma temeljijo na centraliziranih bazah podatkov, kjer posamezna podjetja nimajo celovitega vpogleda v verigo. Ponarejena ali neustrezna zdravila se še vedno pojavljajo, podatkovni tokovi pa so razdrobljeni in vezani na zaupanja vredne posrednike. Trenutne rešitve tako izboljšujejo sledljivost, ne rešujejo pa dovolj dobro vprašanja, komu lahko posamezna organizacija zaupa pri sodelovanju.

Tehnologija veriženja blokov in pametne pogodbe ponujajo nov način upravljanja podatkov v takih okoljih. Verige blokov omogočajo nespremenljiv zapis dogodkov in enoten pogled na podatke v porazdeljenem omrežju. Pametne pogodbe omogočajo izvajanje skupnih pravil, ki veljajo za vse udeležence. Oraklji rešujejo problem povezave med verigo blokov in zunanjimi viri po-

datkov, vendar ponovno odpirajo vprašanje, komu zaupati pri posredovanju podatkov. Semantični splet in ontologije pa omogočajo formalno predstavitev znanja o akterjih in njihovih lastnostih ter avtomatsko sklepanje nad podatki. V literaturi se ti pristopi le redko združijo v enotno arhitekturo, kjer bi imeli akterji možnost, da na podlagi skupne ontologije definirajo svoje kriterije zaupanja, rezultat teh ocen pa se nato trajno zapiše v verigo blokov.

Na tej točki se pokaže osrednja vrzel. Obstoječe rešitve za sledljivost zdravil se osredotočajo na varnost in integriteto podatkov, upravljanje zaupanja med organizacijami pa je običajno implicitno. Temelji na statičnih pravicah dostopa, pogodbah in ročnih pregledih partnerjev. Po drugi strani številni modeli zaupanja ne upoštevajo posebnosti farmacevtske domene in niso povezani z verigo blokov. Manj je pristopov, kjer bi lahko vsak akter izrecno zapisal svojo politiko zaupanja, jo vezal na skupni semantični model ter rezultate ocenjevanja delil na način, ki ga lahko drugi akterji neodvisno preverijo.

V tabeli 2.1 povzamemo ključne kategorije pristopov, ki se pojavljajo v literaturi in industrijskih rešitvah. Primerjava poudari razlike med centraliziranimi metodami, reputacijskimi sistemi, semantičnimi modeli in rešitvami na osnovi verige blokov. V zadnjem stolpcu je prikazano, kako sistem, predstavljen v tem delu, združuje elemente teh pristopov in uvaja nove mehanizme, kot so več neodvisnih orakljev, kriptografsko podpisana poročila ter kombinacija dokaznih in telemetrijskih kriterijev.

Tabela 2.1: Primerjava obstoječih pristopov k upravljanju zaupanja in novosti razvitega prototipa.

Pristop	Vir zaupanja	Dinamika posodabljanja	Razlaga odločitve	Omejitve
Centralizirani sistemi kakovosti (GDP [19]/GMP [45])	ročne revizije, digitalna potrdila, regulatorji	nizka; periodične revizije	delna; opis revizijskih postopkov	centralizacija, počasen odziv, pomanjkanje kriptografske sledljivosti
Reputacijski modeli (ocene partnerjev) [24, 40]	mnenja, zgodovina interakcij	srednja; po vsaki interakciji	nizka–srednja; pogosto neformalna	možnost manipulacij, subjektivnost, neprenosljivost ocen
Semantični modeli zaupanja [4]	ontologije, formalna pravila, sklepanje	srednja; ob posodobitvi virov znanja	visoka; formalna in sledljiva razlaga	ni vgrajenega mehanizma za integriteto podatkov na verigi
Rešitve z uporabo verige blokov (blockchain) [28, 42]	transakcije, digitalna potrdila	visoka; dogodki v realnem času	srednja; preverljivost podatkov, omejena razlaga	ne obravnavajo izračuna zaupanja; brez večnivojskih kriterijev
Predstavljen sistem	VC, ontologija, DON	visoka; vsak izračun sproži nove ocene	zelo visoka; deterministični in verjetnostni del z razlagami	odprta vprašanja upravljanja identitet, strošek implementacije

Prispevek te naloge je predlog sistema, ki omenjene elemente poveže v enoten okvir. Ontologija farmacevtske dobavne verige predstavlja skupno semantično plast, v kateri so akterji in njihove lastnosti opisani na strukturiran in razširljiv način. Vsak akter lahko nad temi podatki definira lastna pravila zaupanja in tako določi, katera merila so zanj pomembna. Mehanizem za sklepanje iz ontologije prebere lastnosti akterjev, uporabi lokalna pravila in za vsak par udeležencev izračuna rezultat zaupanja. Decentralizirani oraklji nato te rezultate prenesejo v pametno pogodbo, ki na verigi blokov hrani register relacij zaupanja. Sistem povezuje tudi decentralizirane identitete (DID) in preverljiva dokazila VC, kadar akterji želijo izvor podatkov podpreti z digitalnimi certifikati.

Tak pristop neposredno naslavlja vrzeli, opisane v tem poglavju. Namesto enotnih, vnaprej določenih meril zaupanja, omogoča, da vsak akter oblikuje lastno politiko na osnovi skupnega modela podatkov. Rezultati teh različnih politik so kljub temu zbrani v enoten register, ki ga lahko delijo vsi udeleženci verige. S tem sistem podpira različne poglede na zanesljivost partnerjev, hkrati pa ohranja konsistentno predstavitev identitet in podatkovnih lastnosti.

Čeprav je sistem razvit na primeru farmacevtske dobavne verige, je zasnova splošna. Ontologijo in nabor atributov bi bilo mogoče prilagoditi drugim domenam, na primer pametnim mestom ali omrežjem interneta stvari, kjer številni akterji izmenjujejo podatke in storitve. Enak vzorec se ponovi tudi tam: skupni semantični model entitet, subjektivne politike zaupanja, modul za sklepanje in register rezultatov na verigi blokov.

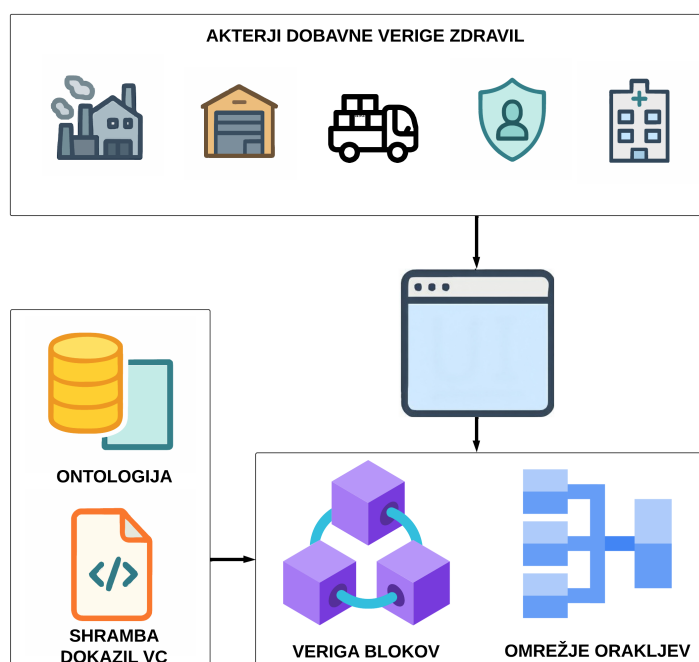
V nadaljevanju naloge bomo opisali, kako je tak sistem konkretno implementiran in kako se odziva v izbranih scenarijih uporabe.

Poglavje 3

Implementacija sistema

V tem poglavju preidemo od konceptualnega modela k dejanski implementaciji prototipa. Na začetku opišemo tehnologije, ki smo jih uporabili. V nadaljevanju bolj podrobno orišemo arhitekturo ter njene ključne komponente. Na koncu predstavimo delovanje sistema skozi tokove podatkov in razložimo, kako so različni deli povezani v celoto. Izvorna koda prototipa, konfiguracijske datoteke in navodila za zagon so dostopni v javnem repozitoriju [38].

Na sliki 3.1 je prikazan koncept ciljne arhitekture sistema. Diagram ponazarja glavne gradnike: uporabnike (akterje dobavne verige), uporabniški vmesnik, plast znanja (ontologija in shramba preverljivih poverilnic), decentralizirano omrežje orakljev (DON) ter verigo blokov s pametno pogodbo, ki služi kot skupni register zaupanja. Ciljni koncept delovanja sistema je bil, da lahko uporabniki preko uporabniškega vmesnika komunicirajo s sistemom zaupanja na verigi blokov, ki črpa znanje iz ontologije in shrambe preverljivih poverilnic. Prototip, ki ga opišemo v nadaljevanju, realizira to zasnovo v testnem okolju in omogoča izvajanje ponovljivih scenarijev.



Slika 3.1: Konceptualni prikaz arhitekture sistema za upravljanje zaupanja

3.1 Uporabljene tehnologije

Pri razvoju sistema smo uporabili različna orodja in platforme, ki skupaj omogočajo obdelavo semantičnih podatkov, izvajanje izračuna zaupanja ter zapis rezultatov v verigo blokov. V tem razdelku podajamo pregled najpo-

membnejših tehnologij in njihovih vlog v sistemu.

Apache Jena Fuseki Semantični podatki o akterjih in njihovih lastnostih so shranjeni v strežniku Apache Jena Fuseki, ki predstavlja RDF podatkovno bazo z dostopom prek vmesnika SPARQL. Fuseki omogoča centralizirano hrambo ontologije in primerkov ter enoten način pridobivanja podatkov za vse oraklje. Zaradi stabilnosti, enostavne uporabe in dobre združljivosti s standardi RDF je bila izbira naravna. Sistem se izvaja v vsebniku Docker, kar omogoča hitro postavitve in ponovljivost okolja.

Ontologija v OWL/RDF Ontologija farmacevtske dobavne verige je zapisana v jeziku OWL, ki definira razrede, lastnosti in odnose med akterji. Podatki so organizirani v RDF trojkah, kar omogoča enostavno obdelavo s knjižnico `rdflib` v programskem jeziku Python. Na ta način lahko sklepanje temelji na aktualnih podatkih, zapisanih v ontologiji, brez potrebe po relacijski bazi ali ročnem povezovanju.

Ethereum, Solidity in pametna pogodba Za zapis rezultatov zaupanja smo uporabili verigo blokov Ethereum. Pametna pogodba `TrustGraph.sol`, napisana v jeziku Solidity, hrani relacije zaupanja med akterji. Ethereum je bil izbran zaradi široke podpore za orodja, stabilnega ekosistema in zmožnosti izvajanja pametnih pogodb brez zaupanja v enega posrednika. Razvoj in testiranje sta izvedena z orodjem Foundry, ki nudi lokalno testno omrežje, prevajalnik ter hitre skripte za izvajanje transakcij.

Pametna pogodba omogoča zapis rezultatov izračuna, preverjanje stanja zaupanja in beleženje dogodkov. Deluje kot nespremenljiv register, ki je dostopen vsem akterjem sistema.

Decentralizirane identitete in preverljiva dokazila Vsak akter v ontologiji ima dodeljen decentraliziran identifikator (v nadaljevanju DID). DID služi kot enoličen identifikator, ki povezuje podatke v ontologiji s predstavnikom akterja v decentraliziranem okolju. Poleg tega sistem podpira uporabo

preverljivih dokazil (VC), ki jih lahko izdajajo regulatorji, kot je EMA. Ta dokazila vsebujejo merila, kot so veljavnost licence, skladnost s standardi GMP ali rezultati presoje kakovosti. Orakelj lahko, kadar akter to določi v pravilih, med izračunom zaupanja prebere VC in uporabi podatke kot dodatne kriterije.

Python in osnovne knjižnice Del logike sistema je implementiran v programskem jeziku Python, kjer se izvaja logika orakljev, agregatorjev ter orodij za upravljanje ontologije. Python je bil izbran zaradi že razvitih uporabnih knjižnic za obdelavo podatkov, delo z RDF in integracijo z verigami blokov. Spodaj so povzete ključne knjižnice, ki jih sistem uporablja:

- **web3** in **eth_account** – knjižnica **web3** omogoča povezavo z omrežjem Ethereum oziroma lokalnim vozliščem Anvil, klicanje funkcij pametne pogodbe in pošiljanje transakcij. Modul **eth_account** se uporablja za podpisovanje poročil orakljev in preverjanje podpisov, ki jih agregator prejme.
- **aiohttp** zagotavlja asinhroni spletni strežnik in odjemalca, ki ju uporabljata agregator in oraklji. Prek njega oraklji prejemaajo zahteve za izračun zaupanja, berejo podatke iz Fusekija in pošiljajo poročila nazaj agregatorju.
- **rdflib** omogoča delo z RDF grafi, lažje branje datotek OWL in manipulacijo ontologije.
- **SPARQLWrapper** omogoča pošiljanje poizvedb SPARQL strežniku Apache Jena Fuseki in s tem branje lastnosti akterjev iz ontologije
- **requests** se uporablja pri pomožnih sinhronih klicih HTTP, kjer asinhrona izvedba ni potrebna ali bi otežila implementacijo.
- **click** se uporablja pri implementaciji ukaznih orodij, kot je **trustkb**, ki omogoča delo z ontologijo, uvoz podatkov in izvajanje poizvedb SPARQL iz ukazne vrstice.

- **pytest** knjižnica je bila uporabljena za enotsko testiranje ključnih modulov sistema, saj omogoča enostavno pisanje testov ter avtomatizirano preverjanje logike orakljev in agregatorja.
- **didkit** – knjižnica se uporablja v primerih, kjer se digitalna dokazila (VC) izdaja ali preverja neposredno iz Python okolja. Omogoča delo z DID metodami in verifikacijo podpisov, skladno s standardi W3C DID in VC.

Pametni oraklji Oraklji predstavljajo most med semantično plastjo in verigo blokov. Vsak orakelj zažene Python modul, ki prebere ontologijo, uporabi pravila akterja in izračuna vrednosti zaupanja. Nato z uporabo knjižnice **web3** pošlje transakcijo pametni pogodbi. Oraklji so zasnovani modularno, zato jih lahko organizacije poganjajo samostojno, kar omogoča decentralizirano izvajanje izračuna.

Docker Vse komponente sistema so zajete v vsebnikih Docker. To nam je omogočilo hitro postavitve razvojnega okolja, enostavno upravljanje odvisnosti in ponovljivost testov. Vsak orakelj, strežnik Fuseki in lokalna veriga Ethereum se izvajajo v ločenih vsebnikih. Docker je uporabljen izključno za potrebe testiranja med razvojem.

3.1.1 Podpisovanje in preverjanje poročil orakljev

Pomembne funkcije, ki jih uporabljamo v našem sistemu, temeljijo na kriptografskih mehanizmih za zagotavljanje integriteto in preverljivost podatkov. V nadaljevanju opišemo funkciji, ki ju uporabljamo za digitalno podpisovanje poročil (**sign_message**) in preverjanje pristnosti podpisnika le teh (**recover_message**). Obe funkciji v Ethereumovem okolju uporabljata algoritem digitalnega podpisa ECDSA.

ECDSA predstavlja naprednejšo različico algoritma DSA, ki za svoje delovanje uporablja matematične lastnosti eliptičnih krivulj. Glavna prednost uporabe tega pristopa namesto DSA je, da zagotavlja enako raven varnosti

pri bistveno krajših dolžinah ključev. To omogoča hitrejše generiranje podpisov in manjšo porabo prostora, kar je pomembno za učinkovitost transakcij v omrežjih veriženja blokov. Ethereum uporablja eliptično krivuljo imenovano **secp256k1** [5].

Proces podpisovanja se začne z ustvarjanjem zgostitve sporočila. Podpisnik nato uporabi svoj zasebni ključ, ki ga algoritem s pomočjo operacij na eliptični krivulji pretvori v digitalni podpis. Rezultat tega postopka je podpis, sestavljen iz trojke (v, r, s) , ki tvori končni podpis. Funkcija **sign_message** v našem sistemu sprejme zgostitev sporočila in zasebni ključ podpisnika ter vrne podpis, vrednosti (v, r, s) in podpisano sporočilo.

Podpis preverjamo s funkcijo **recover_message**. Ta na podlagi podpisa in zgostitve sporočila obnovi naslov podpisnika. Če se izračunani naslov ujema z znanim naslovom pošiljatelja, je pristnost sporočila potrjena.

3.2 Arhitektura sistema

Arhitektura razvitega sistema temelji na večplastnem pristopu, ki ločuje podatkovni model, mehanizme izračuna in zapis rezultatov zaupanja. Takšna zasnova omogoča jasne meje med posameznimi odgovornostmi, kar olajša razvoj, testiranje in morebitno razširjanje sistema na druge domene. V okviru magistrske naloge smo zasnovali arhitekturo, ki je dovolj splošna, da jo je mogoče uporabiti v različnih razpršenih okoljih, od farmacevtskih dobavnih verig do pametnih mest.

Osnovna ideja arhitekture je bila, da združimo tri ključne tehnologije: semantični splet, pametne oraklje in verigo blokov. Semantični splet omogoča opis akterjev in njihovih lastnosti v ontologiji. Oraklji predstavljajo računsko plast, kjer posamezni akter izvede oceno zaupanja po svojih pravilih. Veriga blokov pa služi kot zanesljiva in nespremenljiva plast, kjer se hranijo rezultati teh ocen.

Sistem se razlikuje od ostalih rešitev, ki se osredotočajo zgolj na sledljivost ali upravljanje podatkov. Omogoča, da vsak akter izrecno definira svoje

kriterije zaupanja in jih izvede nad skupnim modelom znanja. Rezultati teh izračunov se zapišejo v decentraliziran register, ki je enak za vse udeležence.

V nadaljevanju poglavja predstavimo zasnovo arhitekture in posamezne plasti, prikažemo tok podatkov med njimi ter utemeljimo vlogo uporabljenih tehnologij.

3.2.1 Pregled večplastne zasnove

Arhitektura sistema temelji na treh jasno ločenih plasteh, ki skupaj tvorijo celoten potek od zajema podatkov do zapisa rezultatov zaupanja na verigo blokov. Takšna razdelitev omogoča boljše razumevanje posameznih vlog in enostavnejše nadgrajevanje sistema, saj lahko vsako plast obravnavamo neodvisno od drugih.

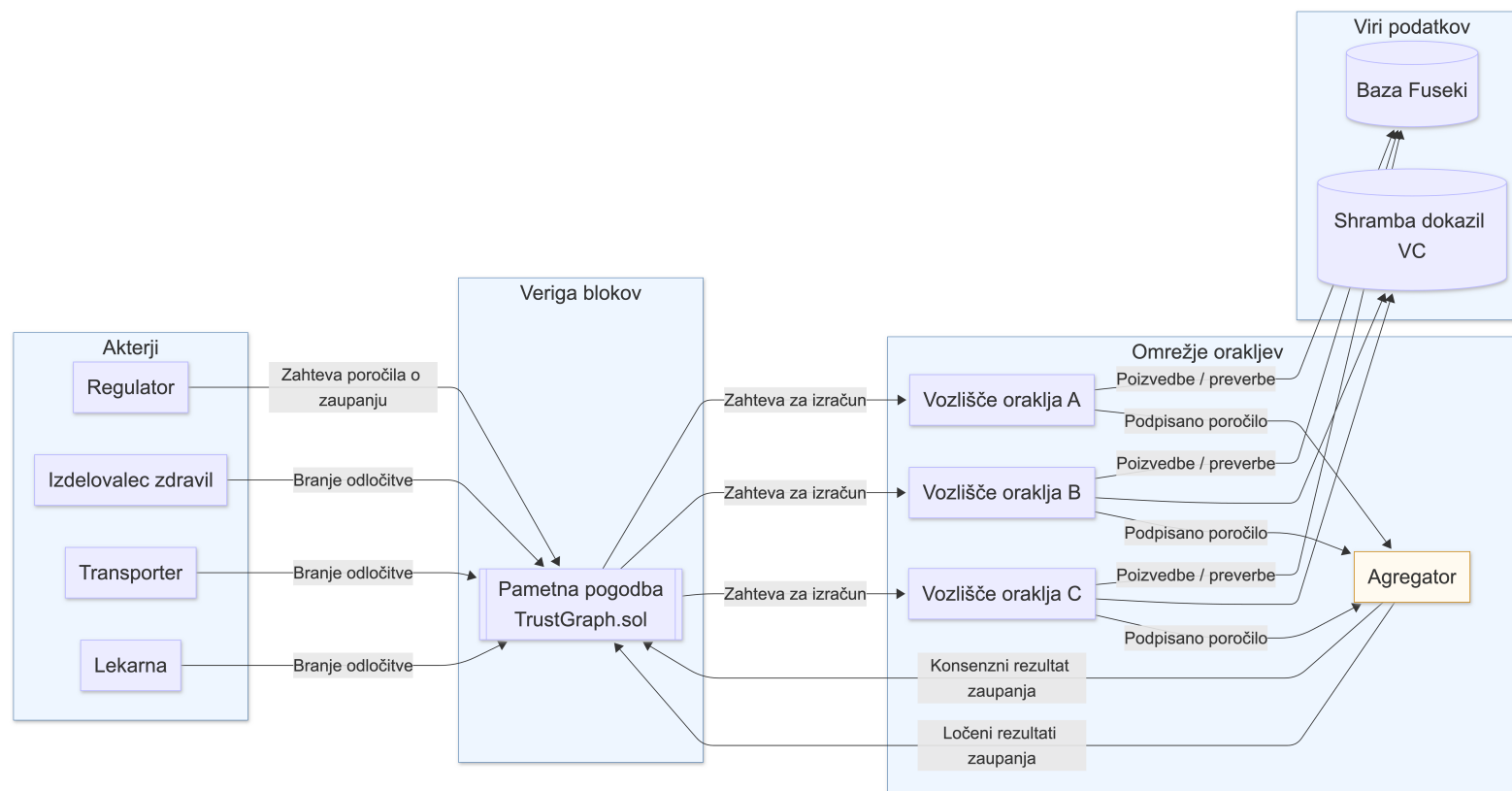
Na sliki 3.2 je prikazana arhitektura celotnega sistema. Diagram združuje tri glavne sklope: akterje iz dobavne verige na levi, infrastrukturo orakljev in podatkovnih virov na desni ter verigo blokov v sredini. Akterji, kot so regulator, proizvajalec zdravil, prevoznik in lekarna, komunicirajo izključno s pametno pogodbo `TrustGraph.sol`. Ta beleži rezultate zaupanja in upravlja zahteve za ponovno oceno posameznih entitet.

Ko regulator ali drug akter sproži zahtevo za izračun zaupanja, pametna pogodba pošlje zahtevo omrežju orakljev. Vsak orakelj nato izvede svoj del naloge: iz ontologije in baze `TrustKB` pridobi podatke, po potrebi preveri pripadajoča preverljiva dokazila, nato pa izračuna rezultat zaupanja. Oraklji rezultate podpišejo in posredujejo agregatorju, ki pripravi končni konsenzni izračun in ga zapiše nazaj v pametno pogodbo.

V skrajnem desnem delu diagrama je prikazana plast znanja. Ta vključuje ontologijo (`TrustKB`), kjer so zbrani koncepti, njihove lastnosti in meritve, ter skladišče dokazil `VC`, ki zajema licence, certifikate in druge verodostojne podatke o akterjih. Oraklji pri izračunu združujejo oba vira: semantične lastnosti iz ontologije in verodostojne podatke iz dokazil.

Sistem tako deluje skozi jasno razmejene sklope, kar je razvidno iz celotnega toka na diagramu – od zahteve, preko izračuna, do zapisa končnega

rezultata zaupanja na verigi blokov.



Slika 3.2: Kontekstni pogled sistema za upravljanje zaupanja.

V nadaljevanju so podrobneje opisane tri plasti, ki sestavljajo arhitekturo:

1. **Plast znanja (Knowledge Layer)** V tej plasti se nahaja ontologija dobavne verige in opisne lastnosti akterjev. Ontologija predstavlja skupni referenčni model, ki omogoča enotno interpretacijo podatkov o akterjih. Poleg ontologije se tukaj nahaja tudi skladišče preverljivih dokazil (VC), ki dodaja podatke o licencah, certifikatih in presojah kakovosti.
2. **Plast izračuna zaupanja (Trust Resolution Layer)** Oraklji v tej plasti izvajajo izračune zaupanja. Podatke pridobijo iz ontologije in preverljivih dokazil ter jih povežejo s pravili, ki jih določi posamezen akter. Pravila so zapisana v JSON obliki in opisujejo pogoje, ki jih mora ocenjevana entiteta izpolnjevati. Oraklji izvedejo izračun, rezultat podpišejo in ga posredujejo agregatorju.
3. **Plast registra zaupanja na verigi blokov (Blockchain Trust Registry Layer)** Ta plast vključuje pametno pogodbo, ki hrani rezultate ocenjevanja zaupanja. Pogodba deluje kot decentraliziran register, ki omogoča vpogled v trenutna in pretekla razmerja zaupanja med akterji. Vanjo se vpisujejo tako posamezni odgovori orakljev kot konsenzni rezultat evalvacije.

V nadaljevanju so opisane tehnične podrobnosti posamezne plasti ter njihove medsebojne povezave.

Plast znanja

Plast znanja predstavlja osnovni vir informacij, na katerem poteka izračun zaupanja. Vsebuje semantični model dobavne verige zdravil, podatke o posameznih akterjih ter dodatna preverljiva dokazila, ki opisujejo regulatorne lastnosti ali skladnost z zahtevami. V tej plasti se združujejo podatki iz dveh virov: ontologije, ki opisuje akterje in njihove lastnosti, ter preverljivih

dokazil (VC), ki dopolnjujejo semantični model z digitalno podpisanimi trditvami. Oraklji te podatke uporabljajo pri izračunu zaupanja v nadaljnjih plasteh sistema.

Ontologija (OWL/RDF) Ontologija farmacevtske dobavne verige je napisana v jeziku OWL in zapisana v formatu RDF. Vsebuje razrede, ki predstavljajo posamezne tipe akterjev, kot so *Proizvajalec*, *Prevoznik*, *Lekarna* in *Regulator*. Tem razredom so pripisane lastnosti, ki opisujejo njihovo vedenje ali skladnost s standardi, na primer:

- **hasDeliveryPunctuality** – delež pravočasnih dostav,
- **hasTempViolationRate** – delež temperaturnih odstopanj pri prevozu,
- **hasLicense** – veljavnost poslovne licence,
- **hasGMP** – skladnost s standardi GMP [36],
- **hasAuditScore** – rezultat presoje kakovosti.

Ontologija vsebuje tudi posamezne primerke akterjev, na primer *Pfizer* ali *DHL*, ki imajo pripisane konkretne vrednosti lastnosti. Spodaj je prikazan primer zapisa prevoznika v obliki RDF/Turtle:

```
ex:DHL a ex:Transporter ;  
  ex:hasDeliveryPunctuality "0.94" xsd:decimal ;  
  ex:hasTempViolationRate "0.02" xsd:decimal ;  
  ex:hasLicense "true" xsd:boolean .
```

Ontologija je shranjena na strežniku Apache Jena Fuseki, ki omogoča izvajanje poizvedb tipa SPARQL 1.1 preko spletnega vmesnika. Oraklji tako dostopajo do podatkov z neposrednimi poizvedbami, kot je na primer naslednja poizvedba za pridobitev lastnosti prevoznika DHL:

```
PREFIX ex: <http://example.org/trust#>

SELECT ?punctuality ?tempRate ?license
WHERE {
    ex:DHL ex:hasDeliveryPunctuality ?punctuality .
    ex:DHL ex:hasTempViolationRate ?tempRate .
    ex:DHL ex:hasLicense ?license .
}
```

Ker je ontologija zgrajena v OWL, omogoča tudi uporabo sklepalnika (angl. reasoner). Sklepalnik lahko samodejno razvrsti akterje v razrede (npr. **TrustedTransporter**), zazna nedoslednosti in izpelje lastnosti, ki niso zapisane neposredno. Vsak orakelj lahko sklepalnik po potrebi zažene lokalno in s tem pridobi izpeljane podatke.

Preverljiva dokazila (VC) Poleg podatkov iz ontologije sistem uporablja še preverljiva dokazila (VC), ki jih izdajajo regulatorji ali druge zaupanja vredne organizacije. Ta dokazila vsebujejo trditve o akterjih, kot so veljavnost licence, GMP skladnost ali datum zadnje presoje kakovosti. Dokazila so zapisana v formatu JSON-LD in digitalno podpisana.

Primer dokazila, ki ga izda regulator:

```
{
  "id": "vc-gmp-pfizer-001",
  "type": ["VerifiableCredential", "GMPCertificate"],
  "issuer": "did:example:ema",
  "credentialSubject": {
    "id": "did:example:pfizer",
    "hasGMP": true
  },
  "proof": {
    "type": "Ed25519Signature2018",
    "signatureValue": "3f9ac..."
  }
}
```

Preverljiva dokazila so shranjena v ločeni podatkovni bazi. Ob izračunu zaupanja orakelj preveri kriptografski podpis izdajatelja, iz podatkov izlušči relevantne lastnosti in jih združi z informacijami iz ontologije. Tako sestavi popoln nabor kriterijev, ki jih akter uporabi pri ocenjevanju zaupanja.

Plast znanja združuje semantične opise akterjev in njihove lastnosti z regulatorno potrjenimi podatki iz preverljivih dokazil. Oraklji pri izračunu zaupanja dostopajo do obeh virov: iz Fusekija pridobijo opisna merila, iz VC-jev pa dodatne podatke, ki jih ontologija ne vsebuje. S tem plast znanja omogoča enoten in strukturiran nabor podatkov, na katerem temelji celoten izračun zaupanja.

Plast izračuna zaupanja

Plast izračuna zaupanja predstavlja osrednjo procesno komponento arhitekture. Njena naloga je, da podatke iz plasti znanja pretvori v konkretne odločitve o zaupanju in jih posreduje verigi blokov. V tej plasti se nahaja omrežje orakljev in agregator, ki skupaj tvorita neodvisen mehanizem sklepanja. Oraklji izvajajo izračune, agregator pa skrbi za preverjanje rezultatov in zapis končnega stanja v pametno pogodbo.

Oraklji Ko sistem prejme zahtevo za izračun zaupanja, pametna pogodba sproži dogodek, ki služi kot signal, da morajo oraklji izvesti novo oceno. Oraklji ne delujejo kot pasivni odjemalci, temveč aktivno spremljajo verigo blokov in zaznavajo zahteve, ki jih je treba obdelati. Za podani subjekt nato pridobijo vse podatke iz plasti znanja: opisne lastnosti iz ontologije ter pripadajoča preverljiva dokazila iz baze poverilnic VC. Na ta način je izračun zaupanja vedno vezan na najnovejše in preverljive podatke.

Pridobljene podatke orakelj združi s pravili ocenjevalca. Pravila določajo, katere lastnosti mora ocenjevan akter izpolnjevati in pod kakšnimi pogoji. Orakelj izvede ocenjevanje in pripravi strukturirano poročilo, ki vsebuje identifikator subjekta, končno odločitev, numerično oceno, kriptografske povzetke uporabljenih dokazil ter čas izračuna. Poročilo je serializirano v JSON obliki in digitalno podpisano z zasebnim ključem oraklja. S tem postane rezultat preverljiv in vezan na točno določen orakelj.

Agregator Podpisano poročilo orakelj pošlje agregatorju, ki deluje kot zbirni člen med oraklji in pametno pogodbo. Ob prejemu poročila preveri digitalni podpis in, če je poročilo prišlo od dovoljenega oraklja. Poročilo zapiše na verigo kot posamično oddajo. Ko prejme zadostno število poročil, izvede konsenz nad prejetimi odločitvami in oceno zaupanja ter prek transakcije posreduje končni rezultat v pametno pogodbo **TrustGraph**.

Na visoki ravni potek izračuna poteka v naslednjih korakih:

1. pametna pogodba na verigi blokov prejme zahtevo za nov izračun zau-

- panja in sproži dogodek,
2. oraklji poslušajo te dogodke, za podanega akterja preberejo podatke iz ontologije in pripadajočih preverljivih dokazil,
 3. podatke združijo s pravili ocenjevalca in izvedejo izračun zaupanja,
 4. rezultat pretvorijo v strukturirano poročilo, ga digitalno podpišejo in pošljejo agregatorju,
 5. agregator preveri prejeta poročila, izračuna konsenz in prek transakcij zapiše posamezne oddaje ter končni rezultat v pametno pogodbo.

Na ta način plast izračuna zaupanja povezuje semantične podatke z decentraliziranim zapisom rezultatov. Oraklji omogočajo neodvisno izvajanje izračunov, agregator pa poskrbi za preverjanje in enoten zapis v register zaupanja na verigi blokov. Plast torej tvori most med modelom znanja in verigo blokov ter predstavlja ključen element celotne arhitekture.

Plast registra zaupanja na verigi blokov

Plast registra zaupanja predstavlja končni sloj arhitekture, kjer se rezultati izračuna zaupanja trajno zapišejo. Njena vloga je zagotoviti podatkovno plast, ki ni odvisna od posamezne organizacije in ki omogoča preverljiv, dosleden ter nespremenljiv zapis vseh odločitev o zaupanju. V tej plasti se nahaja pametna pogodba **TrustGraph**, ki deluje kot decentraliziran register razmerij zaupanja med akterji.

Pametna pogodba je zasnovana tako, da sprejema rezultate evalvacij orakljev in jih zapisuje v podatkovne strukture na verigi. Podpisi orakljev, konsenz agregatorja in mehanizmi preverjanja identitet poskrbijo, da v register pridejo le preverjene in verodostojne informacije. Vsak zapis je povezan z identifikatorjem akterja, časom vnosa in dodatnimi metapodatki, kar omogoča transparenten zgodovinski vpogled v spremembe zaupanja skozi čas.

Ko agregator prejme dovolj veljavnih poročil orakljev, izvede postopek združevanja in prek transakcije pokliče funkcijo v pametni pogodbi, ki zapiše

tako posamične oddaje kot končni konsenzni rezultat. Pogodba sproži dogodke, ki služijo kot signal akterjem ali nadrejenim sistemom, da je za določen subjekt na voljo nova ocena zaupanja. Vsaka sprememba je trajno zapisana v verigo blokov, zato je mogoče rezultate neodvisno preveriti tudi za nazaj.

Register zaupanja hrani dve vrsti podatkov. Prva vrsta so posamične oddaje orakljev, ki predstavljajo neobdelane rezultate izračuna. Druga vrsta so konsenzni rezultati, ki jih agregator izračuna na podlagi več oddaj in jih pogodba obravnava kot veljavno končno stanje zaupanja za posameznega akterja. Na ta način lahko sistem ohranja celovito sliko o tem, kako so se rezultati spreminjali in kako je bila sprejeta končna odločitev.

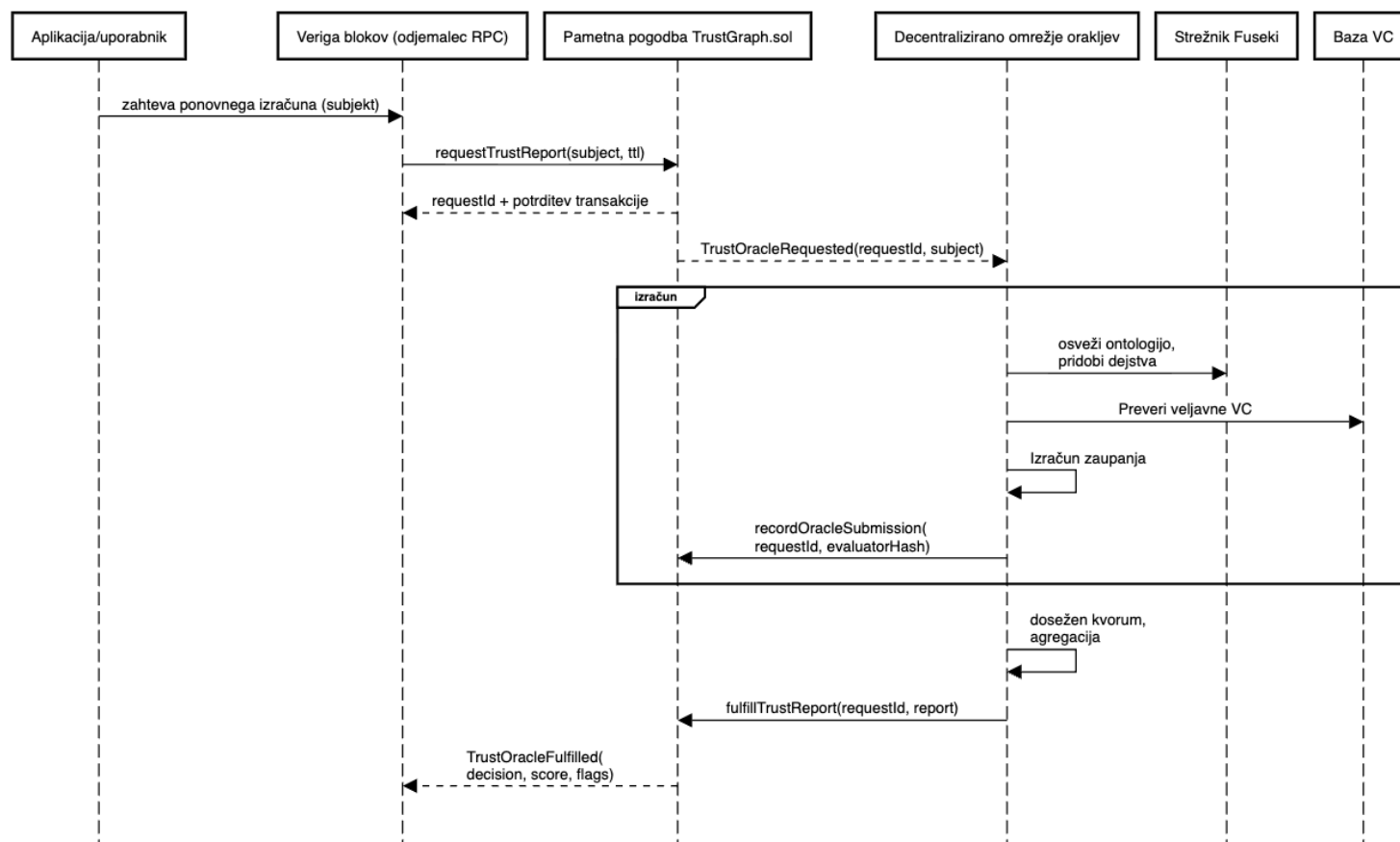
Zapis v register omogoča tudi poizvedovanje nad trenutnim ali zgodovinskim stanjem zaupanja. Akterji v dobavni verigi lahko kadar koli preverijo, ali je določen partner trenutno ocenjen kot zaupanja vreden ali ne. Ker podatki niso shranjeni v centraliziranem sistemu, temveč v verigi blokov, se vnosov ne da naknadno spreminjati ali brisati. To daje sistemu lastnost dolgoročne preverljivosti, kar je ključno v okoljih, kjer se sprejemajo odločitve z regulatornimi posledicami.

Plast registra zaupanja tako predstavlja stičišče celotnega sistema: prejme rezultate iz računske plasti, jih trajno zapiše in omogoča zanesljiv vpogled v relacije zaupanja v vsakem trenutku. V nadaljevanju bomo opisali funkcije pametne pogodbe **TrustGraph**, podatkovne strukture, dogodke ter način, kako zunanji akterji dostopajo do registra.

3.3 Tok podatkov v sistemu

3.3.1 Celoten potek izračuna zaupanja

Na sliki 3.3 je prikazan celoten proces izračuna zaupanja, ki vključuje vse komponente sistema: aplikacijo, pametno pogodbo **TrustGraph.sol**, decentralizirano omrežje orakljev, strežnik Fuseki ter bazo preverljivih dokazil VC. V nadaljevanju se osredotočimo na funkcije uporabljene v procesu. Pri vsakem koraku so navedene ključne funkcije.



Slika 3.3: Diagram poteka izračuna zaupanja za subjekt.

1. Aplikacija ali uporabnik ustvari zahtevo

Uporabnik preko aplikacije ali klica API sproži postopek za izračun zaupanja za zelenega akterja. Akterjev DID pred klicem funkcije pretvorimo v binarno vrednost `bytes32` z zgoščevalno funkcijo Keccak-256, ki je značilna za Ethereumov ekosistem. Aplikacija preko odjemalca RPC pokliče funkcijo na pametni pogodbi:

```
requestTrustReport(subject, ttl)
```

2. Pametna pogodba obdela zahtevo in izda signal za izračun

Po klicu funkcije `requestTrustReport(subject, ttl)` pametna pogodba preveri, ali je za izbranega akterja že aktivna zahteva ter ali ima klicatelj ustrezna dovoljenja (če je seznam dovoljenih nastavljen). Ko so pogoji izpolnjeni, pogodba ustvari nov identifikator zahteve `requestId`. Identifikator temelji na zgoščeni kombinaciji DID akterja, časovnega žiga, naslova pošiljatelja in zaporedne številke zahteve. Pogodba shrani rok za oddajo (`deadline`) in nato sproži dogodek:

```
TrustOracleRequested(requestId, subject)
```

Ta dogodek služi kot edini mehanizem obveščanja decentraliziranega omrežja orakljev, da je treba izvesti nov izračun zaupanja. Obenem funkcija klicatelju tudi vrne `requestId`, na podlagi katerega lahko spremlja stanje in kasneje pridobi končni rezultat.

3. Orakelj zazna novo zahtevo in začne proces izračuna

Vsak orakelj neprestano posluša za nove dogodke, ki jih proži pametna pogodba. Ko ta sproži dogodek `TrustOracleRequested`, orakelj preveri, ali mora obdelati zahtevo za podani `requestId`. Če je orakelj nastavljen za tovrstne izračune, začne postopek ocenjevanja. Najprej pridobi vse potrebne podatke iz plasti znanja: semantične lastnosti akterja preko poizvedb SPARQL strežniku Fuseki ter pripadajoča preverljiva dokazila iz baze VC. Orakelj tako zagotovi, da bo izračun temeljil na najnovejših, verodostojnih in digitalno podpisanih podatkih. Šele

ko so vsi podatki zbrani in preverjeni, orakelj preide na sam izračun zaupanja, kjer uporabi pravila ocenjevalca.

4. Orakelj osveži podatke v plasti znanja

Pred izračunom zaupanja mora orakelj pridobiti najnovejše podatke iz dveh ločenih virov:

- semantične lastnosti akterja iz ontologije prek poizvedb SPARQL na strežniku Fuseki,
- veljavna preverljiva dokazila (VC), ki jih orakelj prebere iz lokalne baze dokazil.

Pridobivanje podatkov iz ontologije poteka prek končne točke SPARQL, ki vrne lastnosti subjekta (npr. točnost dostav, licenčne statuse, skladnost GMP). Preverljiva dokazila se preverijo z validacijo digitalnega podpisa izdaje. S tem orakelj zagotovi, da izračun temelji na najnovejših in verodostojnih podatkih.

5. Izvedba izračuna glede na pravila ocenjevalca

Ko ima orakelj na voljo vse relevantne podatke, izvede izračun zaupanja. Pri tem uporabi pravila, ki si jih bomo v nadaljevanju pogledali na testnih scenarijih. Rezultat tega koraka je numerična ocena in binarna odločitev, ali akter izpolnjuje zahtevane pogoje. Poleg tega orakelj pripravi tudi metapodatke, kot so:

- čas izračuna (`asOf`),
- zgostitev pravil (`policyHash`),
- zgostitev uporabljenega VC (`credentialHash`).

6. Orakelj ustvari poročilo in ga podpiše

Orakelj pripravi končno poročilo v JSON obliki, ki vsebuje ključne informacije o izračunu zaupanja: identifikator subjekta, odločitev, oceno, zastavice in pripadajoče povzetke. Preden ga pošlje agregatorju, mora orakelj poročilo kriptografsko podpisati. Podpis se izvede z uporabo

metode `sign_message`, ki je del knjižnice `web3.py`. Ta temelji na Ethereumovi shemi EIP-191. Najprej se pripravi deterministična kanonična oblika JSON zapisa, ta pa se nato podpiše z zasebnim ključem oraklja. Podpis omogoča agregatorju, da preveri, ali je poročilo prišlo od zaupanja vrednega oraklja.

7. Agregator sprejme poročila in izvede preverjanje

Agregator deluje kot zbirno vozlišče, ki prejema poročila posameznih orakljev prek HTTP vmesnika. Pred prejemom poročila izvede naslednja preverjanja:

- ali podpis pripada dovoljenemu oraklju,
- ali poročilo ni podvojeno,
- ali se zgostitve pravil in dokazil ujemajo z lokalnimi podatki.

Vsako veljavno poročilo agregator takoj zapiše v pametno pogodbo z uporabo funkcije:

```
recordOracleSubmission(requestId, evaluatorHash,  
report, credentialHash)
```

Na ta način je posamezna ocena oraklja shranjena na verigi blokov, neodvisno od končnega konsenza.

8. Agregator izvede konsenz in zapiše končni rezultat

Ko agregator prejme dovolj veljavnih poročil (dosežen kvorum), izvede konsenz nad rezultati. Pri tem kombinira odločitve, oceno in dodatne zastavice, nato pa pripravi končno poročilo. Končni rezultat se zapiše v pametno pogodbo z uporabo funkcije:

```
fulfillTrustReport(requestId, report)
```

Pametna pogodba preveri, ali je agregator pooblaščen, ali je `requestId` veljaven in ali zahteva ni pretekla. Če so vsi pogoji izpolnjeni, pogodba zapiše konsenzni rezultat v verigo blokov in sproži dogodek

`TrustOracleFulfilled`. Pogodba preveri identiteto klica, ujemanje ID in iztek roka, preden zapiše končni rezultat v:

```
subjectMetrics[subject]
```

in sprožen je dogodek `TrustOracleFulfilled`, ki zaključi postopek.

Ker je celoten postopek sestavljen iz več medsebojno odvisnih korakov, se v naslednjih podpoglavjih osredotočimo na bolj podroben pregled posameznih delov. Najprej pogledamo notranje delovanje omrežja orakljev in agregacije rezultatov, nato pa še na branje podatkov iz registra zaupanja.

3.3.2 Agregiranje rezultatov v omrežju orakljev

Na sliki 3.4 je prikazan podroben potek obdelave poročil znotraj decentraliziranega omrežja orakljev (DON). Diagram prikazuje fazo po tem, ko oraklji prejmejo zahtevo za izračun zaupanja, in se osredotoča na izvajanje izračuna, podpisovanje poročil ter postopek agregacije v agregatorju. Ta proces je ključen za zagotavljanje verodostojnih rezultatov, saj združi poročila več neodvisnih orakljev v en sam, konsenzni izračun.

V nadaljevanju so opisani vsi ključni koraki, prikazani v diagramu.

1. Zajem podatkov iz plasti znanja

Orakelj pred izvedbo izračuna pridobi aktualne podatke iz dveh virov:

- podatki ontologije prek strežnika Fuseki (poizvedbe SPARQL), ki vključujejo opisna merila akterja,
- preverljiva dokazila (VC), ki jih orakelj preveri glede veljavnosti in morebitnega preklica.

Ti podatki predstavljajo vhodna merila, ki se uporabijo pri izračunu zaupanja.

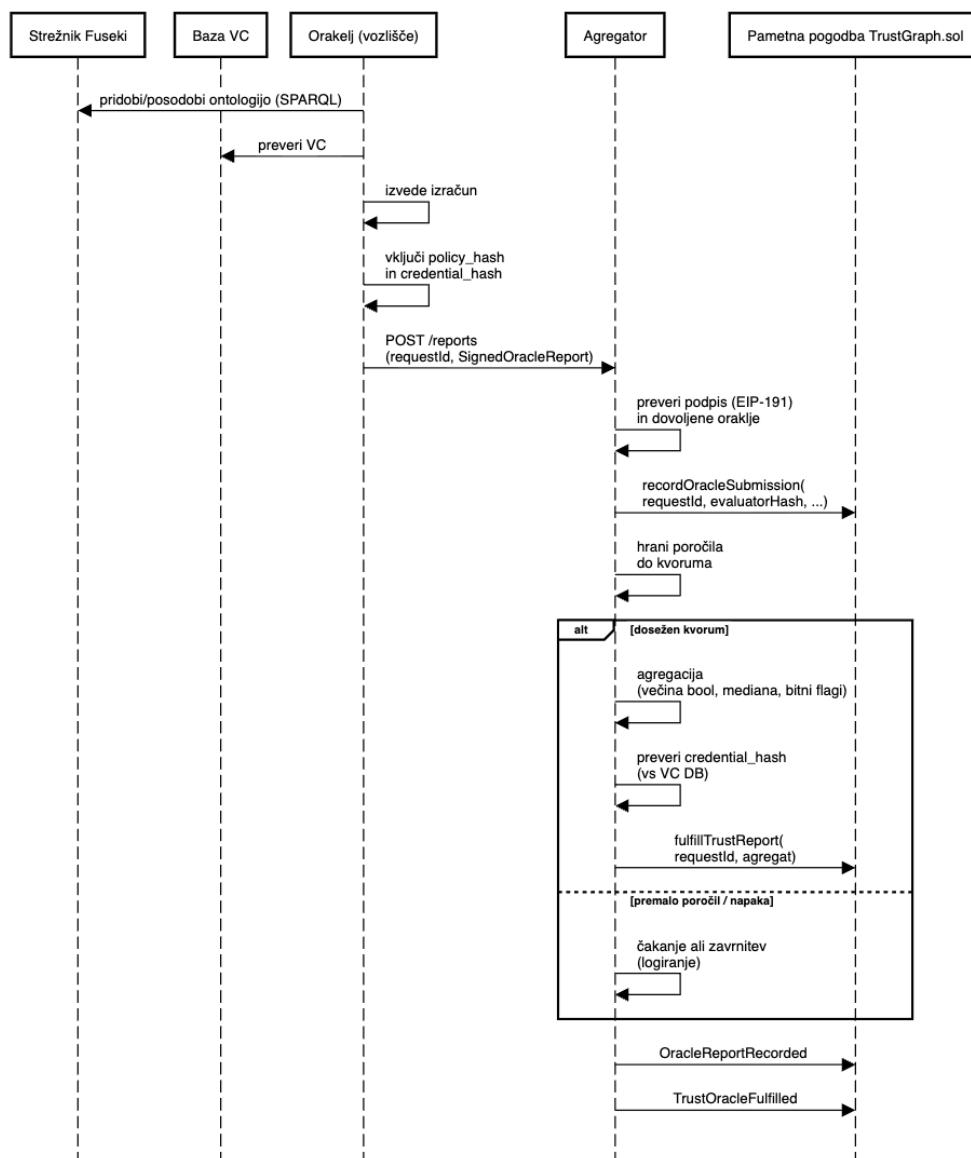
2. Izvedba lokalnega izračuna zaupanja

Orakelj izvede hibridni algoritem ocenjevanja, ki združuje preverjanje pravil ter statističnih meril. V tej fazi orakelj pripravi tudi oba ključna povzetka:

- `policy_hash`, ki predstavlja zgostitev pravil akterja,
- `credential_hash`, ki označuje VC dokazilo uporabljeno oceni.

3. Pošiljanje podpisanega poročila agregatorju

Orakelj oblikuje strukturirano poročilo, ga serializira v kanonični JSON zapis in digitalno podpiše z metodo `sign_message` (EIP-191). Podpisano poročilo nato pošlje agregatorju prek zahteve HTTP (`POST /reports`), skupaj z identifikatorjem zahteve. Da agregator poročilo



Slika 3.4: Potek agregiranja rezultatov v decentraliziranem omrežju orakljev

sprejme, mora orakelj poslati podatke v standardizirani obliki `Signed-OracleReport`. Ta vsebuje dva dela:

- `OracleReport` – dejanski rezultat izračuna,
- `node_id` in `signature` – identiteto oraklja in kriptografski podpis poročila.

Struktura `OracleReport` mora vsebovati naslednja polja:

- `subject` – 32-bajtni identifikator subjekta oz. zgostitev DID,
- `decision` – končna odločitev (bool),
- `score` – številčna ocena zaupanja v odstotkih,
- `flags` – bitna opozorilna polja,
- `as_of` – čas izračuna.

Pred podpisom se poročilo pretvori v kanonično JSON obliko s stabilnim urejanjem ključev. Podpis se izvede s funkcijo `sign_message` iz knjižnice `web3.py`. Agregator poročilo sprejme le, če je podpis veljaven, naslov `node_id` pripada dovoljenemu oraklju, poročilo ni podvojeno in `policy_hash` ter `credential_hash` se ujemata z lokalnimi podatki agregatorja. Standardizirana struktura omogoča, da lahko vsako poročilo neodvisno preverimo in da se rezultati več orakljev združijo v konsenzni izračun, ki se zapiše v pametno pogodbo.

4. Preverjanje podpisov in veljavnosti poročil

Agregator ob prejemu poročila preveri:

- ali podpis pripada dovoljenemu oraklju,
- ali je poročilo unikatno glede na kombinacijo vrednosti `request_id` in `node_id`,
- ali se zgostitve pravil in dokazil ujemajo z njegovimi lokalnimi podatki.

Veljavna poročila agregator takoj zapiše v pametno pogodbo s funkcijo `recordOracleSubmission`.

5. Doseganje kvoruma poročil

Agregator hrani prejeta poročila in spremlja, ali je dosežen kvorum (minimalno število poročil, potrebnih za izvedbo konsenza). Če ta ni dosežen, agregator čaka in nadaljuje zbiranje poročil dokler le ta ni dosežen ali do izteka časa čakanja (`ttl`).

6. Agregacija rezultatov znotraj DON

Ko je kvorum dosežen, agregator izvede agregacijo. Poleg odločitve in ocene agregator združi tudi `flags`, ki predstavljajo bitna opozorilna polja. Vsak bit ima posebej določen pomen, ki označuje posebne okoliščine ali tveganja, ki so se pojavila med izračunom:

- `FLAG_NO_DATA` (bit 0) – orakelj ni našel nobenih vhodnih podatkov za subjekt (npr. subjekt v ontologiji ne obstaja ali nima nobenih lastnosti).
- `FLAG_LOW_SCORE` (bit 1) – verjetnostna ocena zaupanja je padla pod minimalni prag.
- `FLAG_DISAGREEMENT` (bit 2) – različni deli orakljevega notranjega izračuna so podali neskladne odločitve.
- `FLAG_VC_REVOKED` (bit 3) – preverljivo dokazilo (VC), uporabljeno pri izračunu, je označeno kot preklicano.

Te bite oraklji nastavijo v funkcijah za izračun ocen. Agregator pri združevanju uporabi prвило večine: vsak bit v končnem rezultatu je nastavljen, če ga ima vsaj polovica oddanih poročil. Zastavice tako predstavljajo povzetek opozoril, ki jih je zaznalo celotno omrežje orakljev.

7. Zapis konsenznega rezultata v pametno pogodbo

Končni agregirani rezultat se v verigo blokov zapiše s funkcijo:

`fulfillTrustReport(requestId, report)`

Pametna pogodba preveri identiteto klicatelja, veljavnost identifikatorja zahteve in rok izvedbe. Ob uspešnem zapisu sproži dogodek `TrustOracleFulfilled`, ki drugim komponentam signalizira, da je nova ocena na voljo.

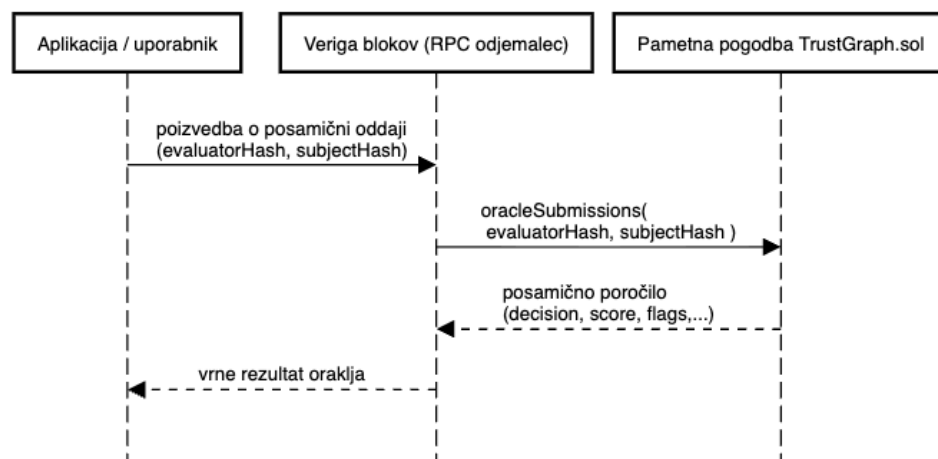
Agregator povezuje izračune več orakljev v enoten rezultat, pametna pogodba pa poskrbi, da je končni izračun trajno in preverljivo zapisan v verigi blokov. V naslednjem podpoglavju si bomo ogledali, kako akterji v sistemu pridobijo te rezultate in kako poteka branje podatkov iz registra zaupanja.

3.3.3 Branje rezultatov zaupanja

Po zaključenem izračunu in zapisu rezultatov v pametno pogodbo lahko akterji iz sistema pridobijo podatke o zaupanju na dva načina: (i) kot posamezne oddaje orakljev in (ii) kot agregiran, konsenzni rezultat omrežja orakljev (DON).

Branje posamičnih izračunov orakljev

Pametna pogodba `TrustGraph.sol` hrani posamične rezultate vsakega oraklja v podatkovni strukturi `oracleSubmissions[evaluator][subject]`. Ti podatki predstavljajo neagregirane rezultate, ki jih je posamezen orakelj izračunal za dani subjekt. Slika 3.5 prikazuje potek branja posamičnih oddaj.



Slika 3.5: Branje posamične oddaje oraklja iz registra zaupanja

Branje posamičnih oddaj je koristno za različne namene. Če je potrebno preveriti, kako je določen orakelj ocenil subjekt, ali pa analizirati morebitna neskladja med oraklji, lahko akterji neposredno dostopajo do teh podatkov. Do teh podatkov dostopamo z direktnim klicem funkcije za branje iz javnega slovarja:

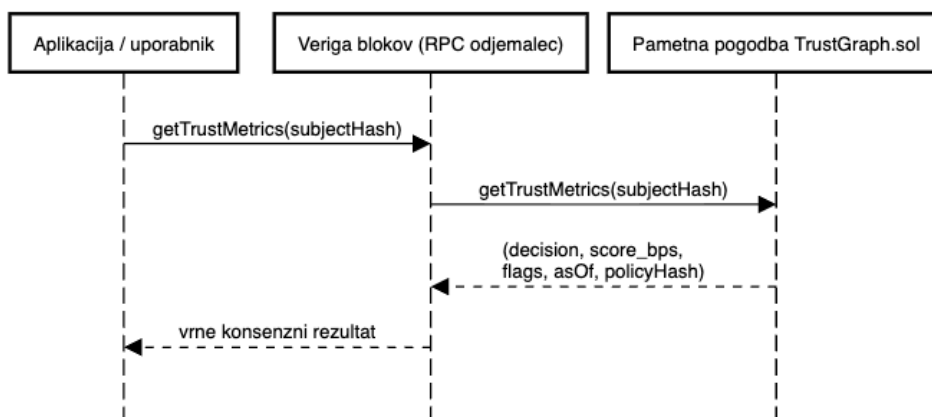
```
oracleSubmissions(evaluatorHash, subject)
```

Rezultat vključuje:

- binarno odločitev,
- numerično oceno,
- zastavice (flags),
- čas izračuna,
- zgostitve pravil in dokazil,
- naslov Ethereum oraklja.

Branje agregiranega (konsenznega) rezultata

Konsenzni rezultat, ki ga zapiše agregator, predstavlja končno in uradno odločitev omrežja orakljev.



Slika 3.6: Branje konsenznega rezultata ocenjevanja iz pametne pogodbe

Ta rezultat je v pametni pogodbi zapisan v strukturi:

```
subjectMetrics[subject]
```

Kot vidimo na sliki 3.6, lahko do rezultatov dostopamo prek javne funkcije:

```
getTrustMetrics(subject)
```


Ta funkcija vrne:

- **odločitev** (`decision`) – končni izračun zaupanja,
- **oceno** (`score_bps`) – numerična vrednost v osnovnih točkah,
- **zastavice** (`flags`) – bitna opozorila, ki so bila prisotna v večini oddaj,
- **čas** (`asOf`) – čas izračuna konsenza.

Konsenzni rezultat je tisti, ki ga uporabi večina akterjev v sistemu, saj predstavlja stabilen in preverjen izračun celotnega omrežja orakljev.

Poglavje 4

Evalvacija

V tem poglavju predstavimo celovito evalvacijsko analizo razvitega sistema za upravljanje zaupanja. Namen poglavja je prikazati, kako posamezne komponente delujejo skupaj v realističnih scenarijih in kako se sistem obnaša pod vplivom različnih vhodnih podatkov, politik ter sprememb v okolju. Evalvacija je zato zasnovana kot kombinacija eksperimentalne postavitve, podrobnega pregleda algoritmov orakljev in zaporedja scenarijev, ki ponazarjajo ključne procese: izračun zaupanja, agregacijo rezultatov ter odziv sistema na spremembe v podatkih.

Najprej opišemo eksperimentalno okolje in konfiguracijo vseh komponent v vsebnikih Docker, kar zagotavlja ponovljivost in transparentnost izvedbe. Nato predstavimo algoritme izračuna zaupanja, ki jih uporabljajo trije oraklji, zasnovani na različnih konceptualnih modelih: hibridnem semantičnem pristopu, dokaznem modelu, temelječem na preverljivih poverilnicah, ter telemetrijsko usmerjenem pristopu. V osrednjem delu poglavja sledijo podrobno izpeljani scenariji, ki prikazujejo delovanje sistema od začetne zahteve do končnega zapisa na verigo, skupaj z dejanskimi številčnimi rezultati posameznih orakljev ter agregatorja. Poglavje zaključimo z evalvacijo odziva sistema na spremembe vhodnih podatkov in s tem pokažemo, kako se ocena zaupanja dinamično prilagodi novim razmeram. Celotna predstavitev tako omogoča sistematičen vpogled v delovanje rešitve, njeno zanesljivost ter transparen-

tnost postopka izračuna zaupanja.

4.1 Predstavitev scenarijev delovanja

4.1.1 Eksperimentalno okolje in konfiguracija sistema

Preden predstavimo konkretne scenarije uporabe, najprej opišemo, kako je postavljen eksperimentalni sistem. Vse komponente zaganjamo v orodju **Docker**, kar nam zagotavlja ponovljivo okolje in jasno ločitev med posameznimi storitvami. S tem zmanjšamo odvisnost od lokalne konfiguracije in omogočimo, da sistem testiramo kakor, da bi bile vse komponente že nameščene v oblaku.

Vsebniki in omrežje

Sistem sestavlja več vsebnikov **Docker**, ki so povezani v skupno virtualno omrežje in delujejo z orodjem **Docker Compose**. V podrazdelku 4.1.2 so opisani posamezni vsebniki in načini ocenjevanja zaupanja za vsak orakelj.

Veriga blokov Prvi vsebnik vsebuje testno Ethereum omrežje v orodju **Anvil**. Na njem namestimo pametno pogodbo **TrustGraph.sol**. Na omrežju takoj dobimo tudi 10 prednastavljenih računov z velikimi količinami testnih kovancev **ETH**, ki jih oraklji in agregator uporabljajo za plačevanje transakcijskih stroškov. Zato prve račune dodelimo agregatorju in orakljem. Potrebujemo tudi en račun za namestitev pametne pogodbe, katerega dodelimo regulatorju **EMA**. Ta račun bo imel potem tudi pravico do sprožanja novih izračunov zaupanja.

Baza znanja V vsebniku **Fuseki** je naložena domenska ontologija. Ta je dostopna preko končne točke **SPARQL**, ki jo oraklji uporabljajo za pridobivanje semantičnih lastnosti akterjev. V eksperimentalnem okolju uporabljamo ontologijo, ki si jo vsi oraklji delijo.

Oraklji Za evalvacijo smo pripravili tri neodvisne oraklje, ki izvajajo svoje politike izračuna zaupanja. Vsak orakelj se izvaja v svojem vsebniku Docker in je konfiguriran z ločeno datoteko, ki določa pravila ocenjevalca, nabor vhodnih podatkov ter težo posameznih kriterijev. Oraklji so implementirani v Pythonu in uporabljajo knjižnice za dostop do verige blokov (`web3.py`), baze znanja (`rdflib`, `SPARQLWrapper`) ter za kriptografske operacije. V podrazdelku 4.1.2 so podrobneje predstavljeni algoritmi izračuna zaupanja za vsak orakelj.

Agregator Zbiranje poročil posameznih orakljev se izvaja v vsebniku agregatorja. Ta preverja njihove podpise in objavi končno odločitev v pametni pogodbi na verigi. Tudi agregator je implementiran v Pythonu in uporablja podobne knjižnice kot oraklji.

Odjemalska aplikacija V našem primeru odjemalska aplikacija nastopa kot ukazna vrstica oziroma skripta, s katero sprožimo zahtevo za izračun zaupanja in kasneje preberemo rezultat z verige. V realnem primeru bi bil to lahko npr. spletni vmesnik ali integracija v obstoječi ERP sistem akterja.

Kontejnerji si delijo notranje Docker omrežje, zato med seboj komunicirajo preko domen DNS (`fuseki:3030`, `ethereum-node:8545`).

Agregator pri združevanju rezultatov ne pozna podrobnosti posameznih politik. Vsako poročilo obravnava kot podpisano izjavo oraklja, ki vsebuje identifikator subjekta, numerično oceno zaupanja, diskretno odločitev (npr. *zaupanja vreden* ali *nezaupanja vreden*) ter morebitne zastavice. Na osnovi konfiguriranega praga in števila prejetih poročil izračuna končno odločitev, ki jo zapiše v pametno pogodbo.

Telemetrijski podatki Poleg ontologije in preverljivih poverilnic sistem uporablja tudi agregirane telemetrijske podatke za izbrane prevoznike. V eksperimentalnem okolju so ti podatki zapisani v JSON datoteki v naslednji obliki:

```
{
  "http://example.org/trust#DHL": {
    "tempDeviation": 0.02,
    "lateDeliveries": 0.01,
    "alerts": 0
  },
  "http://example.org/trust#MedLogix": {
    "tempDeviation": 0.03,
    "lateDeliveries": 0.02,
    "alerts": 0
  },
  "http://example.org/trust#UPS": {
    "tempDeviation": 0.04,
    "lateDeliveries": 0.03,
    "alerts": 1
  },
  "http://example.org/trust#AcmeTransport": {
    "tempDeviation": 0.08,
    "lateDeliveries": 0.05,
    "alerts": 3
  }
}
```

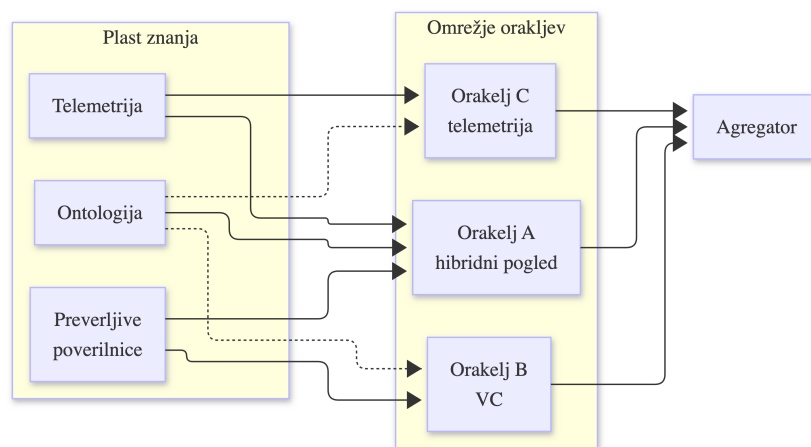
V prototipu so vrednosti zapisane statično in služijo kot simulacija. V realnem okolju bi enak format predstavljal projekcijo podatkov iz zunanjih sistemov (npr. transportnega informacijskega sistema, IoT senzorjev povezanih na verigo, itn.), ki bi jih oraklji periodično brali prek API-jev ali sporočilnih vodil in na njihovi podlagi sproti posodabljali oceno zaupanja.

4.1.2 Algoritmi izračuna zaupanja v orakljih

V poglavju 2.2 smo predstavili različne pristope k upravljanju zaupanja v razpršenih sistemih: dokazne modele, priporočilne modele, porazdeljeno oce-

njevanje ter dinamično posodabljanje zaupanja. Ti koncepti tvorijo teoretično podlago za zasnovo naših orakljev. V nadaljevanju pokažemo, kako vsak orakelj uporabi drugačen pogled na zaupanje. Za potrebe testiranja smo ustvarili tri oraklje z različnimi algoritmi izračuna zaupanja:

- **Orakelj A** uporablja hibridni pristop, ki kombinira dokazne elemente (licence, digitalna potrdila) z dinamičnimi merili. Podobno, kot je to narejeno v priporočilnih in porazdeljenih modelih zaupanja.
- **Orakelj B** je izrazito *dokazni* – odloča skoraj izključno na podlagi preverljivih poverilnic, podobno kot hierarhični in sistem PGP opisanih v podrazdelku 2.2.1.
- **Orakelj C** je *telemetrijsko usmerjen* in se naslanja na dinamično posodabljanje zaupanja prek operativnih meril, kar je skladno z modeli, ki večjo težo dajejo nedavnim interakcijam.



Slika 4.1: Viri podatkov za posamezne oraklje v plasti znanja

Na sliki 4.1 je prikazano, iz katerih virov črpajo podatke posamezni oraklji. Orakelj A ima hibriden pogled: pri izračunu zaupanja hkrati upošteva telemetrijo, preverljive poverilnice VC in podatke iz ontologije. Orakelj B pri odločanju uporablja samo preverljive poverilnice VC, medtem ko Orakelj C

izhaja predvsem iz telemetrijskih podatkov. Vsi trije oraklji ontologijo uporabljajo kot skupni imenik akterjev, iz katerega pridobijo njihove identifikatorje in osnovne oznake.

Skupna struktura izračuna

Ne glede na specifičen pogled vsak orakelj za dani subjekt izračuna:

1. deterministični del (pravila iz politik),
2. verjetnostni del (številsko verjetnostna ocena),
3. kombinirano odločitev.

Deterministični del. Deterministični del temelji na politikah, ki so zapisane v konfiguracijskih datotekah (`policy.json`). V njih so za posamezne tipe subjektov (proizvajalec, distributer, transporter) definirani kriteriji in pragovi, na primer:

- `hasDeliveryPunctuality` $\geq 0,99$,
- `hasTempViolationRate` $\leq 0,05$,
- `hasAuditScore` $\geq 0,90$,
- `hasGDPVC` = `true`.

Če subjekt za vse zahtevane kriterije zadošča pogojem, deterministični del vrne odločitev `PASS`, sicer `FAIL`. Ta del je najboljši približek *dokaznim modelom zaupanja*, kjer se odločitve sprejemajo na podlagi preverljivih dokazov in jasno določenih pragov.

Verjetnostni del. Verjetnostni del iste kriterije pretvori v številsko verjetnostno oceno v intervalu $[0, 1]$, ki jo orakelj kasneje pretvori v osnovne točke med 0 in 10000. Za vsak kriterij se vodi aposteriorni spremenljivki (α in β) porazdelitve beta $B(\alpha, \beta)$, ki modelira zaupanje za izpolnitev tega kriterija.

Uporaba porazdelitve beta je definirana na intervalu $[0, 1]$. V našem parameter α predstavlja število zaželenih oziroma pozitivnih opažanj danega kriterija, parameter β pa število negativnih opazovanj. Razmerje $\frac{\alpha}{\alpha+\beta}$ tako predstavlja pričakovano vrednost zaupanja. Aposteriorna porazdelitev pomeni, da se parametra α in β ne obravnavata kot statična, temveč se sproti posodabljata na podlagi novih opazovanj. Vsaka nova interakcija prispeva informacijo o tem, ali je bil kriterij izpolnjen na zaželen način. S tem novejša opazovanja v modelu dobijo večjo težo kot starejša, kar odraža dejansko obnašanje akterjev skozi čas.

Pri prvem branju ontologije se kriterij inicializira na osnovi trenutne vrednosti x in normalizirane zaželenosti $s_0 \in [0, 1]$ (višja vrednost pomeni bolj zaželen izid). Parametra porazdelitve beta sta:

$$\alpha = 1 + s_0 \cdot \text{obs_scale}, \quad (4.1)$$

$$\beta = 1 + (1 - s_0) \cdot \text{obs_scale}, \quad (4.2)$$

kjer je v implementaciji uporabljena utežni faktor $\text{obs_scale} = 9$. Za »popolno« izpolnjen pogoj ($s_0 \approx 1$) dobimo pričakovano vrednost porazdelitve beta $\mathbb{E}[X] \approx 0,91$, kar predstavlja visoko zaupanje.

Pri novih opazovanjih (interakcijah) se aposterior posodobi po principu eksponentno uteženega drsečega povprečja (EWMA) s faktorjem $\gamma \approx 0,2$. Če opazovanje prinese zaželenost s , se nova parametra α' in β' izračunata kot:

$$\alpha' = (1 - \gamma) \cdot \alpha + \gamma \cdot (1 + s \cdot \text{obs_scale}), \quad (4.3)$$

$$\beta' = (1 - \gamma) \cdot \beta + \gamma \cdot (1 + (1 - s) \cdot \text{obs_scale}). \quad (4.4)$$

S tem pristopom nove informacije hitreje vplivajo na verjetnostno oceno, starejše pa postopoma izgubljajo težo. To je konceptualno skladno z modeli dinamičnega posodabljanja zaupanja z amortizacijskimi faktorji, kjer novejše interakcije prevladajo nad starimi.

Kombinirana odločitev. Za posameznega akterja se končna verjetnostna ocena `prob_score` izračuna kot uteženo povprečje srednjih vrednosti beta

aposteriorjev ($\frac{\alpha}{\alpha+\beta}$) vseh aktivnih kriterijev. Uteži w_i , zapisane v pravih ocenjevalca (v našem primeru oraklja), so normalizirane tako, da velja $\sum w_i = 1$.

Kombinirana odločitev združuje trda pravila z mehko oceno. Če orakelj uporablja prag `prob_threshold`, je odločitev:

$$\text{combined} = \text{deterministic} \vee (\text{prob_score} \geq \text{prob_threshold}).$$

Na ta način se trda pragovna pravila (dokazni pogled) kombinirajo z mehko, številsko oceno, ki je bližje priporočilnim in porazdeljenim modelom zaupanja.

V nadaljevanju opišemo konkretne konfiguracije in primere za omenjene oraklje, ki uporabljajo različne pristope, opisane v razdelku 2.2.

Orakelj A (hibridni pogled)

Orakelj A uporablja hibridni pogled, ki združuje regulatorne kriterije, kakovost delovanja in informacije iz preverljivih poverilnic. S tem v praksi kombinira dokazni model (digitalna dokazila, licence) z dinamičnim ocenjevanjem in uteženjem kriterijev preko Beta posteriorjev. Njegov prag za verjetnostno oceno je nastavljen na:

$$\text{prob_threshold}^{(A)} = 0,75.$$

Kriteriji in uteži. Za posamezne tipe subjektov uporablja naslednje kriterije in uteži:

- **Transporter:**

- `hasDeliveryPunctuality` $\geq 0,99$ (utež 0,7),
- `hasTempViolationRate` $\leq 0,05$ (utež 0,3).

- **Distributor:**

- `hasLicense` = `true` (utež 0,4),

– `hasGDPVC = true` (utež 0,5).

- **Manufacturer:**

– `hasAuditScore  $\geq$  0,90` (utež 0,6).

Primer: MedLogix (distributer) Za distributerja MedLogix v trenutni ontologiji velja `license=true` in `GDPVC=true`. Deterministični del zato vrne `PASS`. Oba kriterija imata visoko zaželenost, zato se Beta posteriorja inicializirata s srednjo vrednostjo približno 0,91. Po normalizaciji uteži (približno 0,44 in 0,56) dobimo verjetnostno oceno:

$$\text{prob_score} \approx 0,91 \geq 0,75.$$

Kombinirana odločitev je:

$$\text{combined}^{(A, \text{MedLogix})} = \text{PASS}.$$

Primer: SkyFreight (transporter) Za transporterja SkyFreight sta za beleženi vrednosti `punctuality = 0,90` in `tempViolationRate = 0,08`. Deterministični pogoji niso izpolnjeni, ker $0,90 < 0,99$ in $0,08 > 0,05$, zato deterministični del vrne `FAIL`. Tudi normalizirana zaželenost s_0 za oba kriterija je nizka, kar vodi v nizko srednjo vrednost Beta posteriorja in verjetnostno oceno $\text{prob_score} < 0,75$. Končna odločitev oraklja je:

$$\text{combined}^{(A, \text{SkyFreight})} = \text{FAIL}.$$

Orakelj B (dokazni model)

Orakelj B je zasnovan tako, da v ospredje postavlja preverljive poverilnice. Konceptualno je najbližje dokaznim modelom zaupanja iz poglavja 2.2, kjer imata veriga potrdil in kriptografska dokazila ključno vlogo pri inicializaciji zaupanja. Njegov prag za verjetnostno oceno je strožji:

$$\text{prob_threshold}^{(B)} = 0,8.$$

Kriteriji in uteži. Glavne kombinacije kriterijev so:

- **Distributor:**

- `hasGDPVC = true` (utež 0,6),
- `hasLicense = true` (utež 0,3),

kar po normalizaciji daje približno 0,67 in 0,33.

- **Manufacturer:**

- `hasAuditScore` $\geq 0,92$ (utež 0,4).

- **Transporter:**

- `hasRecallRate` $\leq 0,02$ (utež 0,5).

Ključna značilnost je, da brez ustreznih preverljivih poverilnic odločitev nagiba k **FAIL**, tudi če so druga operativna merila razmeroma dobra. To odraža močan poudarek na formalnih dokazih in skladnosti.

Primer: MedLogix (distributor) Ker ima MedLogix veljavne poverilnice in licenco, deterministični del vrne **PASS**. Za oba kriterija dobimo srednjo vrednost Beta posteriorja približno 0,91, uteži so po normalizaciji 0,67 in 0,33, zato:

$$\text{prob_score} \approx 0,91 \geq 0,8,$$

in kombinirana odločitev ostane:

$$\text{combined}^{(\text{B}, \text{MedLogix})} = \text{PASS}.$$

Primer: HealthChain (transporter) Za transporterja HealthChain je v bazi znanja zabeležen `recallRate` = 0,015, kar deterministično zadošča pogoju `hasRecallRate` $\leq 0,02$ in daje **PASS**. Pri verjetnostnem delu pa je zaželenost pri vrednosti blizu zgornje meje nizka (npr. $s_0 \approx 0,25$), kar vodi v srednjo vrednost Beta posteriorja približno 0,30 in:

$$\text{prob_score} \approx 0,30 < 0,8.$$

Kombinirana odločitev je zato:

$$\text{combined}^{(\text{B,HealthChain})} = \text{FAIL},$$

kar ilustrira strožji, dokazni pristop: deterministični PASS sam po sebi ni zadosten brez visoke verjetnostne ocene.

Orakelj C (telemetrijski pogled)

Orakelj C uporablja telemetrijske podatke med prevozi zdravil (npr. temperatura, vlaga) kot primarni vir informacij. Konceptualno se približa porazdeljenim in dinamičnim modelom zaupanja iz poglavja 2.2, kjer se zaupanje gradi na osnovi kumulativnih izkušenj in se sproti prilagaja novim dogodkom. Njegov prag za verjetnostno oceno je:

$$\text{prob_threshold}^{(\text{C})} = 0,7.$$

Kriteriji in uteži Za proizvajalce in distributerje uporablja naslednje kriterije:

- **Manufacturer:**

- $\text{hasTempDeviationScore} \leq 0,10$ (utež 0,5),
- $\text{hasGDPVC} = \text{true}$ (utež 0,4).

- **Distributor:**

- $\text{hasDeliveryPunctuality} \geq 0,95$ (utež 0,5).

Pri transporterjih ta orakelj v trenutni konfiguraciji ne izračunava ocene, saj zanje ni definiranih kriterijev; v takem primeru subjekt v tem pogledu ni ocenjen.

Primer: BioPharm (proizvajalec) Za proizvajalca BioPharm sta zabeležena `tempDev = 0,08` in `GDPVC=true`. Oba pogoja sta deterministično izpolnjena (PASS), zaželenost je visoka in oba kriterija dobita srednjo vrednost posteriorja beta približno 0,91. Verjetnostna ocena je zato:

$$\text{prob_score} \approx 0,91 \geq 0,7,$$

kombinirana odločitev pa:

$$\text{combined}^{(C, \text{BioPharm})} = \text{PASS}.$$

Primer: CanaryLabs (proizvajalec) Za proizvajalca CanaryLabs sta zabeležena `tempDev = 0,12` in `GDPVC=false`. Deterministični del zato vrne FAIL. Ker je temperaturno odstopanje prevelike in ni poverilnice o skladnosti, je tudi verjetnostna ocena nizka (`prob_score` $\ll$ 0,7). Kombinirana odločitev je:

$$\text{combined}^{(C, \text{CanaryLabs})} = \text{FAIL}.$$

Takšna predstavitev treh orakljev neposredno gradi na modelih iz poglavja 2.2: Orakelj A predstavlja hibridni semantičen in dinamično podprt pristop, Orakelj B predstavlja strogo dokazno-kriptografski pogled, Orakelj C pa dinamično, na meritvah temeljeno ocenjevanje. V scenarijih, ki sledijo, bomo za izbrani subjekt pokazali, kako različni algoritmi privedejo do različnih, a razločljivih odločitev o zaupanju.

4.1.3 Agregacija poročil orakljev

V scenarijih, predstavljenih v nadaljevanju, vedno nastopi enak vzorec: za določenega akterja (npr. transportno podjetje ali distributerja) pametna pogodba `TrustGraph.sol` sproži zahtevo po poročilu o zaupanju, oraklji A, B in C izračunajo lastne ocene in jih pošljejo agregatorju, ta pa iz treh poročil izpelje enotno odločitev, ki jo zapiše v verigo blokov.

V opisu scenarijev bomo tabelarično pokazali, kako so posamezne ocene orakljev A, B in C videti pred agregacijo, ter kako je iz njih izpeljana enotna odločitev, ki je na voljo ostalim akterjem v dobavni verigi.

Pravila agregacije

Za dane tri (ali več) veljavnih poročil agregator uporabi preprosta deterministična pravila:

Skupni subjekt in politika Najprej preveri, ali so vsa poročila skladna glede na tri kriterije:

- vsa poročila veljajo za isti subjekt (isti zgoščeni identifikator),
- vsa poročila uporabljajo enak povzetek politike (`policy_hash`),
- povzetek poverilnic (`credential_hash`) ni v nasprotju s shrambo poverilnic (če ta obstaja).

Če katero od teh preverjanj pade, agregator poročil ne združi in zahteva v scenariju velja za neuspešno.

Diskretna odločitev (da/ne) Vsak orakelj poda diskretno odločitev, ali je subjekt zaupanja vreden ali ne. Agregator uporabi preprosto večinsko pravilo:

$$\text{odločitev} = \begin{cases} \text{true}, & \text{če vsaj dva od treh orakljev poročata true,} \\ \text{false}, & \text{sicer.} \end{cases}$$

V scenarijih to pomeni, da bo subjekt označen kot zaupanja vreden, če se vsaj dva od treh pogledov (hibridni, dokazni, telemetrijski) strinjata, da so pogoji izpolnjeni.

Številčna ocena zaupanja Vsako poročilo vsebuje številčno oceno zaupanja v razponu $[0, 10000]$. Agregator v trenutni verziji uporablja kar povprečje ocen vseh orakljev.

Povprečje je končna številčna ocena zaupanja, ki se zapiše v pametno pogodbo in jo v scenarijih prikazujemo kot *agregirano* vrednost.

Zastavice (flags) Oraklji z zastavicami označijo posebna stanja, npr. pomanjkanje podatkov, nizko oceno ali uporabo preklicanih poverilnic. Agregator za vsak posamezen bit zastavice preveri, ali ga nastavi vsaj polovica orakljev. Če je tako, je ta bit nastavljen tudi v skupnem rezultatu. V scenarijih to pomeni, da se opozorila, na katera opozori večina orakljev, prenesejo tudi v končno oceno, opozorila posameznega “osamelca” pa lahko ostanejo samo v njegovem individualnem poročilu.

Zapis agregirane odločitve na verigo

Ko je agregacija uspešna, agregator za dani `request_id` izvede klic `fulfill-TrustReport` v pametni pogodbi `TrustGraph.sol`. V verigo blokov se zapiše:

- identifikator subjekta (zgoščena identiteta),
- agregirana diskretna odločitev (zaupanja vreden da/ne),
- agregirana številčna ocena zaupanja,
- agregirane zastavice (večinska opozorila),
- časovni žig in povzetek politike.

V scenarijih, ki sledijo, bomo zato vedno prikazali dva nivoja:

1. *posamična poročila* orakljev A, B in C (njihove odločitve, ocene in zastavice),
2. *agregirano odločitev*, ki jo agregator zapiše na verigo in jo ostali akterji uporabljajo kot referenčno oceno zaupanja za dani subjekt.

Tako je jasno razvidno, kako se različni algoritmi (hibridni, dokazni, telemetrijski) zlivajo v eno, soglasno oceno, ki je dosegljiva vsem udeležencem dobavne verige.

4.1.4 Scenarij 1: Ocena zaupanja transporterja DHL

V prvem scenariju prikažemo celoten potek ocene zaupanja za konkretnega akterja v dobavni verigi: transportno podjetje DHL, ki skrbi za prevoz zdravil med proizvajalci, distributerji in lekarnami. Namen scenarija je pokazati, kako trije različni pogledi (Orakelj A, B in C) vodijo do deloma različnih rezultatov in kako agregator iz njih izpelje enotno, na verigi zapisano odločitev.

Opis izhodišča

Za subjekt DHL v trenutni konfiguraciji veljajo naslednja dejstva:

- v ontologiji je označen kot *transporter*,
- za njega obstaja veljavna preverljiva poverilnica skladnosti (VC), kar je razvidno iz neničelnega povzetka poverilnic (`credential_hash`),
- poverilnica je shranjena v skupnem repozitoriju VC na strežniku, do katerega imajo dostop vsi oraklji,
- telemetrijski podatki kažejo visoko pravočasnost dostav in sprejemljiva temperaturna odstopanja v hladni verigi.

Na tej osnovi:

- Orakelj A (hibridni model) kombinira ontologijo, politike in VC ter pri DHL zazna nekaj slabših kazalnikov (zato vrne precej nižjo številčno oceno kot preostala oraklja, odločitev pa ostane negativna).
- Orakelj B (dokazni, VC model) preverja skoraj izključno prisotnost veljavne poverilnice (ki je dostopna vsem orakljem) in DHL oceni zelo visoko.
- Orakelj C (telemetrija) uporablja predvsem telemetrijo in kombinacijo kazalnikov točnosti ter temperature in vrne pozitivno odločitev z visoko, a nekoliko nižjo oceno kot Orakelj B.

Potek izvedbe scenarija

Scenarij se izvede v naslednjih korakih:

1. Sprožitev zahteve za poročilo

Regulator ali drug pooblaščen akter za transporterja DHL sproži zahtevo za poročilo o zaupanju. V prototipu to izvedemo z ukazom, ki prek skripte pošlje transakcijo `requestTrustReport` pametni pogodbi `TrustGraph.sol`. Pogodba pri tem ustvari nov identifikator zahteve `request_id` in izda dogodek `TrustOracleRequested`.

2. Izračun poročil v orakljih

Oraklji A, B in C spremljajo dogodke pametne pogodbe. Ko prejmejo dogodek za nov `request_id` in subjekt DHL, osvežijo ontologijo, naložijo VC iz skupnega repozitorija in telemetrijo ter izvedejo svoje algoritme:

- Orakelj A uporabi politike in semantične podatke ter za prevoznika DHL izračuna negativno odločitev z nizko oceno.
- Orakelj B preveri veljavnost poverilnice in subjekt oceni zelo visoko, brez opozoril.
- Orakelj C na podlagi telemetrijskih podatkov izračuna pozitivno odločitev in oceno, ki odraža dobro operativno delovanje.

Vsak orakelj pripravi podpisano poročilo `SignedOracleReport`.

3. Pošiljanje poročil agregatorju

Oraklji pošljejo svoja podpisana poročila agregatorju preko HTTP vmesnika `/reports`. Agregator preveri podpise in identitete vozlišč ter poročila začasno shrani pod isti `request_id`.

4. Agregacija rezultatov

Ko agregator za dani `request_id` prejme poročila vseh treh orakljev, uporabi pravila agregacije iz podrazdelka 4.1.3: večinsko glasovanje za diskretno odločitev, povprečje za številčno oceno ter večinsko pravilo za zastavice.

5. Zapis končne odločitve na verigo

Agregator pokliče funkcijo `fulfillTrustReport` v pametni pogodbi in za DHL zapiše končno odločitev in številčno oceno zaupanja, ki sta od takrat naprej vidni vsem udeležencem sistema.

Dejanski rezultati orakljev in agregatorja

Tabela 4.1 prikazuje dejanske rezultate poročil orakljev A, B in C ter agregatorja za subjekt DHL. Vrednosti so izražene v osnovnih točkah (0–10000).

Tabela 4.1: Rezultati orakljev A, B in C ter agregatorja za scenarij 1 (transporter DHL).

Vir	Tip algoritma	Odločitev	Številčna ocena	Zastavice
Orakelj A	hibridni	FAIL	2382	0x00
Orakelj B	dokazni (VC)	PASS	9500	0x00
Orakelj C	telemetrija	PASS	8879	0x00
Agregator	agregirano	PASS	6920	0x00

Interpretacija posameznih poročil:

- **Orakelj A** vrne negativno odločitev (FAIL) z nizko oceno 2382 in brez zastavic (0x00). Hibridni pogled na podlagi politik in semantičnih podatkov subjekt ocenjuje previdno, vendar iz same vrednosti ne izpelje dodatnega opozorila.
- **Orakelj B** vrne pozitivno odločitev (PASS) z zelo visoko oceno 9500 in brez zastavic (0x00). Ker ima DHL veljavno preverljivo poverilnico v skupnem repozitoriju VC, VC-centričen algoritem oceni, da so formalni pogoji za zaupanje izpolnjeni.
- **Orakelj C** vrne pozitivno odločitev (PASS) z visoko oceno 8879 in brez zastavic. Telemetrijski podatki (pravočasne dostave, stabilna temperatura) kažejo dobro operativno delovanje, zato telemetrijsko usmerjen pogled subjekt oceni kot zaupanja vreden.

Agregator iz teh treh poročil izvede:

- **Diskretna odločitev.** Dva od treh orakljev (B in C) poročata **PASS**, eden (A) poroča **FAIL**. Po večinskem pravilu je končna odločitev **PASS** – subjekt DHL je v agregiranem pogledu ocenjen kot zaupanja vreden.
- **Številčna ocena.** Ocene treh orakljev so 2382, 8879 in 9500. Aritmetična sredina teh treh vrednosti je približno 6920, zato agregator zapiše končno oceno zaupanja 6920/10000.
- **Zastavice.** Noben orakelj ne nastavi opozorilnih zastavic, zato agregator po večinskem pravilu ohrani vrednost 0x00. V tem scenariju končni rezultat ne vsebuje globalnih opozoril in predstavlja pozitiven zapis.
- **Politike in poverilnice.** Vsa tri poročila uporabljajo enako vrednost `credential_hash`, kar pomeni, da se sklicujejo na isto preverljivo poverilnico iz skupnega repozitorija VC. Agregator pri tem preveri, da poverilnica v lokalni shrambi ni označena kot preklicana.

Za ostale akterje v dobavni verigi je rezultat scenarija naslednji: ob izvedbi nad `TrustGraph.sol` za subjekt DHL dobijo informacijo, da je transporter trenutno označen kot zaupanja vreden (**PASS**) z agregirano številčno oceno zaupanja približno 6920/10000 in brez globalnih opozoril. Če želijo razumeti razloge za razhajanje med pogledi, lahko pregledajo posamične zapise orakljev A, B in C na verigi ter vidijo, da hibridni pogled Oraklja A subjekt ocenjuje previdneje, medtem ko dokazni in telemetrijski pogled ocenjujeta, da je transporter glede na preverljive poverilnice in operativne kazalnike dovolj zaupanja vreden.

4.1.5 Scenarij 2: Poslabšanje zaupanja po incidentih transporterja DHL

V drugem scenariju nadaljujemo primer transporterja DHL iz scenarija 4.1.4, vendar predpostavimo, da se v naslednjem obdobju zgodi več incidentov v

dostavni verigi. Namen scenarija je pokazati, kako sistem dinamično prilagodi oceno zaupanja na podlagi novih podatkov in kako semantični ter telemetrijski signali lahko pretehtajo preverljiva dokazila (VC). Pri tem posebej poudarimo, da bi v realnem okolju ti podatki prihajali iz zunanjih sistemov, do katerih ima vsak orakelj svoj, ne nujno enak dostop.

Spremembe v telemetriji in zunanjih virih

Simuliramo, da se je v opazovanem obdobju povečalo število poznih dostav in odstopanj v senzorskih meritvah. V prototipu to predstavimo kot spremembo zapisa v datoteki s telemetrijskimi podatki:

```
"http://example.org/trust#DHL": {  
  "tempDeviation": 0.12,  
  "lateDeliveries": 0.12,  
  "alerts": 2  
},
```

V ontologiji in repozitoriju VC subjekt DHL ostane enak kot v scenariju 1: še vedno ima veljavno preverljivo poverilnico skladnosti in status transporterja. Spremeni se torej le telemetrija.

V realnem sistemu tak zapis ne bi bil JSON datoteka, temveč projekcija zunanjih podatkov:

- meritve iz senzorjev IoT (npr. temperatura, vlaga),
- zapisi iz transportnega informacijskega sistema (TMS) o zamudah,
- opozorila iz nadzornih sistemov ali sistemov za kakovost.

Vsak orakelj bi bil odgovoren za svoje poti do takih virov. Nekaterim orakljem so ti podatki ključni, drugi jim dajejo zelo majhno težo. V prototipu te razlike modeliramo z različnimi algoritmi orakljev A, B in C nad istim simuliranim telemetrijskim virom.

Novi izračuni orakljev

Na podlagi spremenjenih podatkov oraklji ponovno izvedejo svoje algoritme in pripravijo nova podpisana poročila. Rezultati so predstavljeni v tabeli 4.2.

Tabela 4.2: Rezultati orakljev A, B in C ter agregatorja za scenarij 2 (transporter DHL).

Vir	Tip algoritma	Odločitev	Številčna ocena	Zastavice
Orakelj A	hibridni	FAIL	2382	0x02
Orakelj B	dokazni (VC)	PASS	9500	0x00
Orakelj C	telemetrija	FAIL	6920	0x02
Agregator	agregirano	FAIL	6267	0x02

Interpretacija posameznih poročil:

- **Orakelj A** (hibridni model) ostane negativen (FAIL) z nizko oceno 2382. Zastavica 0x02 označuje, da je ocena pod notranjim pragom (*nizka ocena*). V realnem okolju bi ta orakelj poleg ontologije in VC bral tudi izbran nabor zunanjih podatkov (npr. regulatorne incidente, povzetke revizij, agregirana merila kakovosti) in jih semantično povezal v končno oceno.
- **Orakelj B** (dokazni, VC model) se opira predvsem na preverljivo po-verilnico, ki je še vedno veljavna. Odločitev ostane pozitivna (PASS) z zelo visoko oceno 9500 in brez zastavic (0x00). V realnem scenariju bi ta orakelj tipično bral samo iz infrastrukture za digitalna dokazila in ne bi bil neposredno povezan z operativno telemetrijo.
- **Orakelj C** (telemetrija) neposredno zazna poslabšanje operativnih me-ril. Pri novih vrednostih `lateDeliveries` = 0,12 in `tempDeviation` = 0,12 so *točnost* in *temperaturna stabilnost* le še približno 0,88, število alarmov pa se poveča na 2. Po vgrajeni formuli za oceno

$$\text{base_score} = 0,6 \cdot 0,88 + 0,3 \cdot 0,88 - 0,05 \cdot 2 \approx 0,692,$$

dobimo številčno oceno 6920/10000. Ker je ocena pod pragom in je število alarmov ≥ 2 , orakelj C spremeni odločitev v FAIL in nastavi zastavico 0x02 (nizka ocena). V realnem sistemu bi ta orakelj poslušal neposredno tokove telemetrije in iz njih sproti gradil oceno zaupanja.

Agregirana odločitev po incidentih

Agregator v tem scenariju združi poročila tako kot v scenariju 4.1.4: uporabi večinsko glasovanje za diskretno odločitev, povprečje za številčno oceno ter večinsko pravilo za zastavice.

- **Diskretna odločitev.** Orakelj A in C poročata FAIL, Orakelj B poroča PASS. Dva od treh orakljev sta negativna, zato je agregirana odločitev FAIL. Z vidika sistema je DHL po incidentih označen kot nezaupanja vreden.
- **Številčna ocena.** Ocene treh orakljev so 2382, 9500 in 6920. Aritmetična sredina je približno 6267, kar je nova agregirana številčna ocena zaupanja:

$$\frac{2382 + 9500 + 6920}{3} \approx 6267.$$

Ocena je bistveno nižja kot v prvem scenariju (kjer je znašala približno 6920), kar odraža vpliv poslabšanih operativnih meril.

- **Zastavice** Oraklja A in C nastavljata zastavico 0x02 (*nizka ocena*), Orakelj B pa zastavic ne nastavlja. Ker ima ta bit nastavljena večina orakljev, agregator v končnem rezultatu prav tako nastavi 0x02. Agregirani zapis tako ne le označi subjekt kot nezaupanja vreden, temveč tudi razloži razlog: nizka številčna ocena, pod pragom sprejemljive ravni.
- **Politike in poverilnice** Vsa tri poročila uporabljajo enako zgostitev poverilnic `credential_hash`, kar pomeni, da se še vedno sklicujejo na isto veljavno preverljivo poverilnico. Agregator pri tem preveri, da poverilnica ni označena kot preklicana. To pomeni, da je sprememba

ocene iz **PASS** (scenarij 1) v **FAIL** (scenarij 2) posledica izključno poslabšanja operativnih meril in zunanjih podatkov, ne pa spremembe formalnih dokazil.

Za ostale akterje v dobavni verigi je rezultat scenarija naslednji: ob poizvedbi nad **TrustGraph.sol** za subjekt DHL dobijo informacijo, da je transporter po novih incidentih označen kot nezaupanja vreden (**FAIL**) z agregirano številčno oceno zaupanja približno 6267/10000 in z zastavico 0x02, ki označuje nizko oceno. Primerjava scenarijev 4.1.4 in 4.1.5 jasno pokaže, da je sistem dinamično prilagodil oceno zaupanja na podlagi novih podatkov, pri čemer so semantični in telemetrijski signali pretehtali formalna dokazila, ki jih je predstavljala preverljiva poverilnica.

4.2 Varnostna analiza sistema

V tem poglavju obravnavamo varnostni vidik ključnih delov razvitega sistema: decentraliziranega omrežja orakljev (DON) in pametne pogodbe. Prav ti dve komponenti skupaj izvajata proces izračuna zaupanja, pripravljata poročila, preverjata njihove podpise in na koncu zapišeta agregirane odločitve v nespremenljivo podatkovno strukturo na verigi.

Ostali deli arhitekture, kot so podatkovna baza preverljivih poverilnic, ontologija v strežniku Fuseki ali lokalni telemetrični viri podatkov, niso predmet podrobne varnostne analize v tem poglavju. Te komponente obravnavamo kot zunanje storitve, ki imajo svoje mehanizme za zagotavljanje integritete, razpoložljivosti in zanesljivosti. Varnostno pomembne so predvsem v toliko, kolikor lahko s svojimi podatki vplivajo na vhodne informacije za oraklje, ne vplivajo pa neposredno na zapisovanje ali branje rezultatov na verigi blokov.

Osrednji del analize zato posvečamo mehanizmu, ki odločajo o tem, komu sistem zaupa pri izračunu in zapisu rezultatov, ter kako se podatki premikajo med posameznimi komponentami. Ključni poudarek je na dveh vidikih: prvič, na zaščiti pametne pogodbe pred nepooblaščenimi vpisi ali manipulacijami, in drugič, na zagotovitvi, da lahko agregator prepozna ve-

ljavne prispevke orakljev, preveri njihove podpise in zavrne kakršne koli nepravilne ali ponarejene podatke. Tok podatkov med oraklji in agregatorjem je zato ena izmed osrednjih točk varnostne presoje, saj se prav tam zgodi največji del logike, ki odloča, katera poročila bodo na koncu vplivala na stanje sistema.

V nadaljevanju najprej opišemo metodologijo in model napadalca, ki ga uporabljamo v analizi, nato pa sistematično pregledamo grožnje, ki zadevajo pametno pogodbo in decentralizirano omrežje orakljev, ter prikažemo obstoječe in predlagane mehanizme za njihovo ublažitev.

4.2.1 Cilji varnostne analize

Prvi cilj varnostne analize je konceptualen: želimo jasno opredeliti, katere varnostne lastnosti mora imeti sistem, ki združuje verigo blokov, pametne pogodbe in decentralizirano omrežje orakljev. Pri takem sistemu mora biti zagotovljeno, da so odločitve o zaupanju, zapisane na verigi, pravilne, preverljive in odporne na manipulacije, ter da jih lahko drugi akterji neodvisno preverijo. Sistem mora vzdržati tudi primere nepoštenih ali kompromitiranih orakljev, napake infrastrukture in poskuse vplivanja na agregirane rezultate.

Drugi cilj je praktičen in izhaja iz konkretne implementacije prototipa. Pregledati moramo, katere od teh ključnih lastnosti so že ustrezno podprte in kje še obstajajo vrzeli, ki bi jih bilo treba zapolniti za namestitev v produkcijsko okolje. Pri tem se osredotočamo na integriteto podatkov, zapisano v pametni pogodbi, na avtentikacijo orakljev in agregatorja, na preverljivost posameznih poročil ter na razpoložljivost mehanizma za izračun zaupanja, tudi kadar pride do izpada ali nepravilnega delovanja posameznih komponent.

Varnostna analiza se tako osredotoča na del sistema, kjer se sprejemajo in zapisujejo odločitve o zaupanju. Posebej nas zanima, ali je potek podatkov med oraklji, agregatorjem in pametno pogodbo zaščiten pred napačnimi podpisi, ponarejenimi identitetami, spreminjanjem vsebine poročil, napadi zavrnitve storitve in drugimi oblikami poskusov vplivanja na rezultat. Ker so prav ti podatki podlaga za nadaljnje odločitve v dobavni verigi, mora biti jedro sistema načrtovano tako, da nepravilne ali zlonamerne posege zazna in zavrne ter omogoča preverljivo delovanje.

4.2.2 Pregled arhitekture DON in pametne pogodbe

Za potrebe varnostne analize na kratko povzamemo del arhitekture, ki je ključen za mehanizem zaupanja. Pametna pogodba `TrustGraph.sol`, ki se izvaja na testnem omrežju Ethereum, hrani agregirane odločitve o zaupanju za posamezne subjekte ter posamične prispevke orakljev, vezane na identifi-

katorje zahtev `requestId`. Agregator deluje kot edini pooblaščen zapisovalec teh rezultatov in ima zato lasten naslov ter zasebni ključ v omrežju. Klice na pametno pogodbo `TrustGraph.sol` pošilja prek varnega klica RPC do vozlišča verige blokov, pri čemer je identiteta klicatelja na ravni Ethereum protokola kriptografsko potrjena s podpisom transakcije.

Funkcije pametne pogodbe, ki sprejemajo rezultate, so zaščitene z vlogo oziroma modifikatorjem `onlyEvaluator`, kar pomeni, da jih lahko kliče le z zasebnim ključem, katerega lastnik je agregator. Klici z drugih naslovov povzročijo zavrnitev transakcije (v pametnih pogodbah se kliče funkcija `revert`). Za spremembe nastavitev, kot so menjava agregatorja ali dodeljevanje novih vlog, je predvideno upravljanje z vlogami in potrjevanjem z več podpisi (angl. *multi-sig*). To preprečuje, da bi en sam zasebni ključ samovoljno spremenil pravila delovanja sistema.

Na verigo se nikoli ne pošiljajo surovi vhodni podatki. Zapisujejo se le zgostitve (povzetki politike, povzetki preverljivih dokazil in podatki iz ontologije), agregirana ocena ter opozorilne zastavice. Pametna pogodba tako deluje kot kriptografsko zavarovana osrednja plast, ki zagotavlja integriteto rezultatov, ne da bi hranila osebne ali poslovno občutljive podatke.

Oraklji spremljajo dogodke `TrustOracleRequested`, pridobijo vhodne podatke iz plasti znanja ter izračunajo svojo oceno zaupanja. Ko je izračun opravljen, orakelj pripravi podpisano poročilo `SignedOracleReport` in ga posreduje agregatorju prek klica `HTTP /reports`. Vsak orakelj ima lasten par javnega in zasebnega ključa, ki ga uporablja za digitalno podpisovanje poročil z digitalnim podpisom ECDSA, ki uporablja eliptično krivuljo imenovano `secp256k1`. Pred podpisom se celotno poročilo pretvori v JSON predstavitev, kjer sta vrstni red ključev in oblika serializacije vnaprej določena. Ta pristop zagotavlja, da ima identičen podatkovni objekt vedno enako binarno predstavitev in s tem tudi enoličen kriptografski podpis, kar je bistveno za preverjanje podpisnika poročila.

Agregator ob prejemu poročila obnovi javni ključ iz podpisa (ECDSA *public key recovery*) in iz njega izpelje naslov Ethereum pošiljatelja. Na

ta način preveri, ali podpis res ustreza podatkom v poročilu in ali naslov pripada oraklju, ki je na seznamu dovoljenih vozlišč. Če se podpis ne ujema ali če naslov ni dovoljen, agregator poročilo zavrne. Tako sistem ne temelji na zaupanju v omrežno povezavo, temveč na kriptografski verifikaciji vsake posamične oddaje.

Pri branju iz verige blokov pametna pogodba ponuja standardne funkcije tipa `view` in `pure`. Prek njih lahko katerikoli odjemalec brez plačila za izvajanje (`gas`) dostopa do trenutnih in zgodovinskih rezultatov ocenjevanja. Dodatni podatki so na voljo tudi prek dnevnikov dogodkov, ki jih objavi pogodba ob zapisovanju posamičnih prispevkov in končnih agregiranih rezultatov. Ker se na verigi hranijo le zgostitve in psevdonimni identifikatorji, branje podatkov ne razkriva nobenih surovih informacij o subjektih ali njihovih lastnostih.

Takšen podatkovni tok določa glavne meje zaupanja in razkriva ključne točke, kjer je sistem potencialno ranljiv: pri klicih pametne pogodbe (kdo lahko sproži izračun in kdo lahko objavi rezultat), na povezavi med pametno pogodbo in oraklji (pravilna interpretacija identitete in politike) ter na povezavi med oraklji in agregatorjem (preverjanje pristnosti pošiljateljev, zaščita pred ponarejenimi podpisi, preverjanje preverljivih dokazil ter zaščita pred napadi ponovnega predvajanja). V nadaljevanju zato ločeno obravnavamo grožnje za pametno pogodbo `TrustGraph.sol` in grožnje za decentralizirano omrežje orakljev, ter pokažemo, kateri mehanizmi so že implementirani in kje bi bilo treba zasnovo dodatno utrditi pri prehodu v produkcijsko okolje.

4.2.3 Metodologija analize in model napadalca

Varnostno analizo izvedemo po načelih formalnega modeliranja groženj, kot jih predlagajo smernice OWASP in Microsoftovi standardi za decentralizirane sisteme. Osrednji okvir, ki ga uporabimo, je model STRIDE, predstavljen v knjigi Howarda in Lipnerja, *The security development lifecycle* [23]. Varnostne grožnje razvrščata v šest razredov glede na vrsto kršitve: lažno predstavljanje identitete (Spoofing), spreminjanje podatkov (Tampering), zanikanje

sodelovanja (Repudiation), razkritje informacij (Information disclosure), zavrnitev storitve (Denial of service) ter povečanje privilegijev (Elevation of privilege). STRIDE je primeren za naš sistem, ker se osredotoča na integriteto, avtentičnost in dostopnost podatkov — lastnosti, ki so v mehanizmu zaupanja ključne, saj neposredno vplivajo na pravilnost izračunov in na zanesljivost registra zaupanja v pametni pogodbi.

Pri analizi ne razčlenjujemo le posameznih komponent, temveč tudi meje med njimi. V sistemu so te meje izrazite: pametna pogodba predstavlja kriptografsko zaščiteno točko trajnega dnevnika, oraklji izvajajo izračune zunaj verige in so zato izpostavljeni manipulacijam, agregator pa deluje kot povezovalni element med izračuni in verigo blokov. STRIDE zato uporabimo kot okvir skozi celoten tok podatkov. Na grožnje se osredotočamo tam, kjer so povezave najšibkejše: pri podpisovanju poročil, pri prenosu rezultatov v agregator, pri preverjanju upravičenosti klicev pametne pogodbe ter pri mehanizmih, ki določajo, kdo lahko objavi končno oceno na verigi.

Čeprav je STRIDE široko uporabljen model, se je potrebno zavedati njegovih omejitev. Ne obravnava vprašanj gospodarskih ali ekonomskih spodbud, napadov na semantiko ali zlorab na ravni ontologije, zato te grožnje obravnavamo ločeno kot dopolnilno analizo.

Pri določanju tipa napadalca predpostavimo več ravni zmogljivosti. Prvi scenarij predstavlja zunanjega napadalca, ki ima enaka dovoljenja kot tipični uporabniki verige blokov: lahko kliče javne funkcije pametne pogodbe, spremlja dogodke in poskuša sprožiti izračune ali preobremeniti pogodbo. Drugi scenarij obravnava orakelj, ki želi škodovati sistemu. Tak orakelj lahko odda napačno poročilo, manipulira z vhodnimi podatki ali poskuša ponarediti podpis. Ker uporabljamo digitalne podpise ECDSA nad deterministično JSON predstavitevijo, tak napadalec ne more ponarediti avtentičnosti, lahko pa poskuša vplivati na izračun s prirejenimi podatki iz ontologije ali preverljivih dokazil. Tretji scenarij predvideva napadalca, ki cilja na spremembo agregiranja. Ta bi lahko poskušal sprejeti lažna poročila, zlorabiti svoj položaj pri zapisovanju rezultatov ali poskušal reproducirati stare podpise. Ker agrega-

tor rekonstruira naslov pošiljatelja iz digitalnega podpisa in preverja časovni žig ter identifikator izvajanja, je zaščiten pred ponarejenimi sporočili in ponavljanjem že obdelanih poročil. Zadnja kategorija so napadi zavrnitve storitve, kjer napadalec poskuša preobremeniti oraklje ali agregator z velikim številom zahtev in tako zmanjšati razpoložljivost izračunov.

Tak model napadalca nam omogoča, da jasno opredelimo, katere komponente se morajo zanašati na kriptografske zaščite (podpisi, zgostitve, preverjanje naslovov), katere so zaščitene z logiko pametne pogodbe (vloge, omejitve klicev) ter kje sistem zahteva dodatne arhitekturne ukrepe za produkcijsko okolje, kot so redundančni agregatorji, nadzor nad ključi ter mehanizmi zaščite pred preobremenitvijo. STRIDE tako ponuja ogrodje, ki ga v nadaljevanju uporabimo za razvrščanje groženj in vrednotenje implementiranih protiukrepov.

4.2.4 Analiza groženj za pametno pogodbo

Pametna pogodba `TrustGraph.sol` predstavlja edini vir resnice sistema na verigi blokov, saj hrani končne odločitve o zaupanju in posamične prispevke orakljev. Kot smo objasnili v prejšnjem podrazdelku, je pomembno, da sprejema le veljavne agregirane rezultate, pravilno povezuje prispevke z identifikatorji zahtev `requestId` in zavrača nepooblaščne spremembe stanj. V tabeli 4.3 je zbran pregled glavnih razredov groženj (po modelu STRIDE), tipičnih protiukrepov ter oblik testnega pokrivanja.

Tabela 4.3: Povzetek glavnih groženj in protiukrepov za pametno pogodbo `TrustGraph.sol`.

Grožnja	Protiukrepi	Testno pokritje
Nepooblašчени klici funkcij za zapis rezultatov	Vloga <code>EVALUATOR</code> oziroma modifikator <code>onlyEvaluator</code> , ki omeji klice na naslov agregatorja	Enotski testi klicev z nedovoljenih naslovov, ki pričakujejo <code>revert</code>
Neveljavne spremembe stanja odločitev o zaupanju	Preverjanje parametrov in stanjskih invariant (ujemanje <code>requestId</code> , monotoni časovni žigi, veljavne zastavice)	Testi fuzzing in regresijski testi zaporednih klicev za preverjanje ne-logičnih stanj
Preobremenitev pogodbe z velikim številom zahtev	Omejitve, kdo lahko sproži nove zahteve, ter preverjanje velikosti in kompleksnosti zapisov, plačilo za klice funkcij pametne pogodbe	Merjenje porabe plina; testi mejnih primerov za funkcije <code>requestTrustReport</code> in <code>fulfillTrustReport</code>
Prevelika količina podatkov o subjektih na verigi	Shranjevanje le zgoščenih identifikatorjev in agregiranih ocen	Pregled sheme stanja in testi za nove različice, ki preverjajo, da se na verigo ne dodajajo surovi identifikatorji ali PII
Zloraba upravljaljskih oziroma administrativnih vlog	Vloge in večpodpisno upravljanje nad kritičnimi konfiguracijskimi funkcijami (npr. zamenjava agregatorja)	Enotski testi dodeljevanja, spreminjanja in preklica vlog; zaščite privilegiranih funkcij

Kot prikazuje tabela 4.3, je prvi sklop groženj povezan z nepooblaščenimi klici funkcij, ki spreminjajo stanje grafa zaupanja. Tovrstne napade imenujemo slepljenje ali zavajanje (angl. spoofing). V implementaciji funkcije za zapis rezultatov klice omejujemo z vlogo **EVALUATOR** oziroma modifikatorjem **onlyEvaluator**, tako da jih lahko kliče le agregator oziroma izrecno pooblaščen naslovi. Enotski testi implementirani v jeziku Solidity simulirajo klice z drugih naslovov in preverjajo, da pogodba takšne poskuse zavrne s funkcijo **revert**.

Drugi sklop groženj se nanaša na integriteto stanja pametne pogodbe (angl. tampering). Čeprav veriga blokov zagotavlja nespremenljivost zgodovine transakcij, to samo po sebi ne prepreči, da pametna pogodba zaradi neustrezno preverjenih parametrov ali pomanjkljivih logičnih pogojev vnese v svoje stanje nekonsistentne podatke. Do takšnih razmer lahko pride denimo takrat, ko se nov rezultat napačno poveže z drugim subjektom ali ko nova odločitev dobi časovni žig, ki je kronološko starejši od časovnega žiga že obstoječe odločitve za istega subjekta. V tem primeru “računanje časovnega žiga v preteklost” pomeni, da se nova odločitev zabeleži prej kot njej predhodna, saj je njen pripisani časovni žig manjši (starejši). V našem primeru časovni žig nastopa kot kriptografsko zaščiten podatek o času, ki se vključi v zgostitev in podpis poročila ter se zapiše v verigo blokov skupaj z ostalimi parametri. Takšno stanje je semantično nelogično. Zato so v implementaciji funkcij uvedeni preverjevalni mehanizmi, ki zagotavljajo ohranjanje osnovnih stanjskih invariant (angl. state invariants). Med njih spadajo: (i) zahteva, da se vsak novi zapis nanaša na isti identifikator subjekta kot začetni zahtevek, (ii) da je časovni žig (**asOf**) monoton in nikoli ne nazaduje v preteklost ter (iii) da so meje dovoljenih vrednosti (npr. razpon ocene) ustrezno nadzorovane. Za dodatno zagotovitev pravilnosti smo izvedli kombinacijo enotskih in regresijskih testov. Enotski testi preverjajo posamezne robne primere na ravni funkcij. Regresijski testi pa odkrivajo nove napake v kodi, ki nastanejo z dodajanjem novih funkcij. T. i. testi *fuzzing* avtomatsko generirajo veliko število naključnih kombinacij parametrov (različni **requestId**, časovni

žigi, zastavice in ocene). S tem preverimo, ali bi katera od kombinacij lahko pripeljala pogodbo v nelogično stanje ali sprožila neželene stranske učinke. Regresijski testi pa zagotavljajo, da popravki logike ne uvajajo novih nepravilnosti in da se znane invariante v kasnejših posodobitvah ohranijo.

Tretji sklop se nanaša na napade zavrnitve storitve tipa DoS. Ker so nekatere funkcije pametne pogodbe javno dostopne, lahko velik del zahtev za izračun zaupanja ali nepravilno nastavljeni klici paketov povzročijo, da postane uporaba pogodbe ekonomsko neupravičena. Lahko bi se uvedli stroški za klic vseh funkcij na pametni pogodbi. S tem bi preprečili velike količine zahtevkov. Pri prehodu na produkcijsko verigo bi bilo zagotovo treba omejiti, kdo lahko sproži `requestTrustReport`, uvesti pragove za velikost paketov ter spremljati porabo stroškov verige blokov, da se pravočasno zazna kršitve.

Četrty sklop pokriva razkritje informacij (angl. information disclosure). Pametna pogodba shranjuje le zgoščene identifikatorje subjektov, agregirano številčno oceno zaupanja ter zastavice, kar sledi načelu minimizacije podatkov. Pri nadaljnjem razvoju je pomembno ohraniti isti pristop tudi pri razširitvah sheme: novi podatki naj bodo vedno agregirani ali zgoščeni. Enotski testi v naši implementaciji preverjajo, da pogodba ne shranjuje neposrednih identifikatorjev podjetij.

Zadnji sklop se nanaša na zlorabo privilegijev (angl. elevation of privilege). V produkcijskem okolju bo potrebno uvesti upravljske funkcije za posodobitev konfiguracije ali migracijo stanja (npr. zamenjava agregatorja, posodobitev parametrov politik). Take funkcije je dobro zaščititi z uveljavljenimi vzorci, kot so vloge in večpodpisno upravljanje. Pri večpodpisnem upravljanju noben posamezen skrbnik ne more sam izvesti kritične operacije, temveč je za odobritev potrebna kombinacija podpisov več neodvisnih udeležencev. Na ta način zmanjšamo tveganje zlorabe enega podpisa. Upravljske funkcije je smiselno podpreti z enotskimi testi, ki preverjajo pravilno dodeljevanje, rotacijo in preklic vlog ter varnostne meje med njimi. Tako bi lahko na primer regulator predlagal spremembo agregatorja, ki bi jo morali potrditi vsi člani sveta za upravljanje sistema, preden bi postala aktivna.

4.2.5 Analiza groženj za decentralizirano omrežje orakljev

Mreža orakljev in agregator skupaj tvorita decentralizirano omrežje, ki je odgovorno za izračun in agregacijo ocen zaupanja. Pretok podatkov poteka tako, da pametna pogodba izda dogodek `TrustOracleRequested` za dani `requestId` in subjekt, oraklji preberejo dogodek, iz zunanjih podatkov izračunajo oceno zaupanja in pripravijo podpisano poročilo. Agregator sprejme poročila prek končne točke `/reports`, preveri podpise, preveri priložene poverilnice, izvede agregacijo in rezultat zapiše v pametno pogodbo. Tabela 4.4 povzema glavne grožnje po modelu STRIDE, tipične protiukrepe ter predvideno testno pokritje na ravni orakljev in agregatorja.

Tabela 4.4: Povzetek glavnih groženj in protiukrepov za mrežo orakljev in agregator.

Grožnja	Protiukrepi	Testno pokritje
Lažni orakelj pošilja poročila agregatorju	Kriptografsko preverjanje podpisov (ECDSA <code>recover</code>) in seznam dovoljenih naslovov orakljev	Integracijski test, ki na <code>/reports</code> pošlje napačno podpisan zahtevek ali zahtevek z neznanega naslova in pričakuje napako
Spreminjanje vsebine poročil med prenosom ali v agregatorju	Podpis predstavitve JSON, preverjanje zgostitev in zavrnitev pri neskladju	Enotski testi serializacije ter preverjanja podpisov in zgostitev
Orakelj zanika, da je oddal določeno poročilo	Shranjevanje zgostitev podpisanih poročil in obnova naslova iz podpisa	Testi obnove naslova iz podpisa in uje-manja z zapisanim <code>node_id</code> ter revizijske sledi
Prekomerno evidentiranje identitet subjektov ali poverilnic	Zgostitve identifikatorjev v dnevnikih	Enotski testi nad vzorčnimi logi, ki preverjajo, da se občutljivi podatki ne pojavijo v izpisu
Preobremenitev orakljev ali agregatorja z zahtevki	Omejitve na nivoju omrežja (rate limiting, požarni zid), kvote in redundanca vozlišč	Obremenitveni testi HTTP vmesnika ter spremljanje odzivnega časa in števila zahtevkov
Kompromitiran agregator zlorabi svojo vlogo pri zapisovanju rezultatov	Več agregatorjev ali delna agregacija na verigi, ločeno upravljanje konfiguracije in jasna omejitev pooblastil	Simulacijski testi različnih konfiguracij kvora in scenarijev kompromitiranih vozlišč

Prva skupina groženj za DON je povezana z istovetnostjo orakljev (spoofing). Agregator pri prejemu poročila kriptografsko obnovi naslov iz podpisa in ga preveri s seznamom dovoljenih vozlišč. Poročila z neznanih naslovov ali napačnimi podpisi zavrne. Ker podpis vključuje tudi **nonce** oziroma **run-id** in časovni žig, lahko agregator zazna in zavrne ponovno predvajanje starega sporočila in s tem prepreči napade ponovitve. Integracijski testi za HTTP vmesnik preverijo, da so spremenjena ali napačno podpisana poročila zavrnjena z ustrezno napako.

Pri napadih s spreminjanjem podatkov ščitimo predvsem integriteto vsebine poročil. Ker je podpisan JSON celotnega poročila, agregator zazna vsako spremembo vsebine med prenosom ali vmesnim shranjevanjem. Za produkcijsko okolje je smiselno dodati revizijsko sled (na primer zapis zgoštitev sprejetih poročil), kar omogoča naknadno preverjanje skladnosti med prejetimi poročili in zapisi na verigi. Enotski testi pri tem preverijo, da serializacija poročila v kanonično obliko vedno vrne enak rezultat in da vsaka sprememba vsebine povzroči neuspeh preverjanja podpisa.

Grožnja z zanikanjem sodelovanja je pomembna z vidika transparentnosti: orakelj bi lahko poskušal zanikati, da je oddal poročilo za določenega partnerja. Ker imamo digitalni podpis poročila in, ker lahko iz verige blokov ponovno oblikujemo poročilo oraklja v kanonično obliko JSON, lahko s klicem funkcije `recover_message` iz knjižnice `web3.py` dobimo naslov oraklja, ki je poročilo podpisal. S tem lahko kadarkoli neodvisno preverimo ali je orakelj res oddal določeno poročilo.

Del zasnove DON je tudi preverjanje preverljivih poverilnic (VC), ki spremljajo poročila. Vsak orakelj poleg ocene zaupanja priloži VC, ki dokazuje njegovo pooblastilo ali lastnosti subjekta. Agregator pred sprejemom poročila preveri podpis VC, status preklica, časovno veljavnost in skladnost sheme z vnaprej definirano politiko. Poročila z neveljavnimi ali preklicanimi poverilnicami so zavrnjena in se nikoli ne zapišejo na verigo. To zmanjša tveganje, da bi se na grafe zaupanja zapisali ponarejeni ali zastareli dokazi.

Informacijsko razkritje v omrežju orakljev se najpogosteje zgodi prek

dnevniških zapisov. V prototipu oraklji in agregator že uporabljajo zgoščene identifikatorje subjektov, saj se DID pred zapisom v dnevnik pretvori v vrednost `bytes32`. Kljub temu je pri nadaljnjem razvoju treba paziti, da v dnevnikih ne ostanejo neposredni naslovi URI ali vsebine poverilnic. Priporočljivo je dosledno uvesti preverjanje dnevnikov (npr. izpis le skrajšanih zgostitev in anonimnih oznak namesto poslovnih imen) ter z enotnimi testi preverjati, da dnevniki ne vsebujejo občutljivih podatkov.

Napadi zavrnitve storitve (DoS) ciljajo na razpoložljivost omrežja. Ker v prototipu omejevanje hitrosti ni povsod implementirano, so oraklji in agregator teoretično dovzetni za preobremenitev z veliko količino zahtev. V produkcijski rabi bi morali definirati omejitev dostopa na nivoju omrežja in zagotoviti redundanco vozlišč, tako da izpad posameznega oraklja ali agregatorja ne poruši sistema in da se promet enakomerno porazdeli.

Zadnji sklop groženj se nanaša na privilegije agregatorja. Trenutni model predvideva en osrednji agregator, ki ima pooblastilo za zapis agregiranih odločitev v pametno pogodbo. Če bi bil ta agregator kompromitiran, bi lahko v imenu legitimnih orakljev pošiljal napačne podatke ali spreminjal rezultat agregacije, preden se ta zapiše na verigo. Pri razširitvi v produkcijo bi bilo smiselno razmisliti o več agregatorjih, ki bi morali doseči soglasje, ali o prenosu dela agregacijske logike v pametno pogodbo samo, tako da posamezno vozlišče ne bi moglo samostojno manipulirati z rezultati. Simulacije različnih nastavitvev kvoruma in scenarijev kompromitiranih vozlišč lahko pomagajo oceniti, kako odporen bi bil tak sistem na zajetje ali okvaro posameznih komponent. Rešitev bi lahko bila tudi zapis ocen orakljev neposredno na verigo, čeprav bi to zahtevalo dodatne arhitekturne prilagoditve in povečane stroške.

4.2.6 Povzetek tveganj in omejitve prototipa

V analizi smo se osredotočili na del sistema, ki deluje na verigi blokov in v mreži orakljev. Implementacija že vključuje več ključnih varnostnih mehanizmov: podpisana poročila orakljev, preverjanje naslovov v agregatorju,

uporabo časovnih žigov za zaščito pred napadi ponovitve, osnovno preverjanje preverljivih poverilnic, shranjevanje le zgoščenih identifikatorjev in agregiranih ocen na verigi ter jasno ločitev med logiko na verigi in izven nje.

Kljub temu za uporabo v produkcijskem okolju ostaja odprtih več pomembnih vprašanj: registracija in upravljanje identitet orakljev ter agregatorjev na nivoju pametne pogodbe, varno upravljanje z zasebnimi ključi (shramba, spremembe dovoljenj, odziv ob kompromitaciji), odpornost na napade zavrnitve storitve v DON (redundantni oraklji, več agregatorjev, mehanizmi soglasja) ter obsežnejše testiranje pametne pogodbe za preprečevanje nelogičnih stanj. Varnostna analiza tako ne služi le kot opis trenutnega stanja, temveč tudi kot usmeritev, katera področja bi bilo treba prioritetno nadgraditi pri prenosu rešitve v realno, produkcijsko okolje.

4.3 Razprava

4.4 Pot do produkcijskega sistema

Trenutna implementacija deluje kot laboratorijska postavitve, v kateri deluje omejeno število orakljev, en agregator in ena instanca pametne pogodbe na testnem omrežju verige blokov, vse pa je vezano na poenostavljeno infrastrukturo Docker. Takšen prototip je primeren za demonstracijo koncepta in ponovljivih scenarijev, ni pa neposredno prenosljiv v okolje, kjer bi morali upoštevati zahtevnejše omejitve glede razpoložljivosti, upravljanja identitet, nadzora nad konfiguracijo in dolgoročne vzdržnosti.

V produkcijskem sistemu bi moral DON upravljati večje število neodvisnih orakljev, ki bi jih upravljali različni akterji v dobavni verigi ali specializirani ponudniki storitev zaupanja. Regulator (na primer EMA ali nacionalna agencija) bi imel v takem modelu vlogo skrbnika nad omrežjem orakljev: skrbel bi za upravljanje registracije orakljev, odobritev novih članov, izključevanje neustreznih ter pregled nad konfiguracijo politik. Za to bi bil potreben namenski upravljavski vmesnik, ki bi regulatorju omogočal pregled

stanja DON, upravljanje s seznamami pooblaščenih vozlišč in konfiguracijo pragov zaupanja brez neposrednega poseganja v kodo pametne pogodbe.

Pomemben del prehoda v produkcijo je formalizacija upravljanja identitet in poverilnic akterjev. Za plast preverljivih poverilnic bi bila naravna izbira uporaba evropske infrastrukture EBSI, ki že definira standardizirane formate VC in DID ter infrastrukturo za preverjanje in preklic. V takem scenariju bi oraklji in akterji v dobavni verigi nastopali z DID, veriga blokov in DON pa bi se zanašala na EBSI kot na zunanji vir resnice za status poverilnic. Vse, kar bi se zapisalo na verigo, bi bili le povzetki teh VC in politik, medtem ko bi se polna vsebina in osebni podatki obdelovali v okviru EBSI in povezanih sistemov.

Za samo infrastrukturo DON bi bilo smiselno raziskati uporabo obstoječe rešitve za decentralizirana omrežja orakljev, kot je Chainlink. Ta platforma že nudi vzorce za varno izvajanje orakljev v heterogenem omrežju, mehanizme za nadzor nad zanesljivostjo vozlišč, podpisane povratne klice in zaščito pred posameznimi kompromitiranimi vozlišči. Namesto lastne implementacije bi lahko mehanizem ocenjevanja zaupanja v prihodnje prevedli v Chainlinkove naloge, kjer bi nadzorni sklop (na primer regulator ali konzorcij akterjev) definiral, kateri oraklji sodelujejo v posamezni nalogi, s kakšnimi utežmi in pod katerimi pogoji.

Na ravni pametne pogodbe bi prehod v produkcijo zahteval utrditev mehanizmov upravljanja in nadgradnje. Ključno je, da pogodba jasno loči vloge (regulator, akterji, tehnični skrbniki) in omeji njihova pooblastila z dobro preizkušenimi vzorci, kot so večpodpisne denarnice (angl. Multi-Sig Wallet) in formalno definirani postopki nadgradnje. Vsak poseg v konfiguracijo, na primer dodajanje ali odstranjevanje oraklja, zamenjava agregatorja ali sprememba pragov zaupanja, bi moral biti transparenten, sledljiv in odobren s strani vnaprej definirane deleža udeležencev.

Poleg identitet in upravljanja vlog bi bilo treba v produkcijskem okolju bistveno več pozornosti nameniti varnemu upravljanju zasebnih ključev ter razpoložljivosti celotne arhitekture. Ključni orakljev in agregatorjev bi morali

biti shranjeni v varnih shrambah ter jasnimi postopki ob zaznani kompromitaciji. Infrastrukturo orakljev bi bilo treba postaviti v ponovljivem okolju, z nadzorovanimi posodobitvami, spremljanjem izpada in mehanizmi za avtomatsko preusmeritev prometa v primeru okvare.

Pot od trenutnega prototipa do produkcijskega sistema bi v praksi potekala v več korakih. Najprej bi bilo smiselno utrditi že implementirane varnostne mehanizme na ravni pametne pogodbe in agregatorja, dodati formalizirano upravljanje vlog ter razširiti testno pokritost. Sledila bi faza razširitve DON z več neodvisnimi oraklji in morda več agregatorji, kjer bi se v pilotnem okolju preizkusile različne konfiguracije kvoruma in algoritmi agregacije. V naslednjem koraku bi se sistem integriral z realno infrastrukturo preverljivih poverilnic (EBSI) in z izbranimi operativnimi sistemi akterjev v dobavni verigi, ob tem pa bi se v sodelovanju z regulatorji definirali formalni postopki uporabe rezultatov zaupanja v odločanju o odpošiljanju in licenciranju. Po uspešnem pilotnem obdobju bi sledil popoln prehod v produkcijsko okolje, z ustrezno dokumentacijo, usposabljanjem uporabnikov in vzpostavitvijo postopkov za dolgoročno vzdrževanje in nadgrajevanje sistema.

4.5 Operativni in regulativni vidiki

Če želimo mehanizem zaupanja prenesti iz eksperimentalnega okolja v dejansko farmacevtsko dobavno verigo, ga moramo umestiti v obstoječe procese kakovosti in regulative. Ocena iz DON ne bi nadomestila regulatorne odgovornosti, temveč bi delovala kot dodatni, standardiziran vhod v postopke kvalifikacije in periodičnega preverjanja partnerjev (proizvajalcev, distributerjev, transporterjev) ter odločanja o odpošiljanju serij. V praksi bi bila integrirana v obstoječe informacijske sisteme (npr. sistem za upravljanje dobaviteljev ali modul za izdajo serije) prek vmesnika API: aplikacija bi ob potrebi sprožila poizvedbo, prejela strukturiran odgovor (vključujoč odločitev, oceno, časovni žig) in ga vključila v revizijsko sled ter dokumentacijo kakovosti. Z vidika upravljanja bi bil DON najverjetneje vzpostavljen kot skupna

infrastruktura, nad katero bi imel nadzor regulator ali konzorcij regulatorjev in industrije, ki bi skrbel za referenčne politike, pravila za vstop in izključitev orakljev ter revizijo njihovih algoritmov.

Regulativno in z vidika varovanja podatkov je ključna ločitev med plastjo na verigi in izven verige. Surovi podatki (osebni, poslovno občutljivi) ostajajo v sistemih podvrženih GDPR [15], na verigo pa se zapisujejo le zgoščeni identifikatorji subjektov, politik in poverilnic ter agregirane ocene in zaslave. Veriga tako služi predvsem kot objekt integritete in neizbrisljiv dnevnik: v primeru spora lahko regulator ali akter neodvisno preveri, da objavljena ocena in pripadajoče poverilnice od trenutka objave niso bile spremenjene. Za produkcijsko rabo bi moral biti tudi sam mehanizem DON obravnavan kot validiran računalniški sistem z jasno opredeljenimi kritičnimi funkcijami, kontroliranim upravljanjem sprememb ter periodičnim pregledom algoritmov in pravil agregacije.

4.6 Širši potencial in vizija uporabe sistema

Čeprav je bila rešitev v tej nalogi konkretizirana na farmacevtski dobavni verigi, je osnovni vzorec precej širši. V jedru ne upravljamo zaupanja v zdravila, temveč v akterje, ki si izmenjujejo podatke in prevzemajo odgovornost v porazdeljenem sistemu. Tukaj imamo v mislih organizacije z različnimi vlogami, pogosto nasprotujočimi si interesi in heterogenimi podatkovnimi viri, katere potrebujejo jasne in preverljive zapise o tem, komu in pod kakšnimi pogoji zaupajo. Enak vzorec se pojavi v drugih dobavnih verigah: pri dobavni verigi hrane, elektronskih komponent, oskrbi z energenti, recikliranju materialov ipd. Povsod, kjer so prisotne tri lastnosti: (i) več neodvisnih organizacij, (ii) različni in pogosto nezdružljivi viri podatkov ter (iii) potreba po transparentnem in neodvisno preverljivem zapisu rezultatov.

Z manjšimi prilagoditvami ontologije in politik bi se isti mehanizem lahko uporabljal tudi v pametnih mestih. Namesto proizvajalcev in transporterjev bi se v grafu zaupanja pojavili upravljavci avtobusnih linij, ponudniki de-

ljenih koles in skirojev, operaterji avtonomnih vozil ter občinske službe, ki skrbijo za prometno infrastrukturo. Regulator (na ravni mesta ali države) bi postavljajal pravila glede varnosti, zanesljivosti in okoljskih vplivov, oraklji bi nad kombinacijo telemetrije, revizijskih poročil in preverljivih poverilnic izračunavali ocene zaupanja, pametna pogodba pa bi skrbela za to, da so končne odločitve transparentno in nespremenljivo zabeležene. V takem scenariju bi graf zaupanja postal neke vrste prometni "kompas": komu lahko mesto zaupa pri upravljanju kritične mobilnostne infrastrukture in kateri ponudniki izpolnjujejo dogovorjene standarde.

Podobno bi lahko pristop razširili na druge podatkovno intenzivne domene, kjer sodeluje več neodvisnih organizacij: od energetske skupnosti, kjer gospodinjstva in podjetja izmenjujejo energijo in fleksibilnost, do čezmejnih podatkovnih prostorov, kjer različni akterji delijo podatke o okolju, zdravju ali mobilnosti. V vseh teh primerih bi se farmacevtska ontologija preprosto zamenjala z ustreznim domenskim modelom, preverljive poverilnice bi zamenjale druge oblike digitalnih dokazil (licence, homologacije, certifikati skladnosti), DON pa bi postal mesto, kjer različni regulatorji, neodvisne institucije, certifikacijski organi ali celo skupnosti uporabnikov prispevajo svoj pogled na zanesljivost in integriteto posameznih akterjev.

Rešitev, predstavljena v tej nalogi je torej le prvi korak na poti k bolj razpršenim, transparentnim in odpornim ekosistemom zaupanja v digitalni dobi. S kombinacijo kriptografskih mehanizmov, decentraliziranih omrežij in pametnih pogodb lahko ustvarimo sisteme, kjer zaupanje ni več centralizirano v rokah posameznih akterjev, temveč je rezultat sodelovanja, preverjanja in neodvisnega nadzora. Takšni sistemi imajo potencial, da izboljšajo varnost v številnih kritičnih sektorjih družbe.

Poglavje 5

Zaključek in nadaljnje delo

5.1 Povzetek dela in prispevki

V magistrski nalogi smo obravnavali problem upravljanja zaupanja v farmacevtskih dobavnih verigah, kjer poteka izmenjava podatkov med številnimi akterji, ki pogosto delujejo z različnimi interesi in v heterogenih podatkovnih okoljih. Obstoječe prakse upravljanja zaupanja temeljijo predvsem na centraliziranih virih, ročnih revizijah in pogodbenih razmerjih. To otežuje, da je sistem transparenten, ponovljiv in onemogoča avtomatizirano preverjanje. Cilj naloge je bil zato razviti konceptualni in arhitekturni model, ki združuje semantično predstavitev domene, mehanizme verige blokov ter decentralizirane izračune ocen zaupanja med akterji dobavne verige zdravil.

Najprej smo predstavili teoretično ozadje in pregled obstoječih modelov zaupanja ter poudarili vrzeli, ki se pojavljajo pri uporabi teh modelov v farmacevtskem kontekstu. Na podlagi teh izhodišč smo zasnovali konceptualni model, ki akterje, njihove lastnosti in dokazila predstavi z ontologijo. Temu modelu smo dodali arhitekturo, ki vključuje pametno pogodbo za zapis rezultatov, omrežje decentraliziranih orakljev za izračun zaupanja ter agregator za usklajevanje njihovih odločitev. V implementaciji smo razvili prototip z uporabo tehnologij Docker, Jena Fuseki, preverljivih poverilnic VC in testnega omrežja Ethereum. Postavili smo tri oraklje, ki izvajajo ocenjevanje

zaupanja z različnimi kriteriji in pravili. Skozi dva eksperimentalna scenarija smo prikazali, kako sistem obravnava realne spremembe podatkov in kako decentralizirani pogledi vodijo do dinamičnih ter razločljivih ocen zaupanja. Z varnostno analizo smo nazadnje ocenili grožnje, ki zadevajo pametno pogodbo in decentralizirano omrežje orakljev. Poudarili smo tudi potrebne korake za prehod na delovanje v produkcijskem okolju.

Pomen opravljenega dela je predvsem v tem, da združuje področja, ki so bila v literaturi pogosto obravnavana ločeno: semantično modeliranje domene, decentralizirane identitete, kriptografsko preverjanje in porazdeljeno odločanje. S tem naloga zapolni vrzel med teoretičnimi modeli zaupanja in praktičnimi sistemi, ki jih lahko dejansko uporabljajo regulatorji in organizacije. Predlagana arhitektura pokaže, da je mogoče v kompleksnih okoljih, kot je dobavna veriga zdravil, oblikovati pregledne in preverljive postopke ocenjevanja. S tem se olajša zaznavanje tveganj in poveča zanesljivost odločanja v takšnih sistemih.

5.2 Nadaljnje delo

Predstavljeni prototip predstavlja dokaz za realizacijo koncepta, ne pa polne implementacije sistema, primerne za produkcijsko okolje. Nadaljnji razvoj mora zato vključevati razširitev decentraliziranega omrežja orakljev, ki bi jih upravljali dejanski akterji farmacevtske verige, ter uvedbo mehanizmov ekonomskih spodbud in kazni, ki bi zagotovili pošteno udeležbo in odpornost na zlonamerna dejanja. Prav tako je nujna integracija polne infrastrukture za izdajo in preverjanje poverilnic, skladnih s standardi, kot je EBSI, ter vzpostavitev upravljalkega modela (angl. governance), ki določa pravila za upravljanje identitet, upravljanje s ključi in sprejemanje sprememb arhitekture.

Ontologija, uporabljena v prototipu, bi morala biti za uporabo v realnih okoljih razširjena z dodatnimi tipi entitet, regulatornimi zahtevami, pogoji skladnosti ter bolj izraznimi pravili, ki omogočajo kompleksnejše metode

sklepanja. Varnostna plat sistema zahteva nadaljnje delo na področju robustnosti pametne pogodbe (formalno preverjanje, zaščita pred napadi zavrnitve storitve, zaščita upravljavskih vlog) ter odpornosti decentraliziranega omrežja orakljev (več agregatorjev, mehanizmi soglasja, varno upravljanje ključev). Pomembno je tudi razviti celovite postopke za validacijo sistema kot računalniškega sistema v skladu z regulativnimi zahtevami farmacevtskega sektorja.

Zanimiva smer nadaljnjega razvoja je tudi preizkus zasnove v drugih sektorjih. Pametna mesta, energetika, logistika, kritična infrastruktura in omrežja IoT predstavljajo naravne kandidate, kjer podobno kot v farmacevtskem sektorju obstaja potreba po transparentnem, razložljivem in preverljivem mehanizmu zaupanja. Razširitev arhitekture na te domene bi omogočila oceno univerzalnosti predstavljenega koncepta in njegovo prilagodljivost različnim operativnim zahtevam. S tem bi lahko nastal generičen okvir za vzpostavitev digitalnih ekosistemov zaupanja, ki bi prispevali k varnejšemu in bolj transparentnemu razvoju porazdeljenih sistemov v digitalni družbi.

Literatura

- [1] Alfarez Abdul-Rahman in Stephen Hailes. A distributed trust model. V *Proceedings of the 1997 workshop on New security paradigms*, str. 48–60, 1998.
- [2] John Adler, Ryan Berryhill, Andreas Veneris, Zissis Poulos, Neil Veira, in Anastasia Kastania. Astraea: A decentralized blockchain oracle. V *2018 IEEE international conference on internet of things (IThings) and IEEE green computing and communications (GreenCom) and IEEE cyber, physical and social computing (CPSCoM) and IEEE smart data (SmartData)*, str. 1145–1152. IEEE, 2018.
- [3] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, in dr. Hyperledger fabric: a distributed operating system for permissioned blockchains. V *Proceedings of the thirteenth EuroSys conference*, str. 1–15, 2018.
- [4] Donovan Artz in Yolanda Gil. A survey of trust in computer science and the semantic web. *Journal of Web Semantics*, 5(2):58–71, 2007.
- [5] Bitcoin Wiki. Secp256k1, 2025. URL <https://en.bitcoin.it/wiki/Secp256k1>. (pridobljeno 21. 1. 2026).
- [6] Susan D. Boon in John G. Holmes. The dynamics of interpersonal trust: Resolving uncertainty in the face of risk. *Cooperation and Prosocial Behavior*, str. 167–182, 1991.

-
- [7] Giulio Caldarelli. Understanding the blockchain oracle problem: A call for action. *Information*, 11(11):509, 2020.
 - [8] Giulio Caldarelli. Overview of blockchain oracle research. *Future Internet*, 14(6):175, 2022.
 - [9] Jin-Hee Cho, Kevin Chan, in Sibel Adali. A survey on trust modeling. *ACM Computing Surveys (CSUR)*, 48(2):1–40, 2015.
 - [10] World Wide Web Consortium in dr. Owl 2 web ontology language document overview. 2012.
 - [11] Council of European Union. Directive 2011/62/eu of the european parliament and of the council of 8 june 2011 amending directive 2001/83/ec on the community code relating to medicinal products for human use, as regards the prevention of the entry into the legal supply chain of falsified medicinal products text with eea relevance, 2011. URL <https://eur-lex.europa.eu/eli/dir/2011/62/oj/eng>. (pridobljeno 10. 11. 2025).
 - [12] Council of European Union. Commission delegated regulation (eu) 2016/161 of 2 october 2015 supplementing directive 2001/83/ec of the european parliament and of the council by laying down detailed rules for the safety features appearing on the packaging of medicinal products for human use (text with eea relevance), 2015. URL https://eur-lex.europa.eu/eli/reg_del/2016/161/oj/eng. (pridobljeno 10. 11. 2025).
 - [13] Michael Crosby, Pradan Pattanayak, Sanjeev Verma, Vignesh Kalyanaraman, in dr. Blockchain technology: Beyond bitcoin. *Applied innovation*, 2(6-10):71, 2016.
 - [14] Richard Cyganiak in Paul Reynolds. Rdf 1.1 concepts and abstract syntax. W3C Recommendation, 2014. URL <https://www.w3.org/TR/rdf11-concepts/>. (pridobljeno 10. 11. 2025).

-
- [15] Richard Cyganiak in Paul Reynolds. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation) (text with eea relevance), May 2016. URL <http://data.europa.eu/eli/reg/2016/679/oj>.
 - [16] Dorothy E. Denning. A new paradigm for trusted systems. *Proceedings of the 1993 New Security Paradigms Workshop*, str. 36–41, 1993.
 - [17] Michael Elkins, David Del Torto, Raph Levien, in Thomas Roessler. Mime security with openpgp. Technical report, 2001.
 - [18] Laurent Eschenauer, Virgil D Gligor, in John Baras. On trust establishment in mobile ad-hoc networks. V *International workshop on security protocols*, str. 47–66. Springer, 2002.
 - [19] European Medicines Agency. EudraLex - volume 4: EU guidelines for good manufacturing practice for medicinal products for human and veterinary use. Regulatory guideline, European Medicines Agency, London, United Kingdom, 2013. URL https://health.ec.europa.eu/medicinal-products/eudralex/eudralex-volume-4_en. Accessed: 2026-01-21.
 - [20] Diego Gambetta. *Trust: Making and breaking cooperative relations*. Basil Blackwell, 1988.
 - [21] Tyrone Grandison in Morris Sloman. A survey of trust in internet applications. *IEEE Communications Surveys & Tutorials*, 3(4):2–16, 2000.
 - [22] Russell Housley, Warwick Ford, W Polk, in David Solo. Rfc2459: Internet x. 509 public key infrastructure certificate and crl profile, 1999.
 - [23] Michael Howard in Steve Lipner. *The security development lifecycle*, volume 8. Microsoft Press Redmond, 2006.

-
- [24] Audun Josang in Roslan Ismail. The beta reputation system. V *Proceedings of the 15th bled electronic commerce conference*, volume 5, str. 2502–2511, 2002.
- [25] S. Karthik, R. Shankar, in N. Arunkumar. Ontology-based trust model for pervasive computing environments. *Journal of Ambient Intelligence and Humanized Computing*, 8:557–568, 2017. doi: 10.1007/s12652-016-0442-y.
- [26] Halid Kayhan. Ensuring trust in pharmaceutical supply chains by data protection by design approach to blockchains. *Blockchain in Healthcare Today*, 5:10–30953, 2022.
- [27] Petar Kochovski, Sandi Gec, Vlado Stankovski, Marko Bajec, in Pavel D Drobintsev. Trust management in a blockchain based fog computing platform with trustless smart oracles. *Future Generation Computer Systems*, 101:747–759, 2019.
- [28] Petar Kochovski, Maroua Masmoudi, Redouane Bouhamoum, Vlado Stankovski, Hajer Baazaoui, Chirine Ghedira, Dan Vodislav, in Thamer Mecharnia. Drug traceability system based on semantic blockchain and on a reputation method. *World Wide Web*, 27(5):62, 2024.
- [29] Ora Lassila, James Hendler, in Tim Berners-Lee. The semantic web. *Scientific american*, 284(5):34–43, 2001.
- [30] Huaizhi Li in Mukesh Singhal. Trust management in distributed systems. *Computer*, 40(2):45–53, 2007. doi: 10.1109/MC.2007.76.
- [31] Kwei-Jay Lin, Haiyin Lu, Tao Yu, in Chia-en Tai. A reputation and trust management broker framework for web applications. V *2005 IEEE International Conference on e-Technology, e-Commerce and e-Service*, str. 262–269. IEEE, 2005.

-
- [32] Roger C. Mayer, James H. Davis, in F. David Schoorman. An integrative model of organizational trust. *Academy of Management Review*, 20(3): 709–734, 1995.
- [33] Carlo Mazzocca, Abbas Acar, Selcuk Uluagac, Rebecca Montanari, Paolo Bellavista, in Mauro Conti. A survey on decentralized identifiers and verifiable credentials. *IEEE Communications Surveys & Tutorials*, 2025.
- [34] D. Harrison McKnight in Norman L. Chervany. Trust and distrust definitions: One bite at a time. *Trust in Cyber-societies*, 2246:27–54, 2001.
- [35] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Available at SSRN 3440802*, 2008.
- [36] Joseph D Nally. *Good manufacturing practices for pharmaceuticals*. CRC Press, 2016.
- [37] Sandeep Kumar Panda in Suresh Chandra Satapathy. Drug traceability and transparency in medical supply chain using blockchain for easing the process and creating trust between stakeholders and consumers. *Personal and Ubiquitous Computing*, 28(1):75–91, 2024.
- [38] Žan Pižmoht. Trust management (izvorna koda), 2025. URL <https://github.com/zanp98/trust-management>. (pridobljeno: 24. 2. 2026).
- [39] Drummond Reed, Manu Sporny, Dave Longley, Christopher Allen, Ryan Grant, Markus Sabadello, in Jonathan Holt. Decentralized identifiers (dids) v1. 0. *Draft Community Group Report*, 2020.
- [40] Paul Resnick, Ko Kuwabara, Richard Zeckhauser, in Eric Friedman. Reputation systems. *Communications of the ACM*, 43(12):45–48, 2000.
- [41] Manu Sporny, Dave Longley, David Chadwick, in dr. Verifiable credentials data model 2.0. W3C Recommendation, 2025. URL <https://www.w3.org/TR/vc-data-model-2.0/>. (pridobljeno 10. 11. 2025).

-
- [42] Mueen Uddin, Khaled Salah, Raja Jayaraman, Sasa Pesic, in Samer Ellahham. Blockchain for drug traceability: Architectures and open challenges. *Health informatics journal*, 27(2):14604582211011228, 2021.
- [43] U.S. Food and Drug Administration. Drug supply chain security act (dscsa), 2017. URL <https://www.fda.gov/Drugs/DrugSafety/DrugIntegrityandSupplyChainSecurity/DrugSupplyChainSecurityAct/ucm382022.htm>. (pridobljeno 10. 11. 2025).
- [44] Gavin Wood in dr. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- [45] World Health Organization. Good distribution practices for pharmaceutical products. *Technical Report Series*, (957), 2010.
- [46] World Health Organization. A study on the public health and socio-economic impact of substandard and falsified medical products: executive summary, 2017.
- [47] World Health Organization. WHO global surveillance and monitoring system for substandard and falsified medical products, 2017.
- [48] Li Xiong in Ling Liu. Building trust in decentralized peer-to-peer electronic communities. V *Fifth International Conference on Electronic Commerce Research (ICECR-5)*. ACM Press New York, 2002.
- [49] Zheng Yan in Silke Holtmanns. Trust modeling and management: from social trust to digital trust. V *Computer security, privacy and politics: current issues, challenges and solutions*, str. 290–323. IGI Global Scientific Publishing, 2008.